

计算机体系结构

新型计算机研究所

陈 衡

西一楼 A 段415室

Email: hengchen@xjtu.edu.cn

QQ群号: 1031141115

第7章 存储系统

- ◆ 7.1 存储系统的层次结构
- ◆ 7.2 Cache基本知识
- ◆ 7.3 降低Cache不命中率
- ◆ 7.4 减少Cache不命中开销
- ◆ 7.5 减少命中时间
- ◆ 7.6 并行主存系统
- ◆ 7.7 虚拟存储器

7.1 存储系统的层次结构

- ◆ 7.1.1 存储系统的层次结构概述
- ◆ 计算机系统结构设计中关键的问题之一：
 - 如何以合理的价格，设计容量和速度都满足计算机系统要求的存储器系统？
- ◆ 人们对这三个指标的要求
 - **容量大、速度快、价格低**
- ◆ 三个要求是相互矛盾的
 - 速度越快，每位价格就越高；
 - 容量越大，每位价格就越低；
 - 容量越大，速度越慢。
- ◆ 解决方法：
 - **采用多种存储器技术，构成多级存储层次结构。**

7.1.1 存储系统的层次结构

Capacity
Access Time Cost

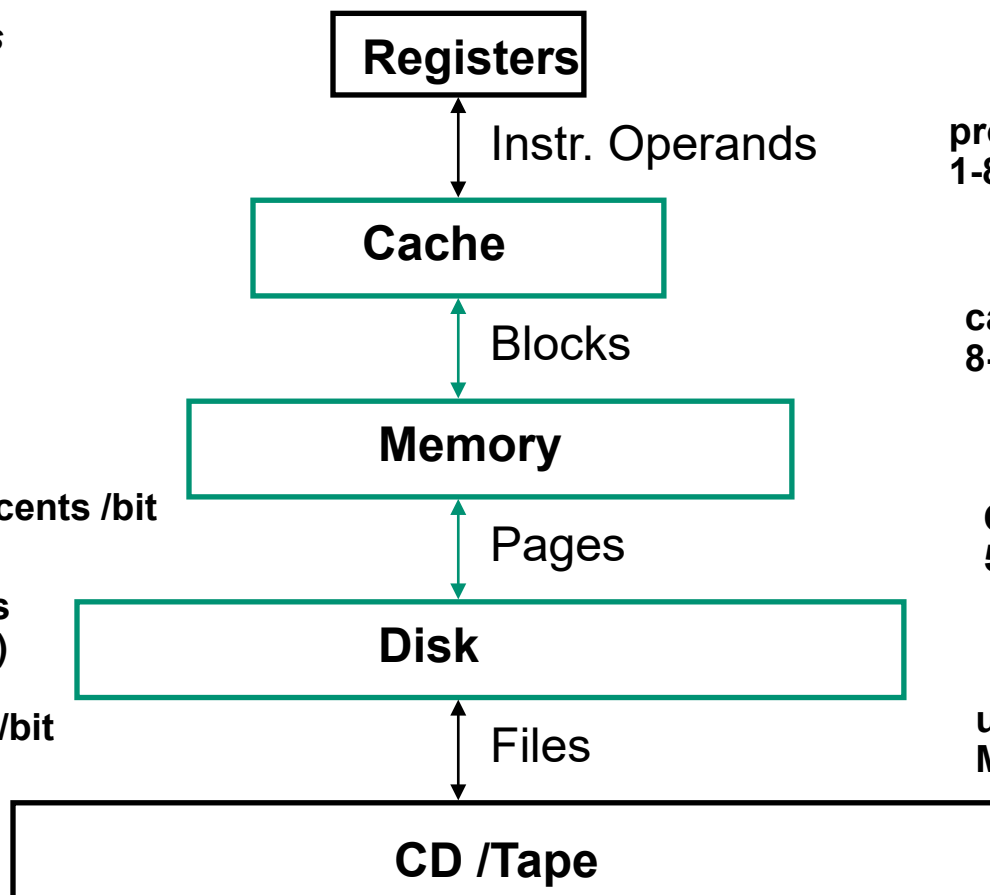
CPU Registers
100s Bytes
1 ns

Cache
K Bytes
4 ns
1-0.1 cents/bit

Main Memory
M Bytes
100ns- 300ns
\$.0001-.00001 cents /bit

Disk
G Bytes, 10 ms
(10,000,000 ns)
-5 -6
10 - 10 cents/bit

Tape
infinite
sec-min
10⁻⁸



Upper Level

*Staging
Xfer Unit*

prog./compiler
1-8 bytes

cache cntl
8-128 bytes

OS
512-4K bytes

user/operator
Mbytes

faster

Larger

Lower Level

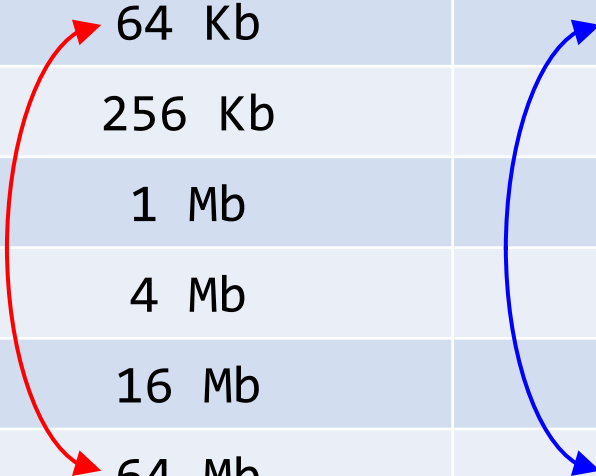
7.1.1 存储系统的层次结构

- ◆ 为什么要设计多级存储层次结构？
 - 1、**处理器与主存之间的性能差异越来越大**
 - CPU性能提高大约 60%/year
 - DRAM 性能提高大约 9%/year
 - 2、**程序访问的局部性原理**：对于绝大多数程序来说，程序所访问的指令和数据在地址上不是均匀分布的，而是相对簇聚的。
 - 程序访问的局部性包含两个方面
 - **时间局部性**：程序马上将要用到的信息很可能就是现在正在使用的信息。
 - **空间局部性**：程序马上将要用到的信息很可能与现在正在使用的信息在存储空间上是相邻的。

内存技术发展趋势

	Capacity	Speed (latency)
Logic:	2x in 3 years	2x in 3 years
DRAM:	4x in 3 years	2x in 10 years
Disk:	4x in 3 years	2x in 10 years

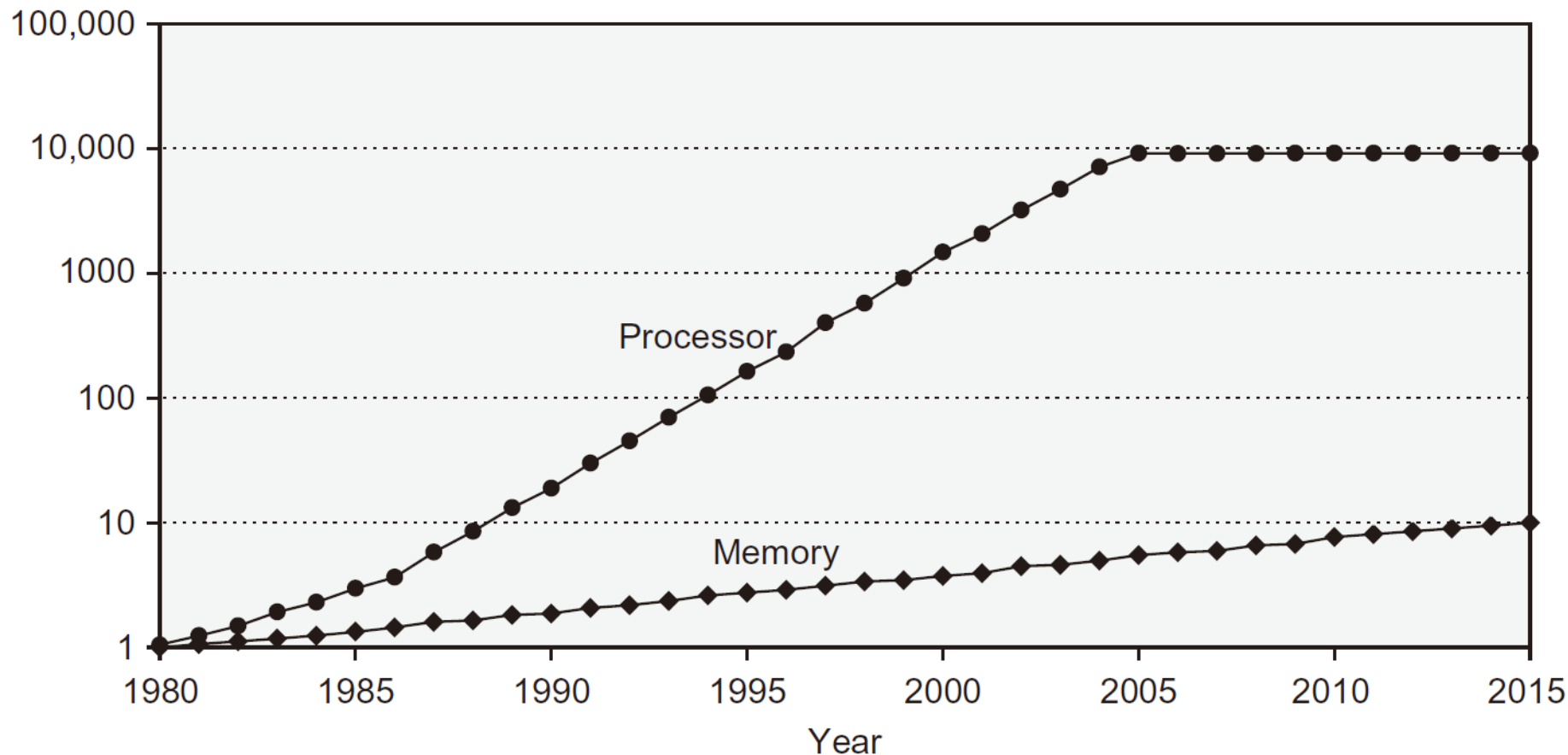
Year	DRAM Size	Cycle Time
1980	64 Kb	250 ns
1983	256 Kb	220 ns
1986	1 Mb	190 ns
1989	4 Mb	165 ns
1992	16 Mb	145 ns
1995	64 Mb	120 ns



内存技术发展趋势

Year Produced	Chip Size	DRAM Type	Bus Cycle	Latency New Request
1980	64 Kbit	Asynchronous DRAM		250 ns
1983	256 Kbit	Asynchronous DRAM		190 ns
1986	1 Mbit	Asynchronous DRAM		140 ns
1989	4 Mbit	Asynchronous DRAM		110 ns
1992	16 Mbit	Asynchronous DRAM		90 ns
1996	64 Mbit	SDRAM	10 ns	70 ns
1998	128 Mbit	SDRAM	7.5 ns	60 ns
2000	256 Mbit	DDR SDRAM	6 ns	55 ns
2002	512 Mbit	DDR SDRAM	5 ns	55 ns
2004	1 Gbit	DDR2 SDRAM	3 ns	50 ns
2006	2 Gbit	DDR2 SDRAM	2 ns	45 ns
2010	4 Gbit	DDR3 SDRAM	1.5 ns	37 ns
2012	8 Gbit	DDR3 SDRAM	1 ns	31 ns

微处理器与DRAM的性能差异

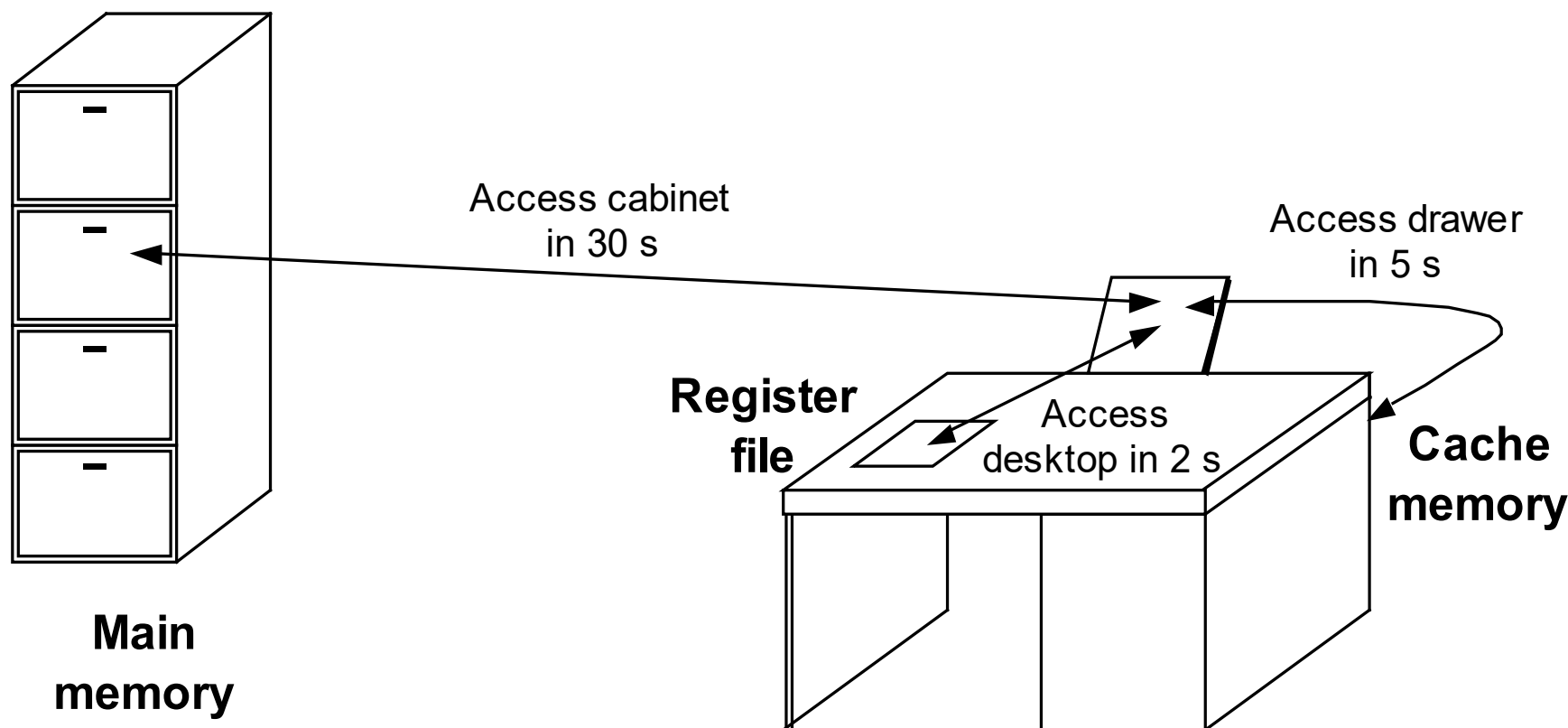


利用Cache缓解微处理器与存储器性能上的差异

程序访问的局部性

◆ 桌面，抽屉和文件柜类比

- 与文件柜（**主存储器**）中的项目相比，抽屉（**缓存**）或桌面（**寄存器**）中的项目更易于访问。



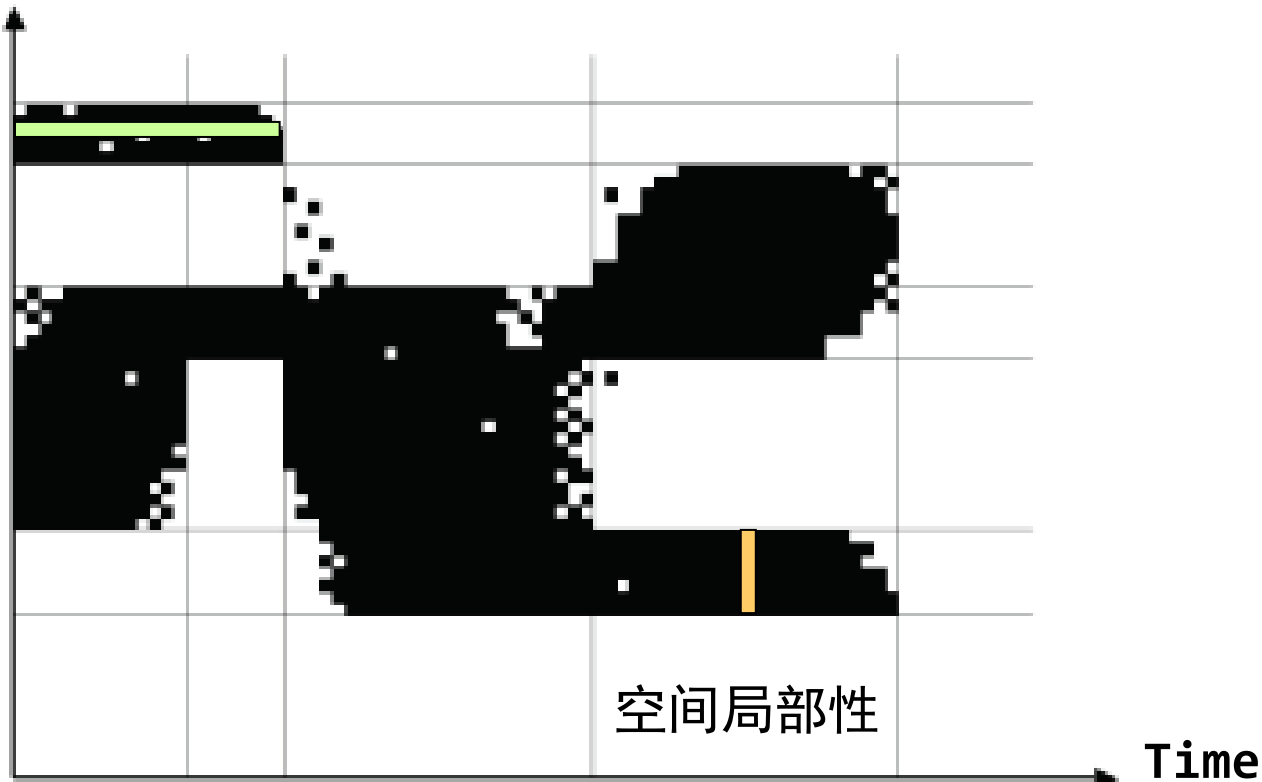
程序访问的局部性

◆ 局部性原则

- 2005年7月
- Peter Denning's CACM (COMMUNICATIONS OF THE ACM)
- 程序的局部性在许多应用程序中计算的基本原则。

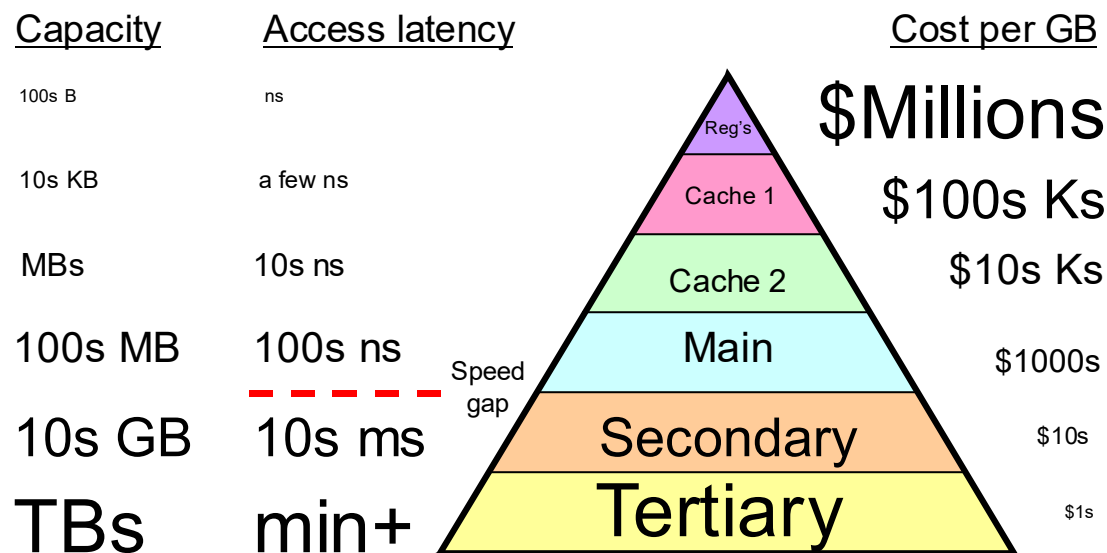
Addresses

时间
局部性



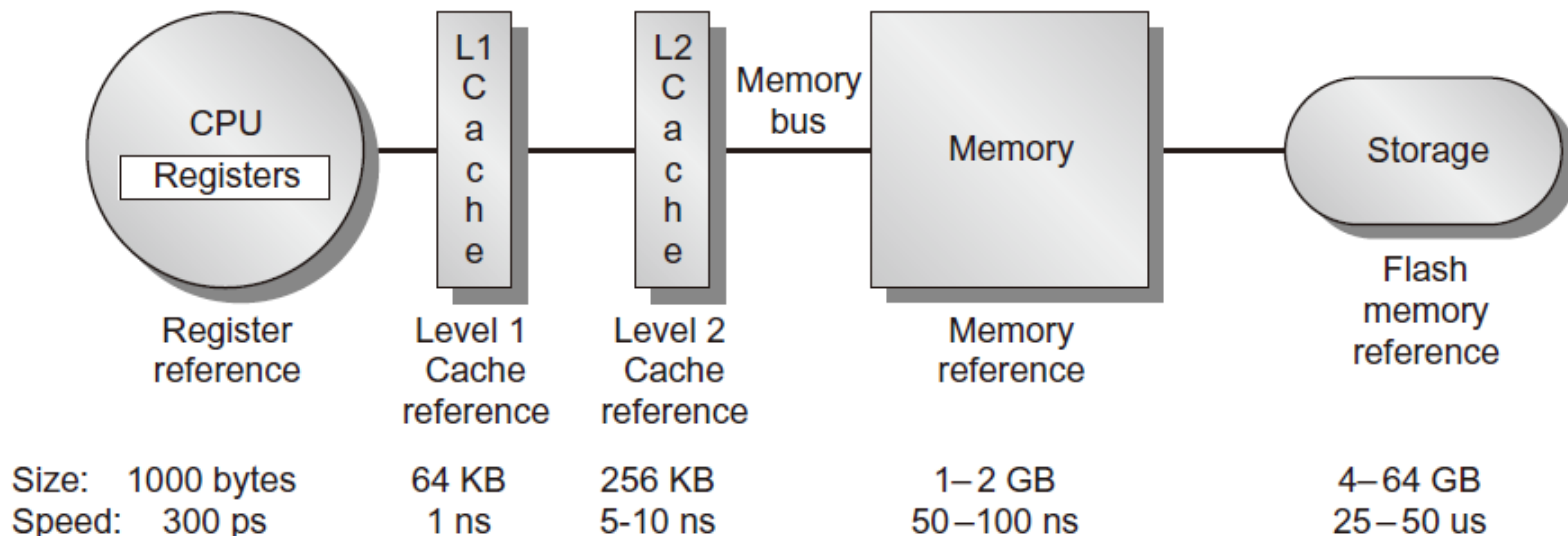
7.1.1 存储系统的层次结构

- ◆ **局部性原理+较小的内存更快**导致基于不同速度和大小的内存的**层次结构**。
- ◆ 设计目标
 - 通过优化存储系统的组织来使得**针对典型应用平均访存时间最短**

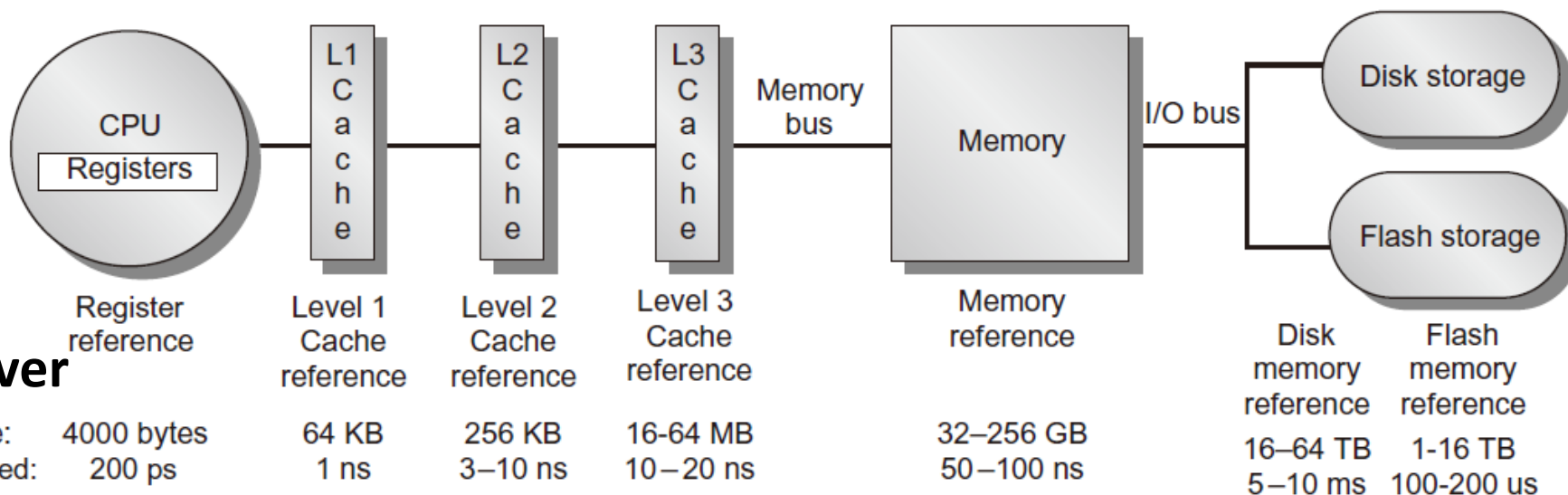


存储多级层次结构

PMD



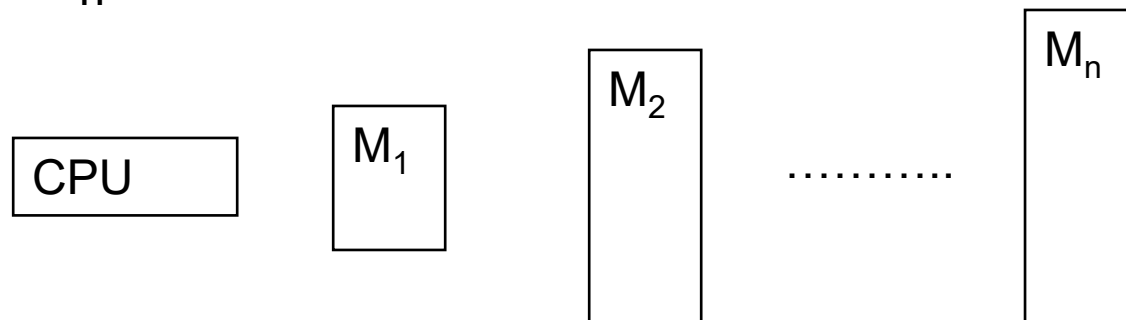
Server



基本解决方法：多级层次结构

◆ 多级分层结构

- M_1 速度最快，容量最小，每位价格最高
- M_n 速度最慢，容量最大，每位价格最低



□ 访问时间：

$$T_1 < T_2 < \dots < T_n$$

□ 容量：

$$S_1 < S_2 < \dots < S_n$$

□ 平均每位价格：

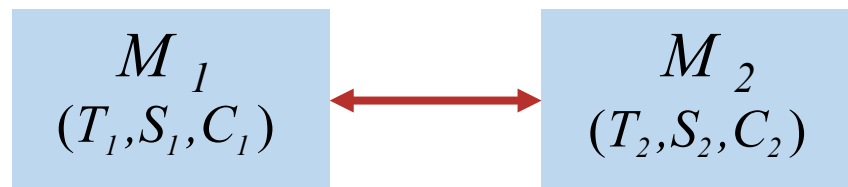
$$C_1 > C_2 > \dots > C_n$$

◆ 整个存储系统要达到的目标：

- 从CPU来看，该**存储系统的速度接近于 M_1** 的，而**容量和每位价格都接近于 M_n** 的。
- 存储器越靠近CPU，则CPU对它的访问频度越高，而且**最好大多数的访问都能在 M_1 完成**。

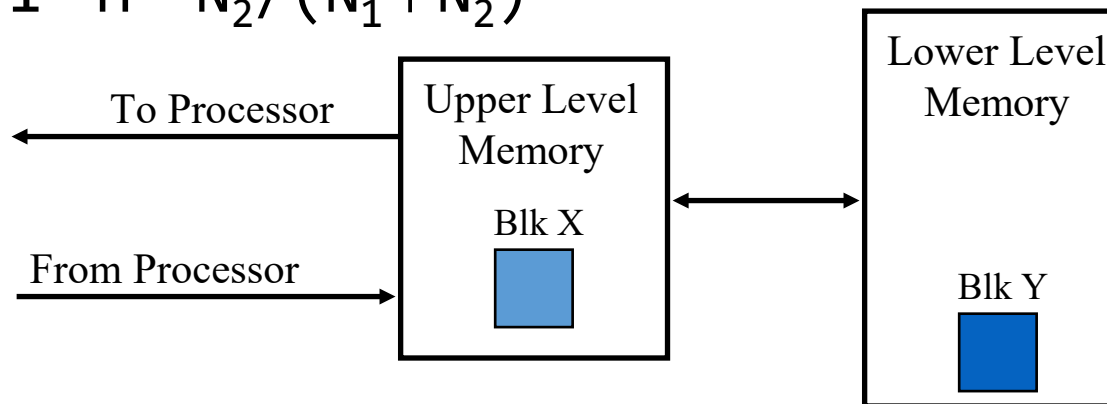
7.1.2 存储系统的性能参数

- ◆ 下面仅考虑由 M_1 和 M_2 构成的两级存储层次：
 - M_1 的参数： S_1 , T_1 , C_1
 - M_2 的参数： S_2 , T_2 , C_2
- ◆ 1、**存储容量S**
 - 一般来说，整个存储系统的容量即是第二级存储器 M_2 的容量，即 $S=S_2$ 。
- ◆ 2、**每位价格C**
$$C = \frac{C_1 S_1 + C_2 S_2}{S_1 + S_2}$$
 - 当 $S_1 \ll S_2$ 时， $C \approx C_2$



7.1.2 存储系统的性能参数

- ◆ 3、命中率H和不命中率F
 - **命中**：访问的块在存储系统的较高层次上
 - **命中率**：CPU访问存储系统时，在 M_1 中找到所需信息的概率。 $H = N_1 / (N_1 + N_2)$
 - N_1 — 访问 M_1 的次数
 - N_2 — 访问 M_2 的次数
 - **失效（不命中）**：访问的块不在存储系统的较高层次上
 - **不命中率**： $F = 1 - H = N_2 / (N_1 + N_2)$



7.1.2 存储系统的性能参数

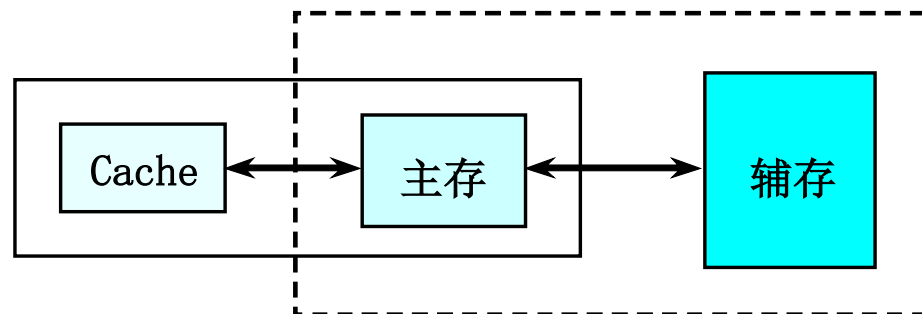
◆ 4、平均访问时间 T_A

- 当命中时，访问时间即为 T_1 （命中时间）
- 当不命中时，情况比较复杂。
 - 不命中时的访问时间为： $T_2 + T_B + T_1 = T_1 + T_M$
其中： $T_M = T_2 + T_B$
 - **不命中开销 T_M** ：从向 M_2 发出访问请求到把整个数据块调入 M_1 中所需的时间。
 - 传送一个信息块所需的时间为 T_B 。
- $T_A = HT_1 + (1-H)(T_1 + T_M)$
 $= T_1 + (1-H)T_M$
- 或 $T_A = T_1 + FT_M$

7.1.3 三级存储系统

◆ 三级存储系统

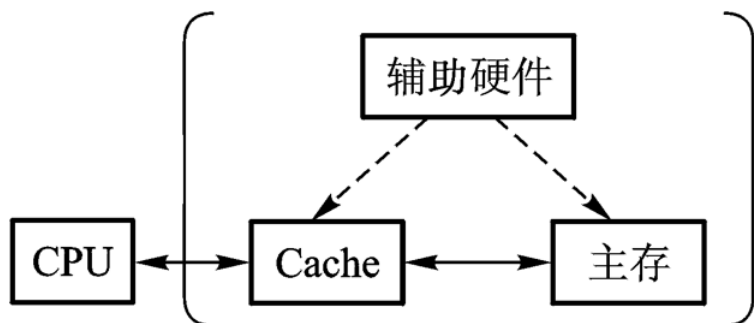
- Cache（高速缓冲存储器）
- 主存储器
- 磁盘存储器（辅存）



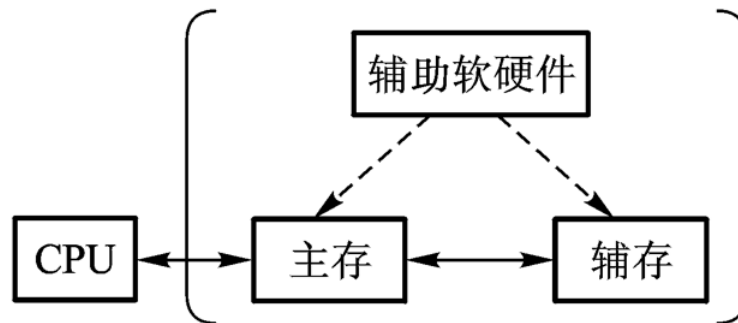
◆ 可以看成是由“Cache—主存”层次和“主存—辅存”层次构成的系统。

◆ 从主存的角度来看

- “Cache—主存”层次：弥补主存速度的不足
- “主存—辅存”层次：弥补主存容量的不足



(a) “Cache—主存”层次



(b) “主存—辅存”层次

7.1.4 存储层次的四个问题

- ◆ 当把一个块调入高一层(靠近CPU)存储器时, 可以放在哪些位置上?
 - 映像规则
- ◆ 当所要访问的块在高一层存储器中时, 如何找到该块?
 - 查找算法
- ◆ 当发生不命中时, 应替换哪一块?
 - 替换算法
- ◆ 当进行写访问时, 应进行哪些操作?
 - 写策略

7.2 Cache基本知识

- ◆ 7.2.1 基本结构和原理
- ◆ 小而快（SRAM）的存储技术
 - 存储正在访问的部分指令和数据
- ◆ Cache无处不在
 - 寄存器：Cache on variables
 - 一、二级Cache：Cache on memory
 - Memory：Cache on hard disks
 - 存储最近执行的程序和数据
 - Hard disks可以视为主存的扩展（VM）
 - 分支目标缓存及分支预测缓存
 - 缓存分支目标及预测信息

Cache和主存分块

◆ Cache主要目标:

- 提高访存速度
- 降低存储系统成本

◆ 用于减少平均访存时间

- 通过保持最近访问的数据在处理器附近来挖掘时间局部性
- 通过以块为单位在不同层次移动数据来挖掘空间局部性

◆ Cache是按块进行管理的。Cache和主存均被分割成大小相同的块。信息以块为单位调入Cache。

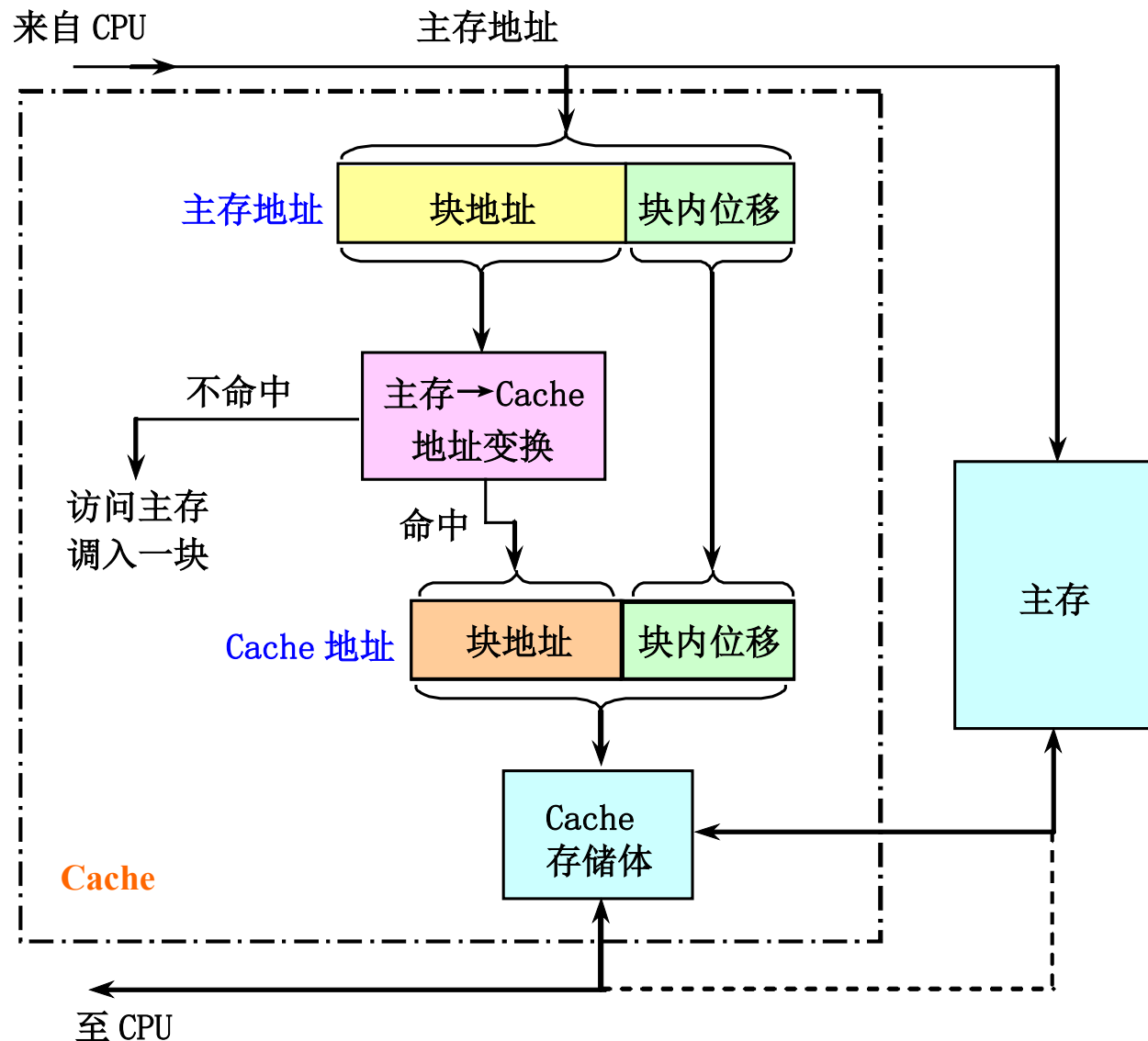
- 主存块地址（块号）用于查找该块在Cache中的位置。
- 块内位移用于确定所访问的数据在该块中的位置。

主存地址:

块地址

块内位移

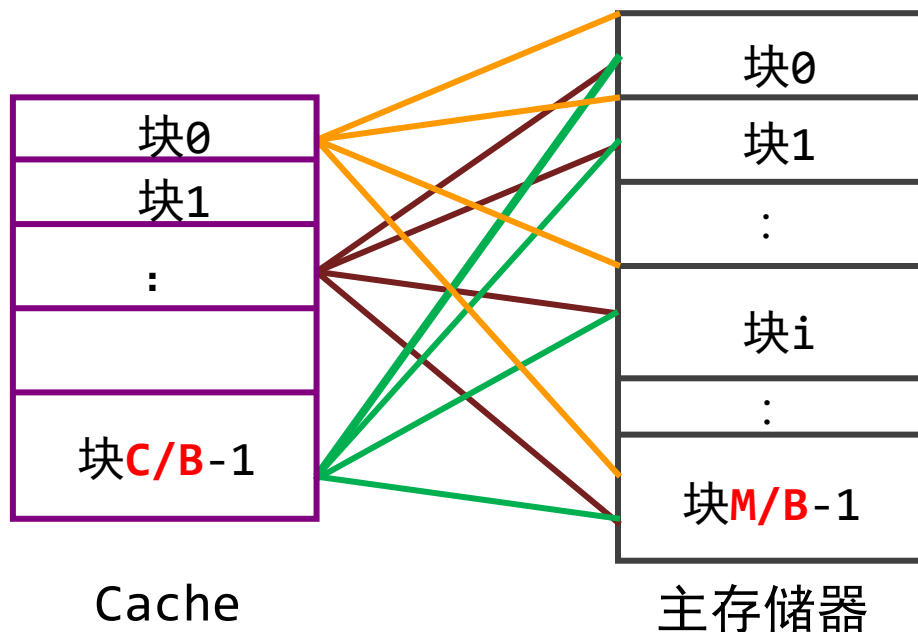
Cache的基本工作原理示意图



7.2.2 映像规则

◆ 1、全相联映像

- 全相联：主存中的任一块可以被放置到Cache中的任意一个位置。
- 对比：阅览室位置 —— 随便坐
- 特点：空间利用率最高，冲突概率最低，实现最复杂。



B: 每块大小

C: Cache容量

M: 主存容量

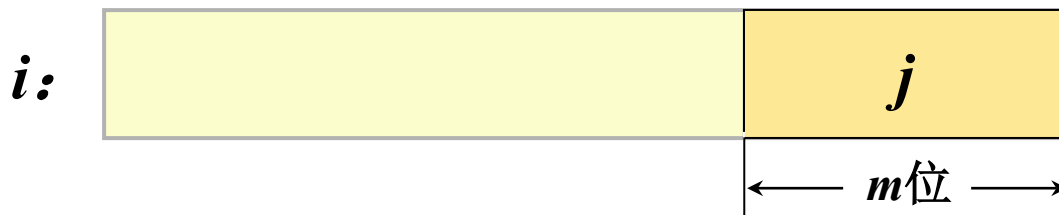
7.2.2 映像规则

◆ 2、直接映像

- 直接映像：主存中的每一块只能被放置到Cache中唯一的一个位置。
- 对比：阅览室位置 — 只有一个位置可以坐
- 特点：空间利用率最低，冲突概率最高，实现最简单。
- 对于主存的第*i*块，若它映像到Cache的第*j*块，则：

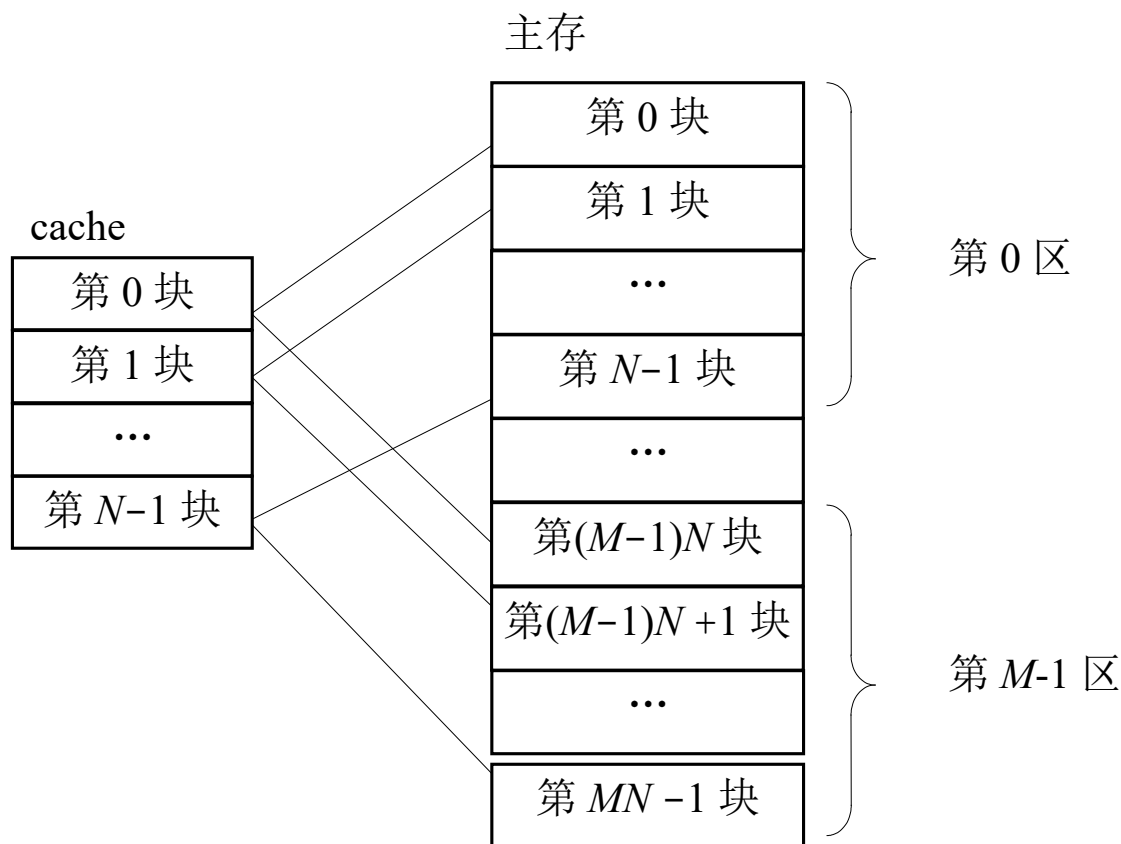
$$j = i \bmod (M) \quad (M \text{ 为 Cache 的块数})$$

- 设 $M=2^m$ ，则当表示为二进制数时，*j* 实际上就是 *i* 的低 *m* 位



7.2.2 映像规则

- Cache和RAM仍以块为单位，但在RAM中将空间划分为大小和Cache完全一样的区，在RAM中，将区再划分为若干块
- Cache中的块和RAM中的块形成**一对多**的映射关系



7.2.2 映像规则

◆ 3、组相联映像

- 组相联：主存中的每一块可以被放置到Cache中唯一的一个组中的任何一个位置。
- 组相联是直接映像和全相联的一种折中
 - 全相联方式速度慢，造价较高，但命中率最高
 - 直接相联方式速度快，但命中率低。

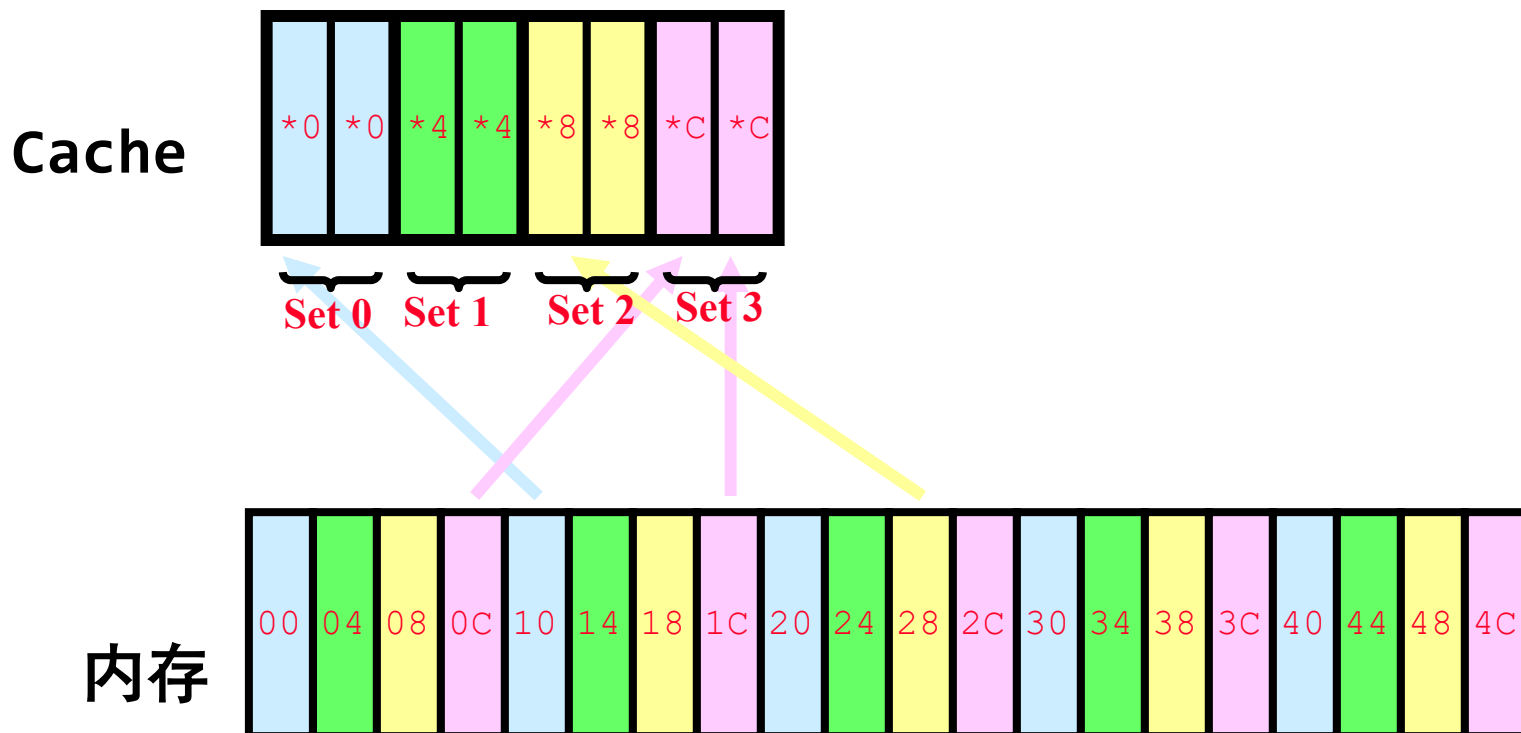
◆ 组的选择常采用位选择算法

- 若主存第*i*块映像到第*k*组，则：
 $k = i \bmod (G)$ （*G*为Cache的组数）
- 设 $G = 2^g$ ，则当表示为二进制数时，*k*实际上就是*i*的低*g*位
- 低*g*位以及直接映像中的低*m*位通常称为索引。



7.2.2 映像规则

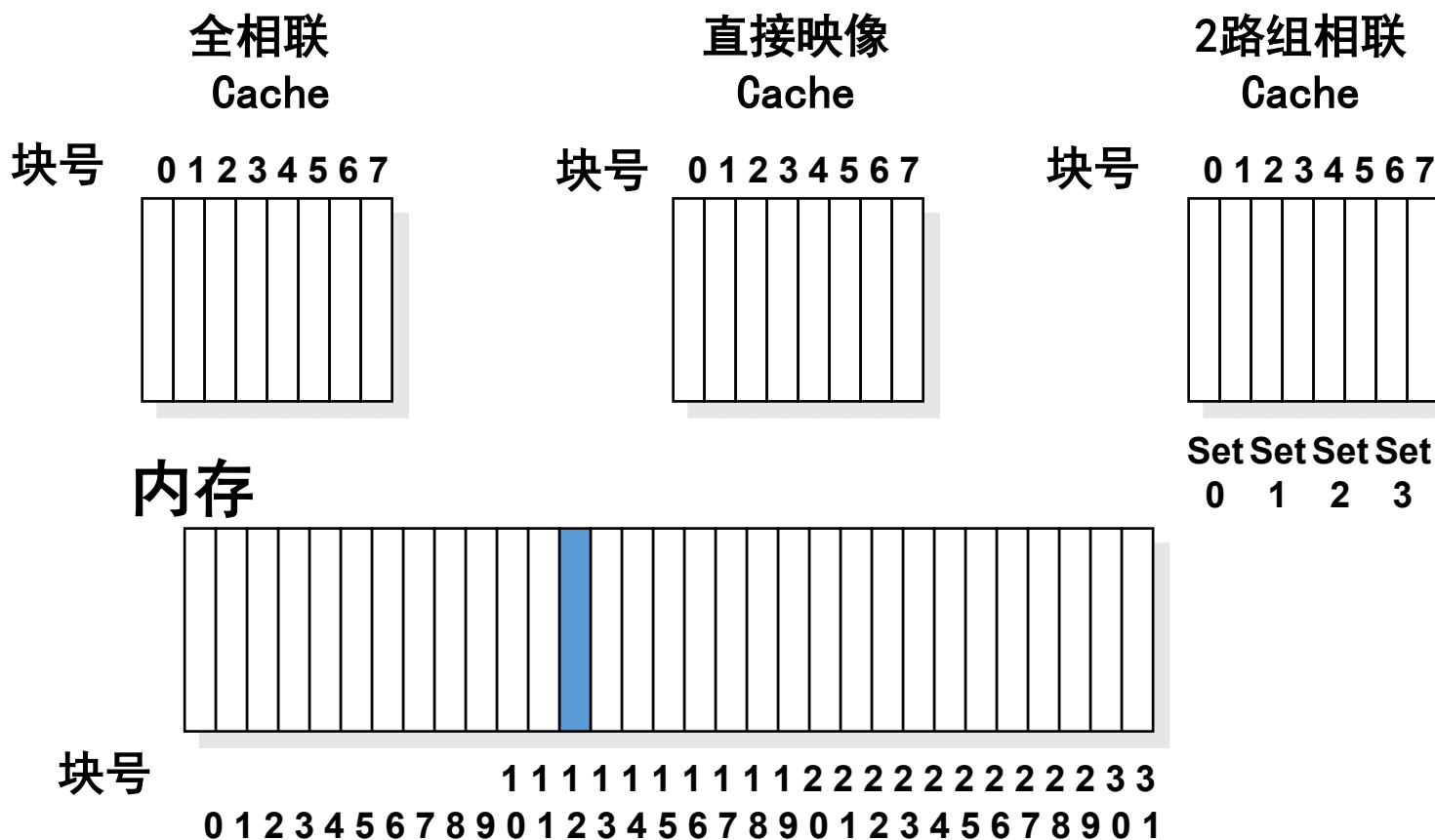
- ◆ **n路组相联**：每组中有n个块($n=M/G$)。
 - n称为相联度。
 - 相联度越高，Cache空间的利用率就越高，块冲突概率就越低，不命中率也就越低。



- ◆ **要点：组间直接，组内全相联**

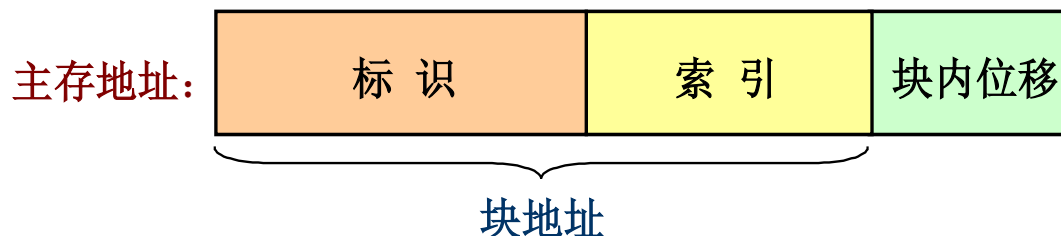
映像规则举例

- ◆ 内存中的第12块应该放置在Cache中的哪个位置？
 - 全相联，直接映像，2路组相联



7.2.3 查找方法

- ◆ 当CPU访问Cache时，如何确定Cache中是否有所要访问的块？
- ◆ 若有的话，如何确定其位置？
- ◆ 通过查找**目录表**来实现
- ◆ **目录表的结构**
 - 主存块的块地址的高位部分，称为**标识**。
 - 每个主存块能唯一地由其标识来确定
- ◆ 只需查找候选位置所对应的目录表项



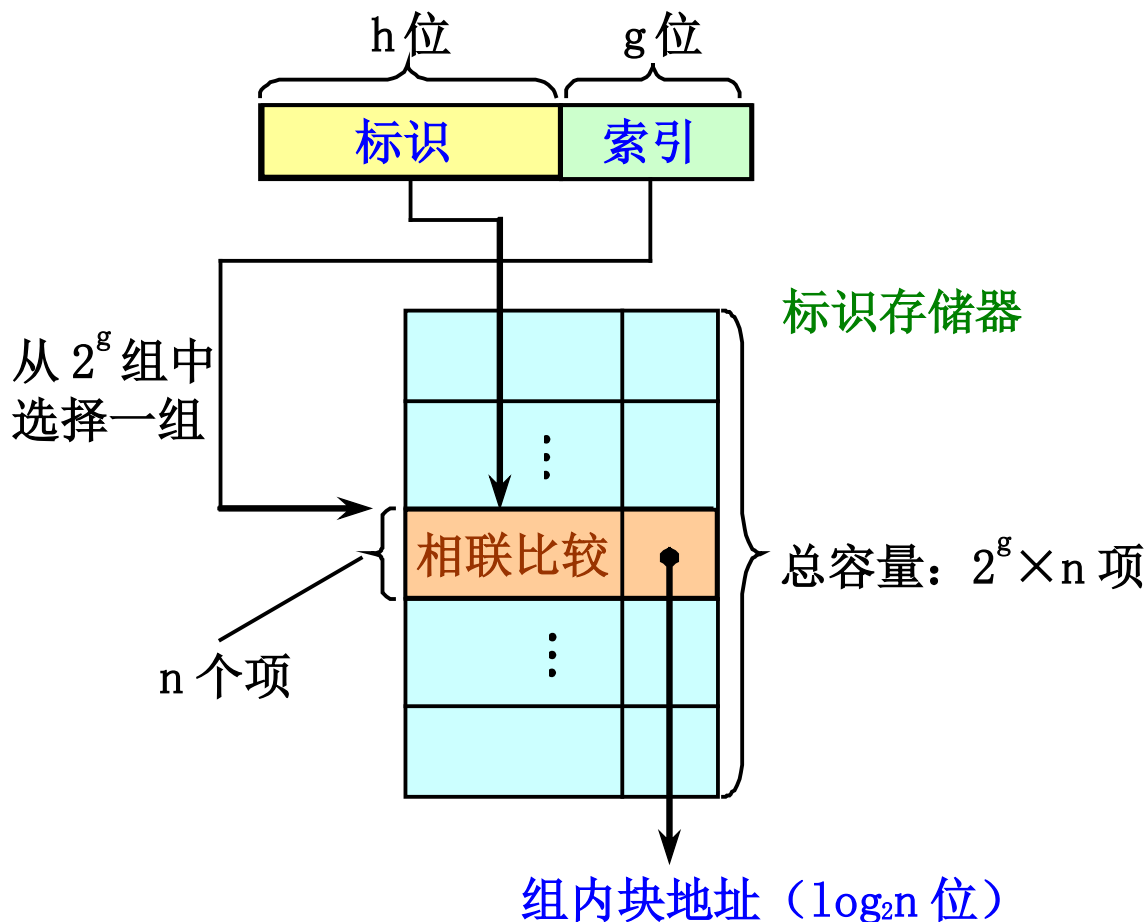
7.2.3 查找方法

- ◆ 原则：**所有可能的标记并行查找**，cache的速度至关重要，即并行查找
- ◆ **并行查找的方法**
 - 用相联存储器实现，按内容检索
 - 用单体多字存储器和比较器实现
- ◆ 显然相联度 N 越大，实现查找的机制就越复杂，代价就越高
- ◆ 无论直接映象还是组相联，查找时，**只需比较标识字段**，索引字段无需参加比较

并行查找的实现—相联存储器

◆ 相联存储器

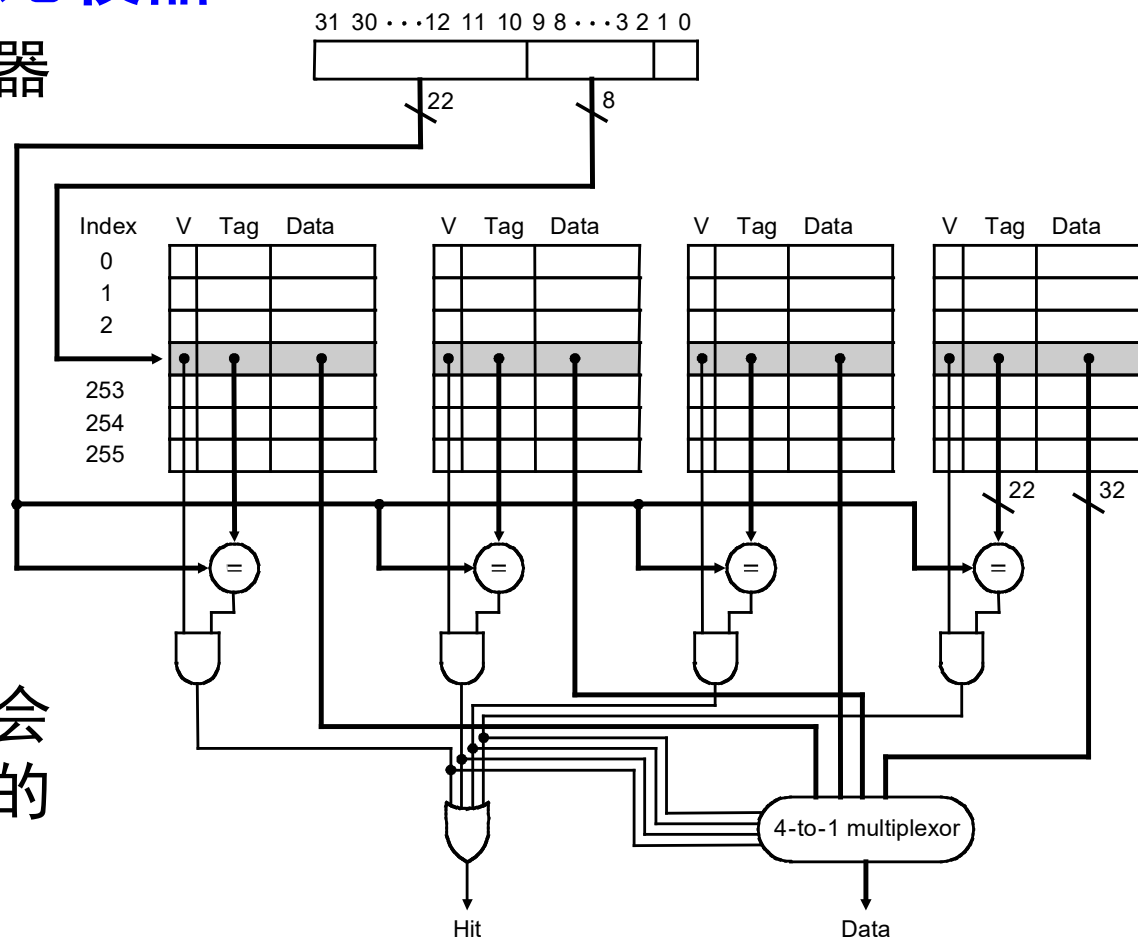
- 目录由 2^g 个相联存储区构成，每个相联存储区的大小为 $n \times (h + \log_2 n)$ 位
- 根据所查找到的组内块地址，从Cache存储体中读出的多个信息字中选一个，发送给CPU。



并行查找的实现—比较器

◆ 单体多字存储器+比较器

- 不必采用相联存储器，而是用**按地址访问的存储器**来实现
- 所需要的硬件为：
大小为 $2^g \times n \times h$ 位的存储器和 n 个 h 位的比较器。
- 当相联度 n 增加时，不仅比较器的个数会增加，而且比较器的位数也会增加。



7.2.4 替换算法

◆ 提出的背景：

- 主存中块数一般比cache中的块多，可能出现该块所对应的一组或一个Cache块已全部被占用的情况，这时需强制腾出其中的某一块，以接纳新调入的块

◆ 替换哪一块，这是替换算法要解决的问题。

- 直接映象，因为只有一块，别无选择
- **组相联和全相联有多种选择**

7.2.4 替换算法

- ◆ 随机法 (Random), 随机选择一块替换
 - 优点: 简单, 易于实现
 - 缺点: 没有考虑Cache块的使用历史, 反映**程序的局部性较差**, 失效率较高
- ◆ FIFO—选择最早调入的块
 - 优点: 简单
 - 虽然利用了同一组中各块进入Cache的顺序, 但还是反映**程序局部性不够**, 因为最先进入的块, 很可能是经常使用的块
- ◆ 最近最少使用法 (LRU) (Least Recently Used)
 - 优点: 较好的利用了程序的局部性, 失效率较低
 - 缺点: 比较复杂, 硬件实现较困难

替换算法失效率比较

Size	Associativity								
	Two-way			Four-way			Eight-way		
	LRU	Random	FIFO	LRU	Random	FIFO	LRU	Random	FIFO
16 KB	114.1	117.3	115.5	111.7	115.1	113.3	109.0	111.8	110.4
64 KB	103.4	104.3	103.9	102.4	102.3	103.1	99.7	100.5	100.3
256 KB	92.2	92.1	92.5	92.1	92.1	92.5	92.1	92.1	92.5

- ◆ Data Cache misses per 1000 instructions
- ◆ the Alpha architecture using 10 SPEC2000 benchmarks
- ◆ 观察结果（失效率）
 - 相联度高，失效率较低。
 - Cache容量较大，失效率较低。
 - LRU在Cache容量较小时，失效率较低
 - 随着Cache容量的加大，Random的失效率在降低

7.2.5 写策略

- ◆ “写”在所有访存操作中所占的比例
 - 统计结果表明，对于一组给定的程序：
 - load指令：26%
 - store指令：9%
 - “写”在所有访存操作中所占的比例： $9\% / (100\% + 26\% + 9\%) \approx 7\%$
 - “写”在访问Cache操作中所占的比例： $9\% / (26\% + 9\%) \approx 25\%$
- ◆ 大概率事件优先原则—优化Cache的读操作
- ◆ Amdahl定律：不可忽视“写”的速度

7.2.5 写策略

◆ “写”的问题

- 读出标识，确认命中后，对Cache写（串行操作）
- “写”访问有可能导致Cache与主存内容的一致性问题的

◆ 写策略就是要解决：何时更新主存问题

- 写策略是区分不同Cache设计方案的一个重要标志。
- 两种写策略

◆ 写直达法（也称为存直达法）

- 执行“写”操作时，不仅写入Cache，而且也写入下一级存储器。

◆ 写回法（也称为拷回法）

- 执行“写”操作时，只写入Cache。仅当Cache中相应的块被替换时，才写回主存。（设置“修改位”）

两种写策略

◆ 写直达法 (Write through)

- 优点：易于实现，容易保持不同层次间的一致性
- 缺点：速度较慢

◆ 写回法

- 优点：速度快，减少访存次数
- 缺点：一致性问题

◆ 当发生写失效时的两种策略

- **按写分配法** (Write allocate)：写失效时，先把所写单元所在块调入Cache，然后再进行写入，也称写时取 (Fetch on Write) 方法
- **不按写分配法** (no-write allocate)：写失效时，直接写入下一级存储器，而不将相应块调入Cache，也称绕写法 (Write around)

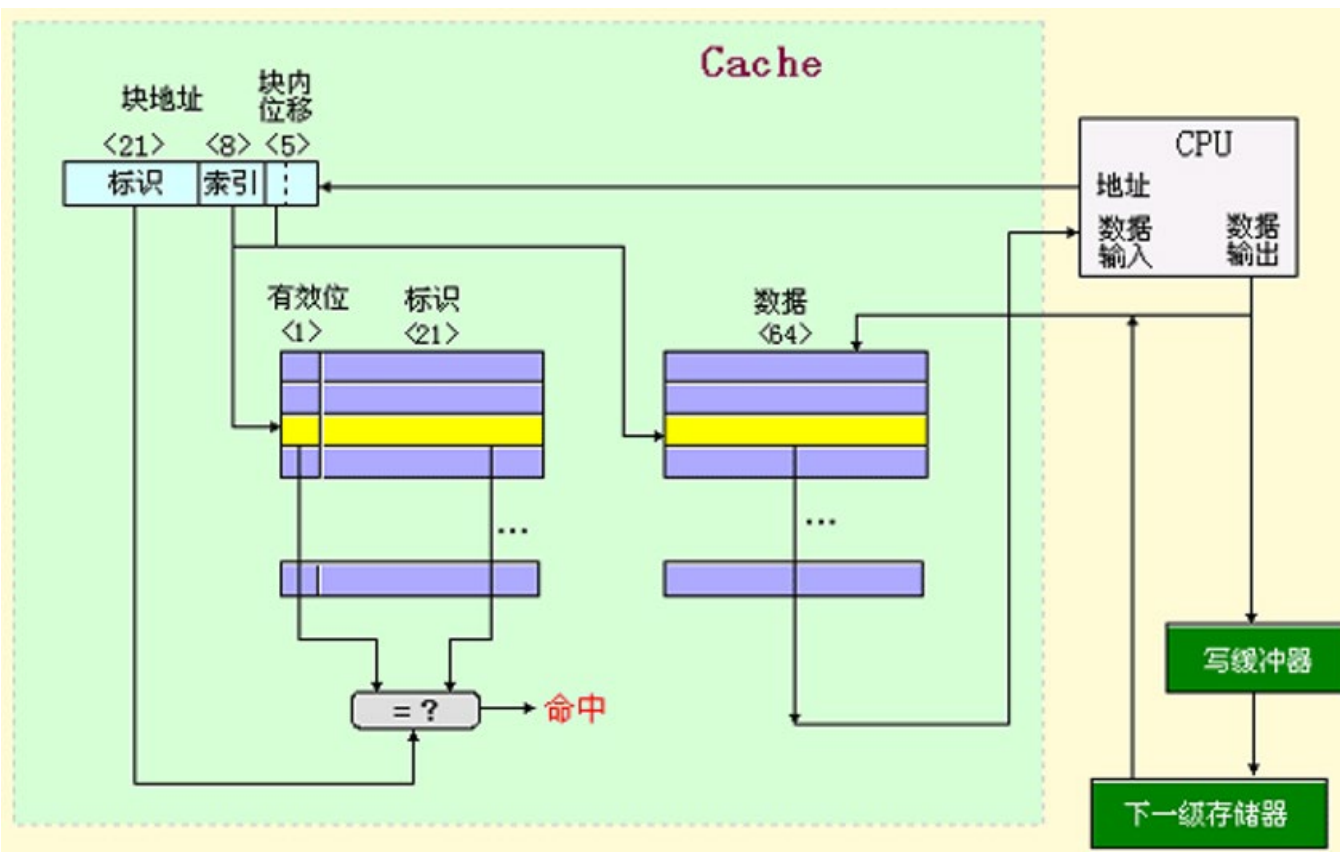
◆ 一般情况下

- **写回法采用按写分配法**
- **写直达法采用不按写分配法**

7.2.6 Cache工作过程举例

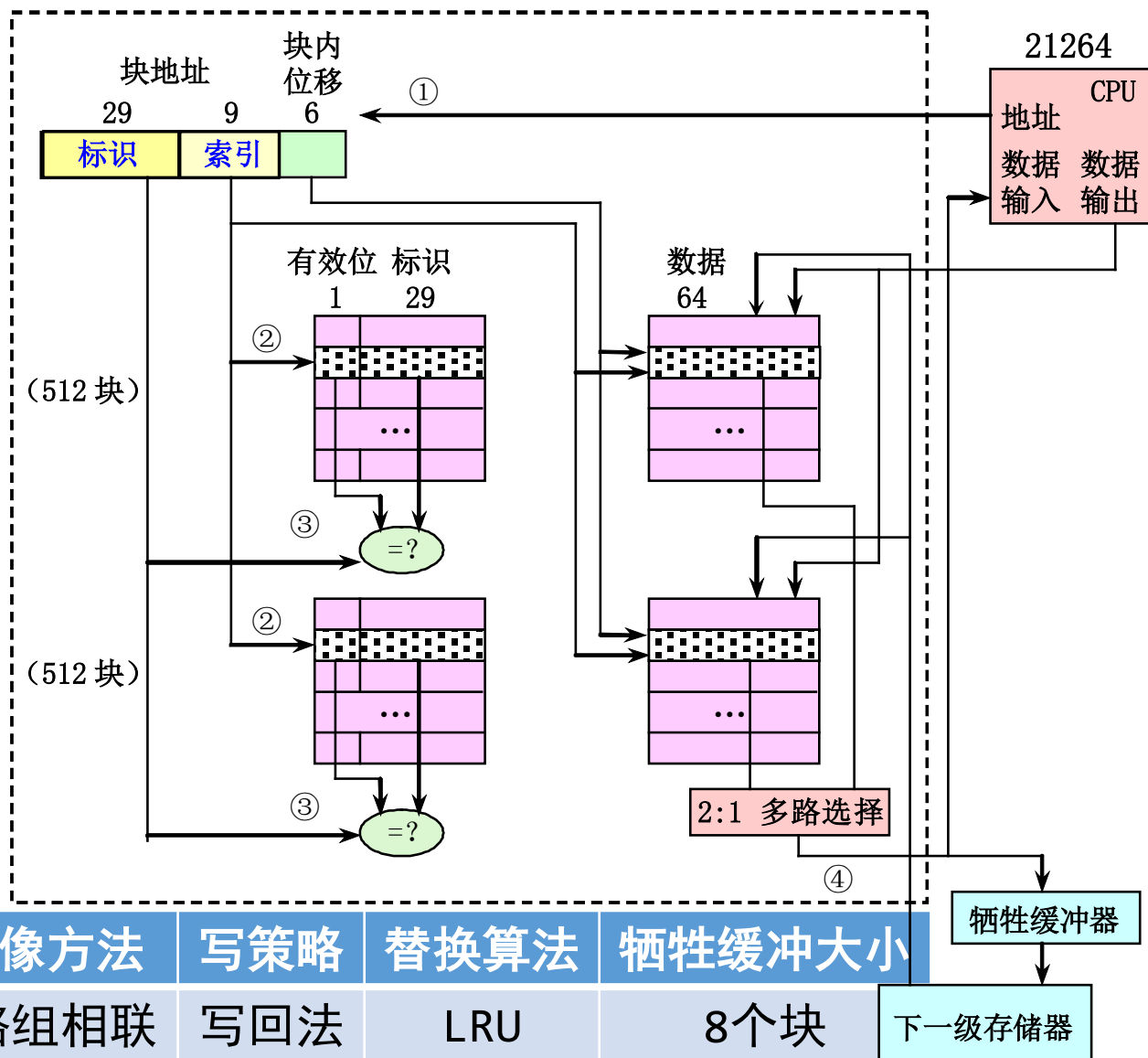
◆ DEC的Alpha AXP21064中的内部数据Cache

容量	块大小	块数	映像方法	写缓冲大小	写策略
8 KB	32 B	256	直接映像	4个块	写直达法



7.2.6 Cache工作过程举例

- ◆ DEC的Alpha AXP21264中的内部数据Cache
 - 没有写缓冲



7.2.7 Cache的性能分析

◆ 1、平均访存时间 (AMAT: Average Memory Access time)

平均访存时间 = 命中时间 + 不命中率 × 不命中开销

◆ 2、程序执行时间

● CPU时间 = (CPU执行周期数 + 存储器停顿周期数) × 时钟周期时间

● 其中：

■ 存储器停顿时钟周期数 = “读” 的次数 × 读不命中率 × 读不命中开销 + “写” 的次数 × 写不命中率 × 写不命中开销

■ 存储器停顿时钟周期数 = 访存次数 × 不命中率 × 不命中开销

$$\begin{aligned} \text{CPU时间} &= IC \times \left(CPI_{\text{execution}} + \frac{\text{访存次数}}{\text{指令数}} \times \text{不命中率} \times \text{不命中开销} \right) \times \text{时钟周期时间} \\ &= IC \times \left(CPI_{\text{execution}} + \text{指令的平均访存次数} \times \text{不命中率} \times \text{不命中开销} \right) \times \text{时钟周期时间} \end{aligned}$$

7.2.7 Cache的性能分析-举例

- ◆ 用一个和Alpha AXP类似的机器作为第一个例子。
- ◆ 例7.1 假设Cache不命中开销为50个时钟周期，当不考虑存储器停顿时，所有指令的执行时间都是2.0个时钟周期，访问Cache不命中率为2%，平均每条指令访存1.33次。试分析Cache对性能的影响。

◆ 解：

- ◆ (1) 首先考虑没有Cache的情况

$$CPI = 2.0 + 1.33 * 50 = 68.5$$

- ◆ (2) 再考虑有Cache的情况

$$CPI = 2.0 + 1.33 * 2\% * 50 = 3.33$$

$$\text{CPU时间} = IC * CPI * \text{时钟周期}$$

	无访存指令	有访存指令	
		有Cache	无Cache
CPI	2.0	3.33	68.5
CPU时间	1	1.67	34.25

7.2.7 Cache的性能分析-举例

- ◆ 例7.2 考虑两种不同组织结构的Cache：直接映像Cache和两路组相联Cache，试问它们对CPU的性能有何影响？先求平均访存时间，然后再计算CPU性能。分析时请用以下假设：
- ◆ (1) 理想Cache（命中率为100%）情况下的CPI为2.0，时钟周期为2ns，平均每条指令访存1.3次。
- ◆ (2) 两种Cache容量均为64KB，块大小都是32字节。
- ◆ (3) 在组相联Cache中，由于多路选择器的存在而使CPU的时钟周期增加到原来的1.10倍。这是因为对Cache的访问总是处于关键路径上，对CPU的时钟周期有直接的影响。
- ◆ (4) 这两种结构Cache的不命中开销都是70ns。（在实际应用中，应取整为整数个时钟周期）
- ◆ (5) 命中时间为1个时钟周期，64KB直接映像Cache的不命中率为1.4%，相同容量的两路组相联Cache的不命中率为1.0%。

2路组相联
Cache的平均
访存时间
比较低

- ◆ 解:
- ◆ 平均访存时间 = 命中时间 + 不命中率 × 不命中开销
- ◆ 因此, 两种结构的平均访存时间分别是:

$$\text{平均访存时间}_{1\text{路}} = 2.0 + (0.014 \times 70) = 2.98\text{ns}$$

$$\text{平均访存时间}_{2\text{路}} = 2.0 \times 1.10 + (0.010 \times 70) = 2.90\text{ns}$$

$$\begin{aligned} \text{CPU时间} &= IC \times \left(CPI_{\text{execution}} + \frac{\text{访存次数}}{\text{指令数}} \times \text{不命中率} \times \text{不命中开销} \right) \times \text{时钟周期时间} \\ &= IC \times (CPI_{\text{execution}} + \text{指令的平均访存次数} \times \text{不命中率} \times \text{不命中开销}) \times \text{时钟周期时间} \\ &= IC \times (CPI_{\text{execution}} \times \text{时钟周期时间} + \text{指令的平均访存次数} \times \text{不命中率} \times \text{不命中时间}) \end{aligned}$$

$$\begin{aligned} \text{CPU时间}_{1\text{路}} &= IC \times [2.0 \times 2 + (1.3 \times 0.014 \times 70)] \\ &= 5.27 \times IC \end{aligned}$$

$$\begin{aligned} \text{CPU时间}_{2\text{路}} &= IC \times [2.0 \times 2 \times 1.10 + (1.3 \times 0.010 \times 70)] \\ &= 5.31 \times IC \end{aligned}$$

$$\frac{\text{CPU时间}_{2\text{路}}}{\text{CPU时间}_{1\text{路}}} = \frac{5.31 \times IC}{5.27 \times IC} = 1.01$$

直接映像Cache的平均性能好一些

7.2.8 改进Cache的性能

- ◆ **平均访存时间 = 命中时间 + 不命中率 × 不命中开销**
- ◆ 可以从三个方面改进Cache的性能：
 - 降低不命中率
 - 减少不命中开销
 - 减少Cache命中时间
- ◆ 下面介绍17种Cache优化技术
 - 8种用于降低不命中率
 - 5种用于减少不命中开销
 - 4种用于减少命中时间

7.3 降低Cache不命中率

◆ 7.3.1 三种类型的命中

◆ Cache失效的原因可分为三类 3C

◆ 强制性不命中/失效 (Compulsory miss)

- 第一次访问某一块，该块不在Cache中，只能从下一级Load，也称为冷启动或首次访问失效

◆ 容量不命中/失效 (Capacity miss)

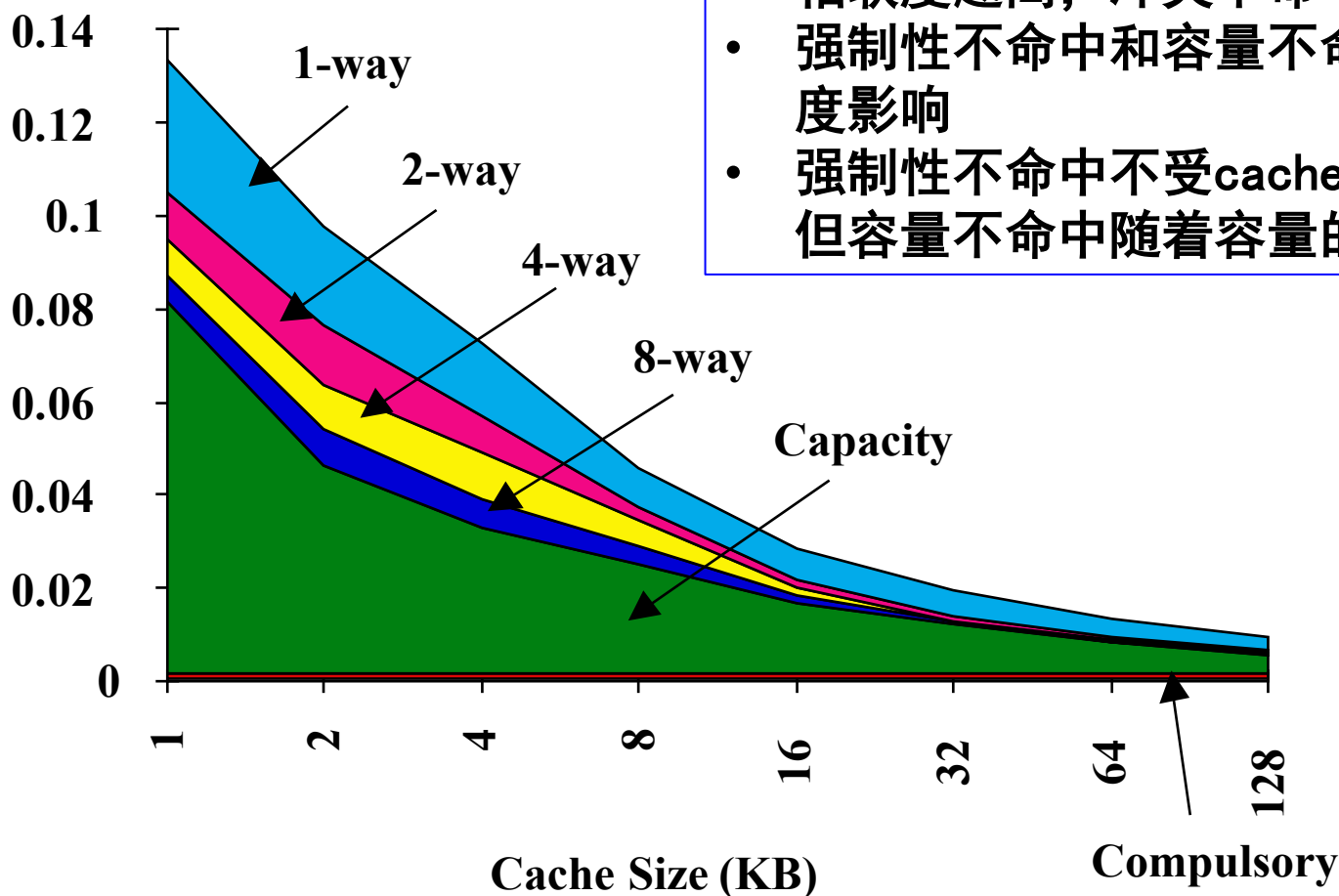
- 如果程序执行时，所需块由于容量不足，不能全部调入Cache，则当某些块被替换后，若又重新被访问，就会发生失效。
- 可能会发生“抖动”现象

◆ 冲突不命中/失效 (Conflict miss)

- 组相联和直接映像的副作用
- 若太多的块映像到同一组（块）中，则会出现该组中某个块被别的块替换（即使别的组或块有空闲位置），然后又被重新访问的情况，这就属于冲突失效。（碰撞不命中，干扰不命中）

各种类型的失效率

◆ 3Cs Absolute Miss Rate (SPEC92)

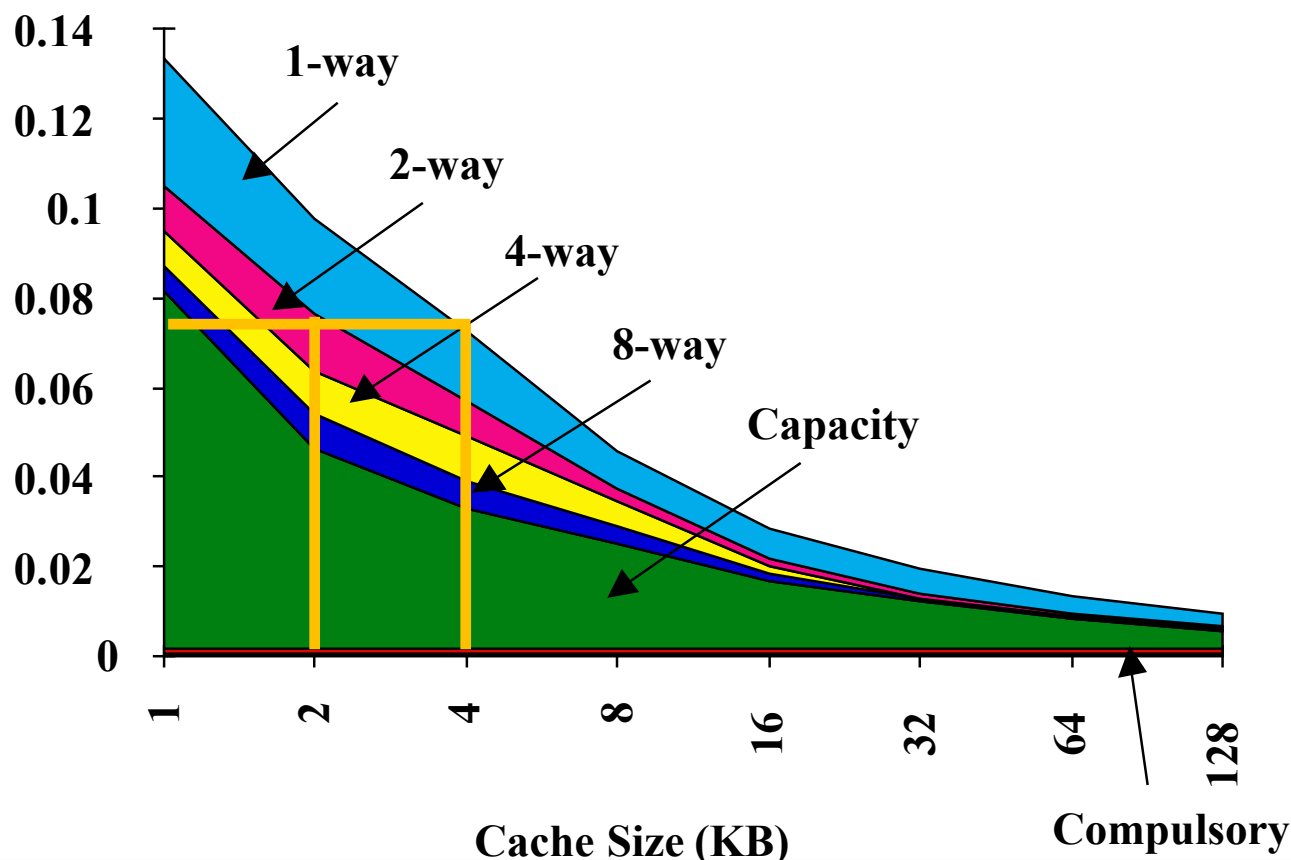


- 相联度越高，冲突不命中就越少
- 强制性不命中和容量不命中不受相联度影响
- 强制性不命中不受cache容量的影响，但容量不命中随着容量的增加而减少

各种类型的失效率

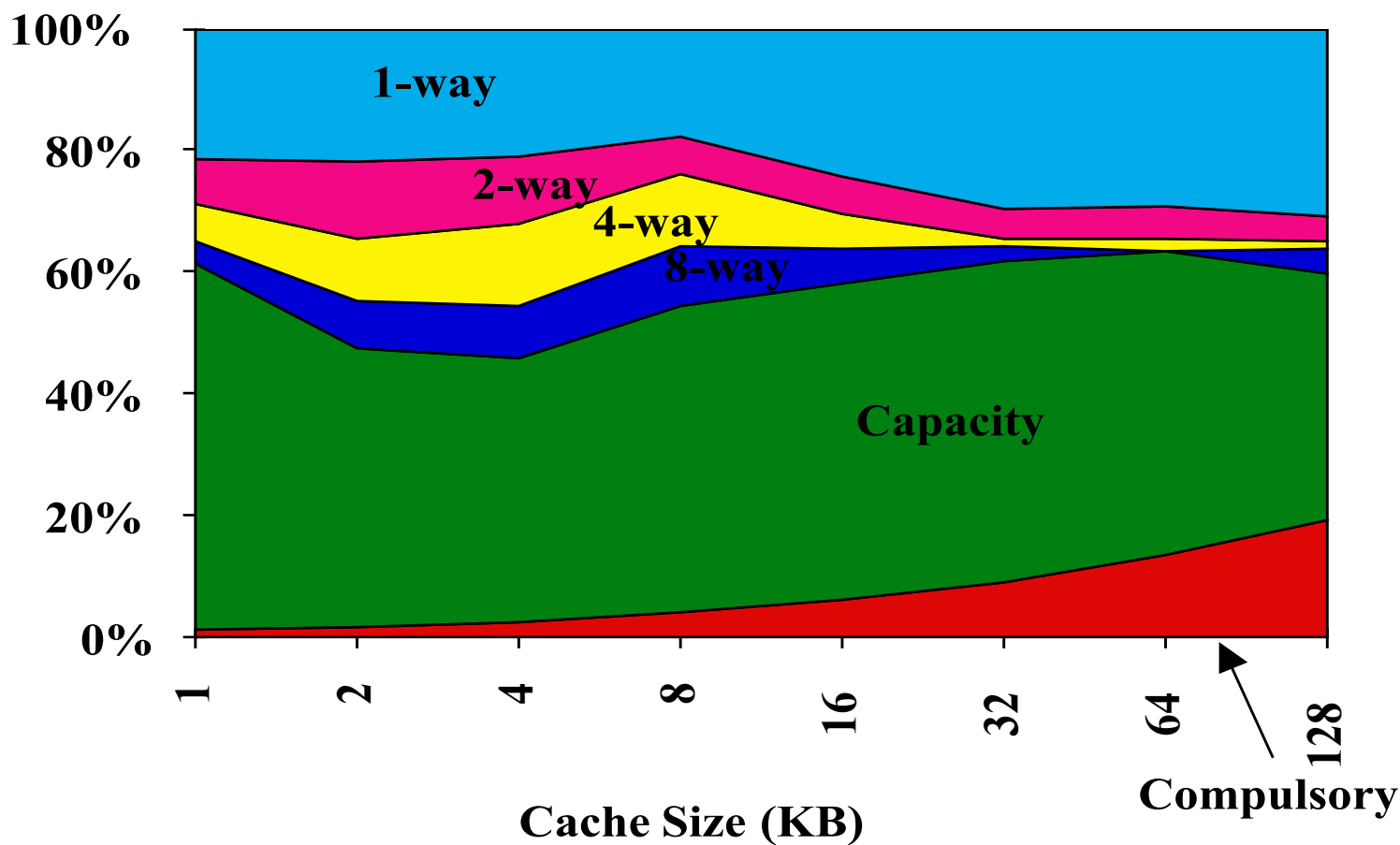
◆ 2:1 Cache经验规则

- 即大小为N的直接映象Cache的失效率约等于大小为N/2的两路组相联的Cache失效率。



总不命中率及3C的分布情况

◆ 3C Relative Miss Rate



减少3C的方法

◆ 增大块

- 减缓强制性失效
- 可能会增加冲突失效（因为在容量不变的情况下，块的数量减少了）

◆ 增大Cache容量

- 对冲突和容量失效的减少有利

◆ 提高相联度

- 对冲突不命中有利。理想情况：全相联

◆ 通过预取可帮助减少强制性失效

- 必须小心不要把你需要的东西换出去
- 需要预测比较准确（对数据较困难，对指令相对容易）

◆ 许多降低不命中率的方法会增加命中时间或不命中开销

7.3.2 增加Cache块大小

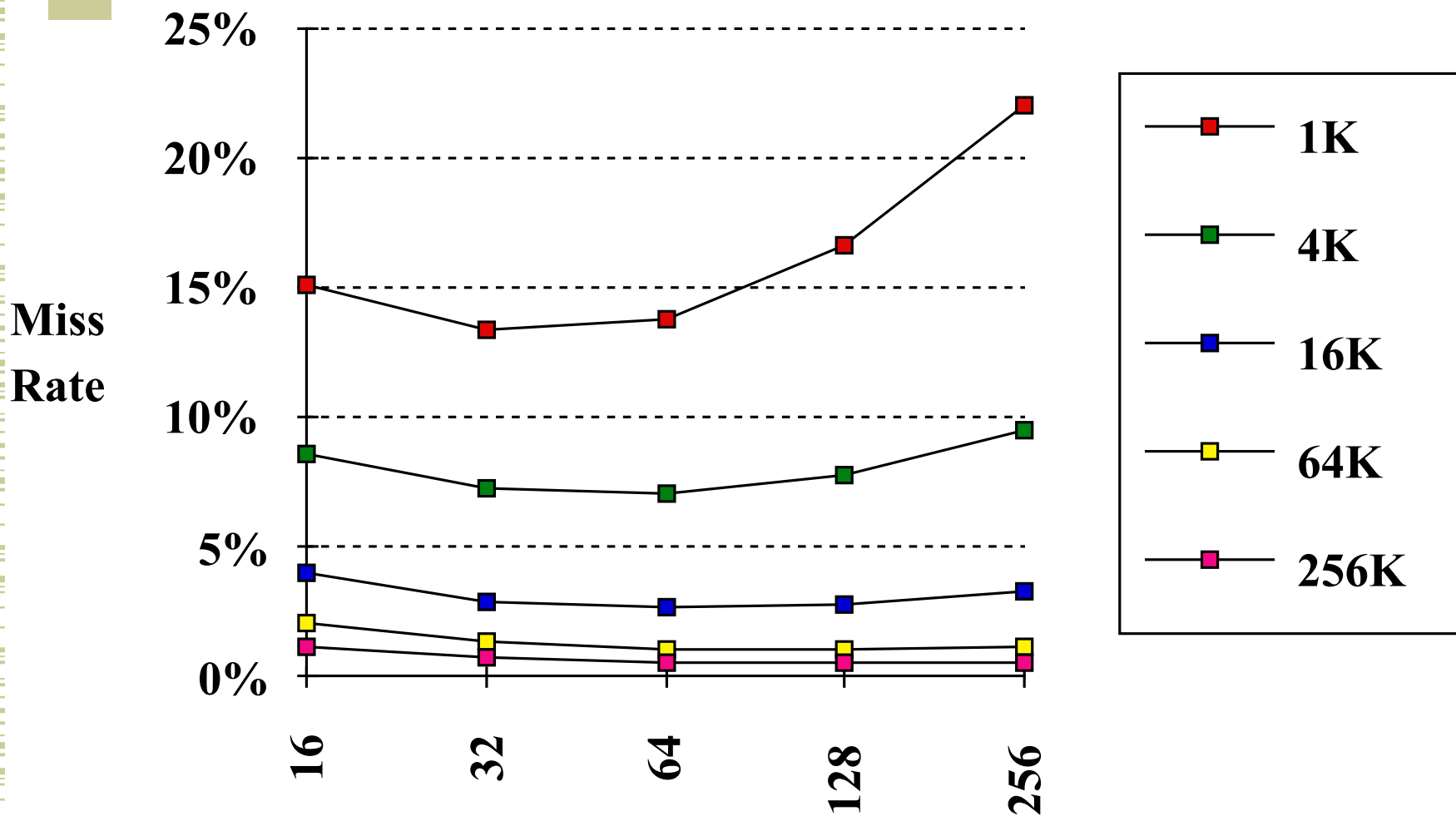
◆ 1、不命中率与块大小的关系

- 对于给定的Cache容量，当块大小增加时，不命中率开始是下降，后来反而上升了。
- 原因：
 - 一方面它减少了强制性不命中；
 - 另一方面，由于增加块大小会减少Cache中块的数目，所以有可能会增加冲突不命中。
- Cache容量越大，使不命中率达到最低的块大小就越大。

◆ 2、增加块大小会增加不命中开销

- 上下层间移动，数据传输时间变大

不命中率随块大小变化的曲线



Block Size (bytes) 64-byte blocks are common in L1 cache
128-byte blocks are common in L2 cache

块大小、容量对失效率的影响

Block Size	Cache = 4 KB	Cache = 16 KB	Cache = 64 KB	Cache = 256 KB
16 bytes	8.57%	3.94%	2.04%	1.09%
32 bytes	7.24%	2.87%	1.35%	0.70%
64 bytes	7.00%	2.64%	1.06%	0.51%
128 bytes	7.78%	2.77%	1.02%	0.49%
256 bytes	9.51%	3.92%	1.15%	0.49%

- ◆ Memory latency = 80 cycles + 1 cycle per 8 bytes
 - Latency of 16-byte block = $80 + 2 = 82$ clock cycles
 - Latency of 32-byte block = $80 + 4 = 84$ clock cycles
 - Latency of 256-byte block = $80 + 32 = 112$ clock cycles
- ◆ **Which block has smallest AMAT for each cache size?**

块大小、容量对AMAT的影响

- ◆ Solution: assume hit time = 1 clock cycle
 - Regardless of block size and cache size
- ◆ Cache Size = 4 KB, Block Size = 16 bytes
 - $AMAT = 1 + 8.57\% \times 82 = 8.027$ clock cycles
- ◆ Cache Size = 256 KB, Block Size = 256 bytes
 - $AMAT = 1 + 0.49\% \times 112 = 1.549$ clock cycles

Block Size	Cache = 4 KB	Cache = 16 KB	Cache = 64 KB	Cache = 256 KB
16 bytes	AMAT = 8.027	AMAT = 4.231	AMAT = 2.673	AMAT = 1.894
32 bytes	AMAT = 7.082	AMAT = 3.411	AMAT = 2.134	AMAT = 1.588
64 bytes	AMAT = 7.160	AMAT = 3.323	AMAT = 1.933	AMAT = 1.449
128 bytes	AMAT = 8.469	AMAT = 3.659	AMAT = 1.979	AMAT = 1.470
256 bytes	AMAT = 11.65	AMAT = 4.685	AMAT = 2.288	AMAT = 1.549

7.3.3 增加Cache的容量

- ◆ 从前面统计分析结果可以看到，降低失效率最直接的方法是增加Cache的容量
- ◆ 缺点：
 - 增加成本
 - 可能增加命中时间
- ◆ 这种方法在片外Cache中用得比较多

7.3.4 提高相联度

- ◆ 采用相联度超过8的方案的实际意义不大。
 - 8路组相联在降低失效率方面的作用已经和全相联一样有效
- ◆ 2:1 Cache经验规则
 - 容量为N的直接映象Cache失效率与容量为N/2的两路组相联Cache的失效率差不多相同
- ◆ **提高相联度是以增加命中时间为代价**
 - TTL或ECL板级Cache, 2路组相联: 增加10%
 - 定制的CMOS Cache, 2路组相联: 增加2%

7.3.4 提高相联度-举例

- ◆ 例7.3 假定提高相联度会按下列比例增大处理器时钟周期：

$$\text{时钟周期}_{2\text{路}} = 1.36 \times \text{时钟周期}_{1\text{路}}$$

$$\text{时钟周期}_{4\text{路}} = 1.44 \times \text{时钟周期}_{1\text{路}}$$

$$\text{时钟周期}_{8\text{路}} = 1.52 \times \text{时钟周期}_{1\text{路}}$$

假定命中时间为一个时钟周期，直接映象情况下失效开销为25个时钟周期，而且假设不必将失效开销取整。使用**下表所示的失效率**，试问当Cache为多大时，以下不等式成立？

$$\text{平均访存时间}_{8\text{路}} < \text{平均访存时间}_{4\text{路}}$$

$$\text{平均访存时间}_{4\text{路}} < \text{平均访存时间}_{2\text{路}}$$

$$\text{平均访存时间}_{2\text{路}} < \text{平均访存时间}_{1\text{路}}$$

	4KB	8KB	16KB	32KB	64KB	128KB	256KB	512KB
1路	0.098	0.068	0.049	0.042	0.037	0.021	0.013	0.008
2路	0.076	0.049	0.041	0.038	0.031	0.019	0.012	0.007
4路	0.071	0.044	0.041	0.037	0.03	0.019	0.012	0.006
8路	0.071	0.044	0.041	0.037	0.029	0.019	0.012	0.006

7.3.4 提高相联度-举例

◆ 解：

在各种相联度的情况下，失效开销均相同，均为25时钟周期

在各种相联度的情况下，平均访存时间分别为：

$$\text{平均访存时间}_{8\text{路}} = \text{命中时间}_{8\text{路}} + \text{失效率}_{8\text{路}} \times \text{失效开销}_{8\text{路}}$$

$$= 1.52 + \text{失效率}_{8\text{路}} \times 25$$

$$\text{平均访存时间}_{4\text{路}} = 1.44 + \text{失效率}_{4\text{路}} \times 25$$

$$\text{平均访存时间}_{2\text{路}} = 1.36 + \text{失效率}_{2\text{路}} \times 25$$

$$\text{平均访存时间}_{1\text{路}} = 1.00 + \text{失效率}_{1\text{路}} \times 25$$

把相应的失效率代入上式，即可得平均访存时间

对于4KB的Cache有：

$$\text{平均访存时间}_{1\text{路}} = 1.00 + 0.098 \times 25 = 3.44$$

$$\text{平均访存时间}_{8\text{路}} = 1.52 + 0.071 \times 25 = 3.28$$

7.3.4 提高相联度-举例

- ◆ 在各种容量和相联度情况下Cache的平均访存时间

Cache size (KiB)	相联度			
	1-way	2-way	4-way	8-way
4	3.44	3.25	3.22	3.28
8	2.69	2.58	2.55	2.62
16	2.23	2.40	2.46	2.53
32	2.06	2.30	2.37	2.45
64	1.92	2.14	2.18	2.25
128	1.52	1.84	1.92	2.00
256	1.32	1.66	1.74	1.82
512	1.20	1.55	1.59	1.66

- ◆ 小于等于8KB时，4路组相联<2路组相联<1路组相联
- ◆ **大于等于16KB时，以上等式不成立**

7.3.5 伪相联 Cache（列相联 Cache）

◆ 1、多路组相联的低失效率 and 直接映象的命中速度

伪相联Cache 的优点		优 点	缺 点
	直接映象	命中时间小	失效率高
	组相联	失效率低	命中时间大

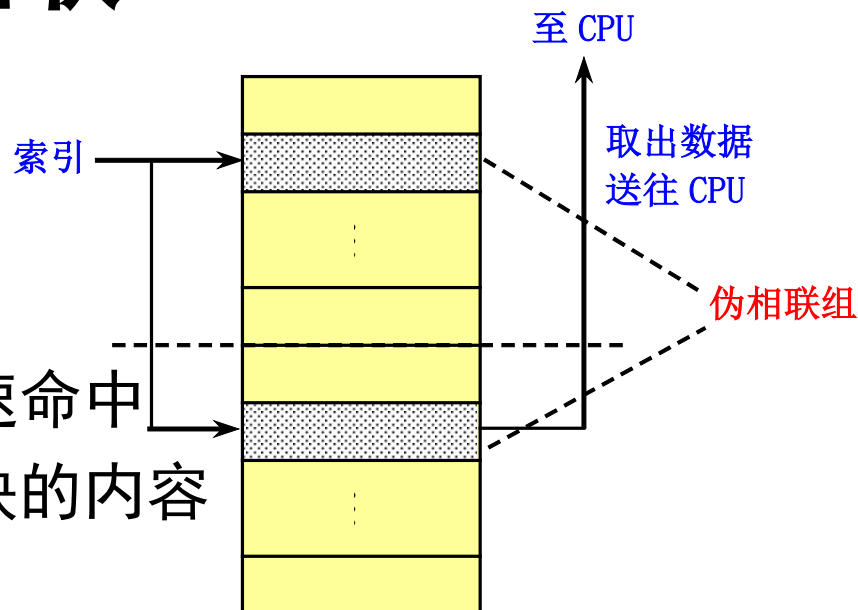
◆ 2、基本思想及工作原理

- 在逻辑上把直接映象Cache的空间平分为上下两个区。
- 对于任何一次访问，伪相联Cache先按直接映象Cache的方式去处理。
- 若命中，则其访问过程与直接映象Cache的情况一样。
- 若不命中，则再到另一区相应的位置去查找。若找到，则发生了伪命中，否则就只好访问下一级存储器。

7.3.5 伪相联 Cache

◆ 3、快速命中与慢速命中

- 快速命中：正常命中
- 慢速命中：伪命中
- 要保证绝大多数命中都是快速命中
- 当发生伪命中时，交换两个块的内容



正常命中时间

伪命中时间

失效开销

时间

7.3.5 伪相联 Cache-举例

- ◆ 例7.4 假设当在直接映象找到的位置处没有发现匹配，而在另一个位置才找到数据（伪命中）需要2个额外的周期。仍用例7.3中的数据，问：当Cache容量分别为4KB和128KB时，直接映象、2路组相联和伪相联这三种组织结构中，哪一种速度最快？

- ◆ 解：首先考虑标准的平均访存时间公式：

$$\text{平均访存时间}_{\text{伪}} = \text{命中时间}_{\text{伪}} + \text{失效率}_{\text{伪}} \times \text{失效开销}_{\text{伪}}$$

$$\text{由于：失效率}_{\text{伪}} = \text{失效率}_{2\text{路}}$$

$$\text{命中时间}_{\text{伪}} = \text{命中时间}_{1\text{路}} + \text{命中率}_{\text{伪}} \times 2$$

伪相联查找的命中率等于2路组相联Cache的命中率和直接映象Cache命中率之差。

$$\begin{aligned}\text{命中率}_{\text{伪}} &= \text{命中率}_{2\text{路}} - \text{命中率}_{1\text{路}} \\ &= (1 - \text{失效率}_{2\text{路}}) - (1 - \text{失效率}_{1\text{路}}) \\ &= \text{失效率}_{1\text{路}} - \text{失效率}_{2\text{路}}\end{aligned}$$

7.3.5 伪相联 Cache-举例

- ◆ 综合分析，有

$$\text{平均访存时间}_{\text{伪}} = \text{命中时间}_{1\text{路}} + (\text{失效率}_{1\text{路}} - \text{失效率}_2) \times 2 \\ + \text{失效率}_{2\text{路}} \times \text{失效开销}_{1\text{路}}$$

根据例7.3中的失效率数据，有

$$\begin{aligned} \text{平均访存时间}_{\text{伪}, 4\text{KB}} &= 1 + (0.098 - 0.076) \times 2 + 0.076 \times 25 \\ &= 2.944 \end{aligned}$$

$$\begin{aligned} \text{平均访存时间}_{\text{伪}, 128\text{KB}} &= 1 + (0.021 - 0.019) \times 2 + 0.019 \times 25 \\ &= 1.479 \end{aligned}$$

- ◆ 根据例7.3中的平均访存时间数据，有

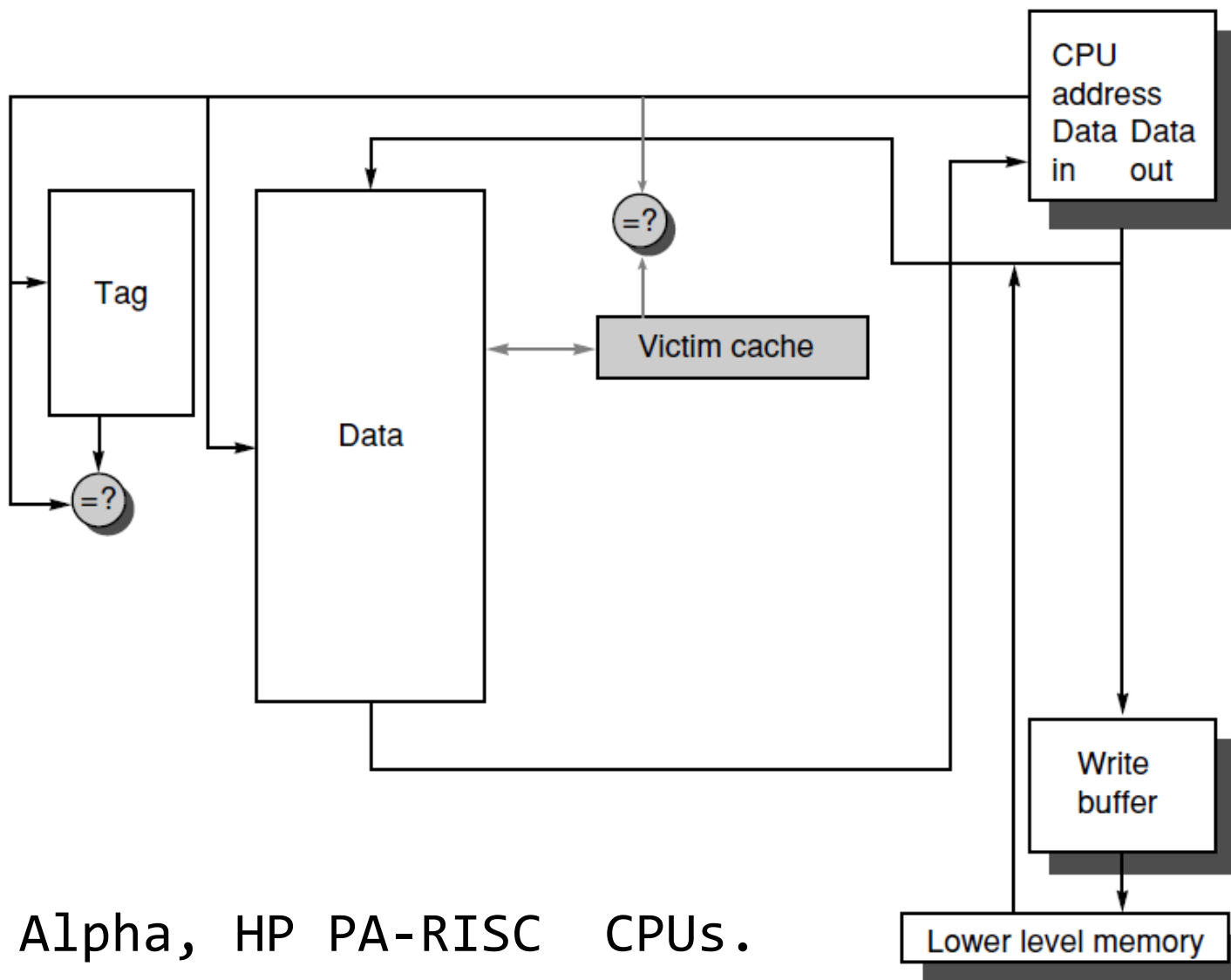
	4KB	128KB
1路	3.44	1.52
2路	3.25	1.84

可知，对于这两种
Cache容量，伪相联
Cache都是速度最快。

7.3.6 “牺牲” Cache

- ◆ 1、一种能减少冲突失效次数而又不影响时钟频率的方法。
- ◆ 2、基本思想：在Cache和它从下一级存储器之间的通路中设置一个**全相联的小Cache**，用于存放被替换出去的块(称为Victim)，以备重用。
 - 通常Cache为直接映象时冲突失效率较大
 - Victim cache采用全相联—失效率较低
- ◆ 3、作用
 - 对减小冲突失效很有效，特别是对于小容量的直接映象数据Cache，作用尤其明显
 - 例如：项数为4的Victim Cache：能使4 KB Cache的冲突失效减少20%~90%
 - Jouppi [1990]

7.3.6 “牺牲” Cache



◆ Used in Alpha, HP PA-RISC CPUs.

7.3.7 硬件预取

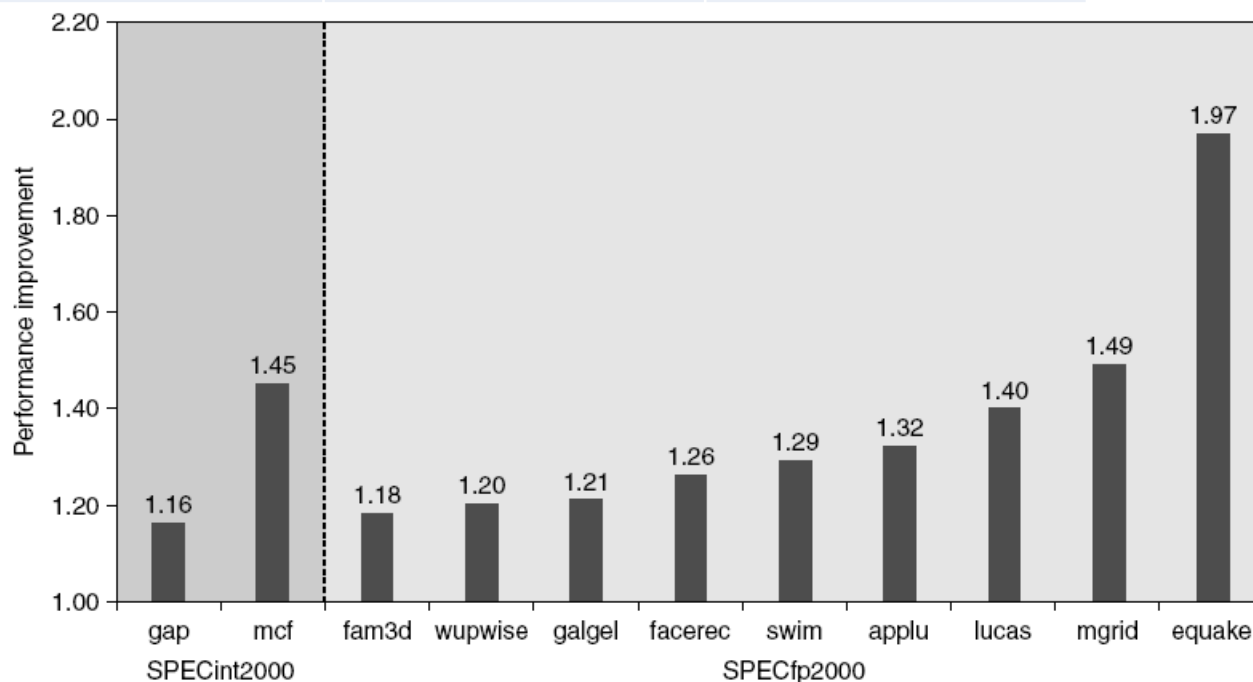
- ◆ CPU在执行当前块代码时，硬件预取下一块代码
 - CPU可能马上就要执行这块代码，这样可以降低或消除Cache的访问失效
 - 指令和数据都可以预取，**当块中有控制指令时，预取失效**
 - 指令预取通常由Cache之外的硬件完成
- ◆ 预取内容既可放入Cache，也可放在外缓冲器中。
 - 例如：指令流缓冲器ISB，（存取速度比Memory快的地方）
- ◆ 预取应利用存储器的空闲带宽，不能影响对正常不命中的处理，否则可能会降低性能。
- ◆ AXP21064失效时，取2块指令块
 - 目标块放在指令cache，下一块放在ISB（指令流缓冲）中
 - 如果访问的块在ISB中，取消访存请求，直接从ISB中读，并发出对下一指令块的预取访存请求

7.3.7 硬件预取

- ◆ 预取效果, Jouppi的研究结果, 块大小为16B, 4KB的Cache, 使用直接映像

Cache类型	1个块的缓冲	4个块的缓冲	16个块的缓冲
指令Cache	15% - 25%	50%	72%
数据Cache	25%	43%	

- ◆ Pentium 4预取
 - 失效时取2个块



7.3.7 硬件预取-举例

- ◆ 例7.5 Alpha AXP 21064采用指令预取技术，假设预取命中率为25%，命中时间为2个时钟周期，失效开销为25个时钟周期。试问：若不采用指令预取技术，Alpha AXP 21064的指令Cache必须为多大才能保持平均访存时间不变？

解：假设当指令不在指令Cache里，而在预取缓冲器中找到时，**需要多花一个时钟周期**。此时：

$$\text{平均访存时间}_{\text{预取}} = \text{命中时间} + \text{失效率} \times \text{预取命中率} \times 1 \\ + \text{失效率} \times (1 - \text{预取命中率}) \times \text{失效开销}$$

通过查找例7.3中的失效率表，有 **失效率_{8KB} = 6.8%**

$$\text{平均访存时间}_{\text{预取}} = 2 + 0.068 \times 0.25 + 0.068 \times (1 - 0.25) \times 25 = 3.292$$

为了得到相同性能下的实际失效率，由原始公式得：

$$\begin{aligned} \text{平均访存时间} &= \text{命中时间} + \text{失效率} \times \text{失效开销} \\ \text{失效率} &= (\text{平均访存时间} - \text{命中时间}) / \text{失效开销} \\ &= (3.292 - 2) / 25 = 5.2\% \end{aligned}$$

- ◆ 该失效率介于普通8KB Cache失效率和16KB Cache失效率之间。
- ◆ **若不采用指令预取，则指令Cache须介于 8~16KB 之间。**

7.3.8 编译器控制的预取

- ◆ 在编译时加入**预取指令**，在数据被用到之前发出预取请求。
- ◆ 目的：**使执行指令和读取数据能重叠执行。**
- ◆ 按照预取数据所放的位置，可把预取分为两种类型
 - **寄存器预取**：把数据取到寄存器中。
 - **Cache预取**：只将数据取到Cache中。
- ◆ 按照预取的处理方式不同，可把预取分为：
 - **故障性预取**：在预取时，若出现虚地址故障或违反保护权限，就会发生异常。
 - **非故障性预取**：在预取时，若出现虚地址故障或违反保护权限则不会发生异常，因为这时它会放弃预取，转变为空操作。
- ◆ **假定Cache预取都是非故障性的**，也叫做非绑定预取

7.3.8 编译器控制的预取

- ◆ 在预取数据的同时，处理器应能继续执行。只有这样，预取才有意义。
 - 称为**非阻塞Cache**（非锁定Cache）
- ◆ **循环是预取优化的主要对象**
 - **不命中开销小时**：循环体展开1~2次，调度好预取与执行的重叠
 - **不命中开销大时**：循环体展开许多次，以便为较远的循环预取数据
- ◆ **每次预取需要花费一条指令的开销**
 - 保证这种开销不超过预取所带来的收益
 - 编译器可以通过把重点放在那些**可能会导致不命中的访问**上，使程序避免不必要的预取，从而较大程度地减少平均访存时间。

7.3.8 编译器控制的预取-举例

- ◆ 例7.6 对于下面的程序，首先判断哪些访问可能会导致数据Cache失效。然后，加入预取指令以减少失效。最后，计算所执行的预取指令的条数以及通过预取避免的失效次数。

- ◆ 假定：

(1) 我们用的是一个容量为8KB、块大小为16B的直接映象Cache，它采用写回法并且按写分配。

(2) a、b分别为 3×100 （3行100列）和 101×3 的双精度浮点数组，每个元素都是8B。当程序开始执行时，这些数据都不在Cache内。

```
for ( i = 0 ; i < 3 ; i = i + 1 )  
    for ( j = 0 ; j < 100 ; j = j + 1 )  
        a[i][j] = a[i][j] + b[j][0]* b[j+1][0];
```

7.3.8 编译器控制的预取-举例

- ◆ 解:
 - ◆ 分析计算过程
 - ◆ 失效情况: 总的失效次数 = $50 \times 3 + 101 = 251$ 次
 - ◆ 假设失效开销很大, 预取必须至少提前7次循环进行。
 - ◆ 改进后的程序的失效情况
 - ◆ 总的失效次数 = 20次, 但多执行了400次预取指令。
- ```
for(j=0; j<100; j=j+1){
 prefetch(b[j+1+7][0]);
 /*预取7次循环后所需的b(j+1,0)*/
 prefetch(a[0][j+7]);
 /*预取7次循环后所需的a (0,j)*/
 a[0][j]=a[0][j]+b[j][0]*b[j+1][0];
}
for(i=1; i<3; i=i+1){
 for(j=0; j<100; j=j+1) {
 prefetch(a[i][j+7]);
 /*预取7次循环后所需的a(i,j)*/
 a[i][j]=a[i][j]+b[j][0]*b[j+1][0];
 }
}
```

## 7.3.8 编译器控制的预取-举例

- ◆ 计算以上示例中节省的时间

- ◆ 假设：

- (1) 忽略指令Cache未命中，并假定数据Cache中没有冲突或容量未命中。

- (2) 假定预取可以相互重叠，并且可以与Cache未命中重叠，从而以最大内存带宽进行传输。

- (3) 以下是忽略Cache未命中的关键循环时间：

- 原始循环每次迭代需要7个时钟周期，
    - 第一个预取循环每次迭代需要9个时钟周期，
    - 第二个预取循环每次迭代需要8个时钟周期（包括外部for循环）
    - Cache未命中时需要49个时钟周期。



## 7.3.8 编译器控制的预取-举例

### ◆ 预取前:

- 计算 $a[i][j]$ ,  $3 \times 100$ 次迭代, 每次迭代需要7个cycles
- 失效251次, 每次 49 cycles
- 共计:  $300 \times 7 + 251 \times 49 = 14399$  cycles

### ◆ 预取后:

- 第1个循环, 100次迭代, 每次迭代9个cycles, 失效12次
- 第1个循环共计:  $100 \times 9 + 12 \times 49 = 1488$  cycles
- 第2个循环,  $2 \times 100$ 次迭代, 每次8个cycles, 失效8次
- 第2个循环共计:  $2 \times 100 \times 8 + 8 \times 49 = 1992$  cycles
- 总计:  $1488 + 1992 = 3480$

### ◆ 加速比: $14399 / 3480 = 4.14$

## 7.3.9 编译器优化

### ◆ 基本思想

- 无需对硬件做任何改动，通过软件优化降低失效率

### ◆ 研究从两方面展开：

- 减少指令失效
- 减少数据失效

### ◆ 1、**减少指令失效**，重新组织程序（指令调度）而不影响程序的正确性

- 把一个程序中的过程重新排序，就可能会减少冲突不命中，从而降低指令不命中率。
- 把基本块对齐，使得程序的入口点与Cache块的起始位置对齐，就可以减少顺序代码执行时所发生的Cache不命中的可能性。

## 7.3.9 编译器优化

- ◆ 如果编译器知道一个分支指令很可能会成功转移，那么它就可以通过以下两步来改善空间局部性：
  - 将转移目标处的基本块和紧跟着该分支指令后的基本块进行对调；
  - 把该分支指令换为操作语义相反的分支指令。
- ◆ 研究方法：通过使用profiling信息来判断指令组间可能发生的冲突，并将指令重新排序以减少失效
- ◆ 研究表明：
  - 容量为2KB，块大小为 4Bytes的直接映象指令Cache，通过使用指令调度可以使失效率降低50%。容量增大到 8KB，失效率可降低75%
  - 优化后（指令调度）的程序在直接映象Cache中的失效率也低于未优化程序在同样大小的8路组相联Cache中的失效率

## 7.3.9 编译器优化

- ◆ 2、**减少数据失效**，主要通过优化来改善数据的空间局部性和时间局部性，基本方法为：
  - **数组合并**
    - 将本来相互独立的多个数组合并成为一个复合数组，以提高访问它们的局部性。
  - **内外循环交换**
    - 通过交换循环的嵌套关系，使程序按数据在存储器中存储的顺序进行访问。
  - **循环融合**
    - 将若干个独立的循环融合为单个的循环。这些循环访问同样的数组，对相同的数据作不同的运算。这样能使得读入Cache的数据在被替换出去之前，能得到反复的使用。
  - **分块**

# 编译器优化方法举例：内外循环交换

## 以下为C语言代码

```
/* 优化前 */
```

```
for (j = 0; j < 100; j = j+1)
 for (i = 0; i < 5000; i = i+1)
 x[i][j] = 2 * x[i][j];
```

```
/* 优化后 */
```

```
for (i = 0; i < 5000; i = i+1)
 for (j = 0; j < 100; j = j+1)
 x[i][j] = 2 * x[i][j];
```

# 编译器优化方法举例：循环融合

// 优化前

```
for (i = 0; i < N; i++)
 a[i] = b[i] + c[i];
for (i = 0; i < N; i++)
 d[i] = a[i] + b[i] * c[i];
```

// 优化后

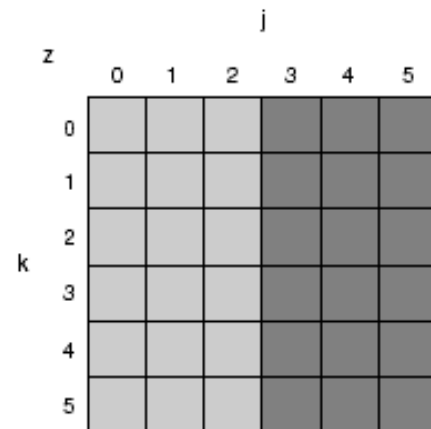
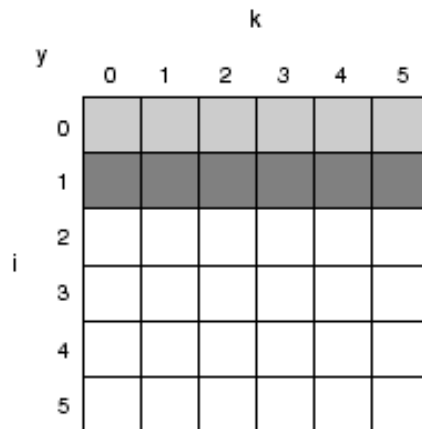
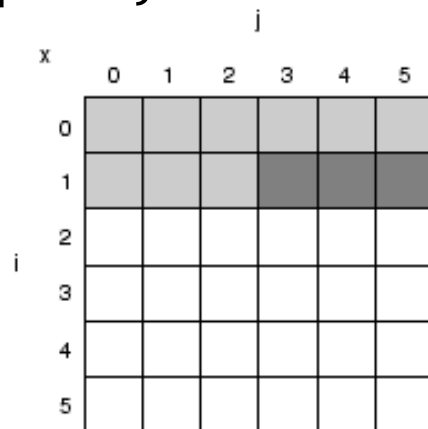
```
for (i = 0; i < N; i++) {
 a[i] = b[i] + c[i];
 d[i] = a[i] + b[i] * c[i];
}
```

- ◆ 优化后的代码更好的考虑了数据访问的**时间局部性**

# 编译器优化方法举例：分块

/\* 优化前 \*/

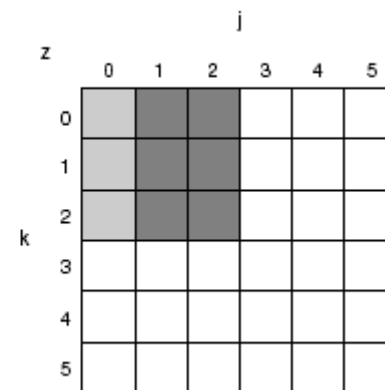
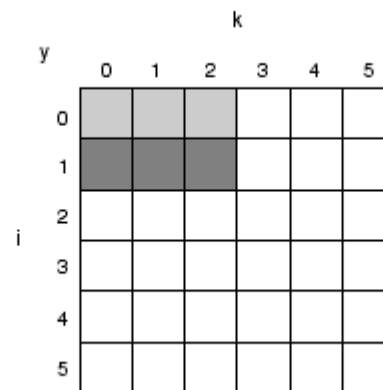
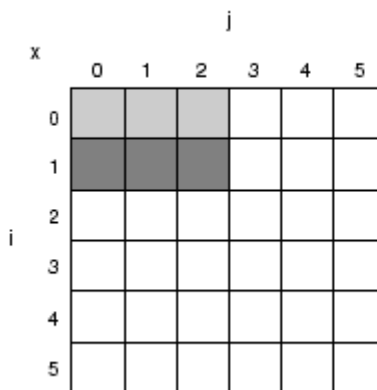
```
for (i = 0; i < N; i = i+1) {
 for (j = 0; j < N; j = j+1) {
 r = 0;
 for (k = 0; k < N; k = k + 1) {
 r = r + y[i][k] * z[k][j];
 }
 x[i][j] = r;
 }
}
```



◆  $N^3$ 次操作，产生的存储器访问次数为:  $2N^3+N^2$

# 编译器优化方法举例：分块

```
for(jj = 0; jj < N; jj = jj+B) /*优化后*/
 for(kk = 0; kk < N; kk = kk+B)
 for(i = 0; i < N; i = i+1)
 for(j = jj; j < min(jj+B, N); j = j+1) {
 r = 0;
 for(k = kk; k < min(kk+B, N); k = k+1) {
 r = r + y[i][k] * z[k][j];
 }
 x[i][j] = x[i][j] + r;
 }
 }
```



- ◆ 只考虑容量不命中
- ◆ 存储器访问次数为:  $2N^3/B + N^2$



## 7.4 减少Cache不命中开销

### ◆ 7.4.1 采用两级Cache

- 一级cache保持较小容量

- 降低命中时间
- 降低每次访问的能耗

- 增加二级cache

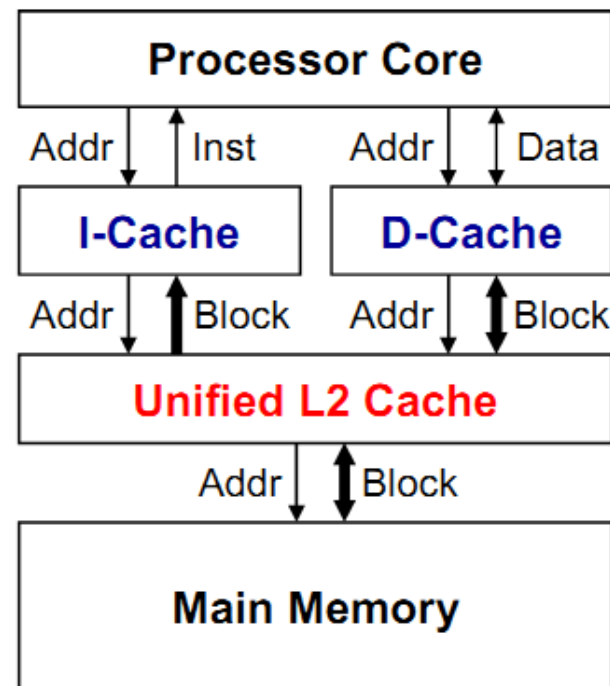
- 减少与存储器的gap
- 减少存储器总线的负载

- 多级cache的优点

- 减少失效开销
- 缩短平均访存时间（AMAT）

- 较大容量的L2 cache可以捕捉许多L1 cache的失效

- 降低全局失效率

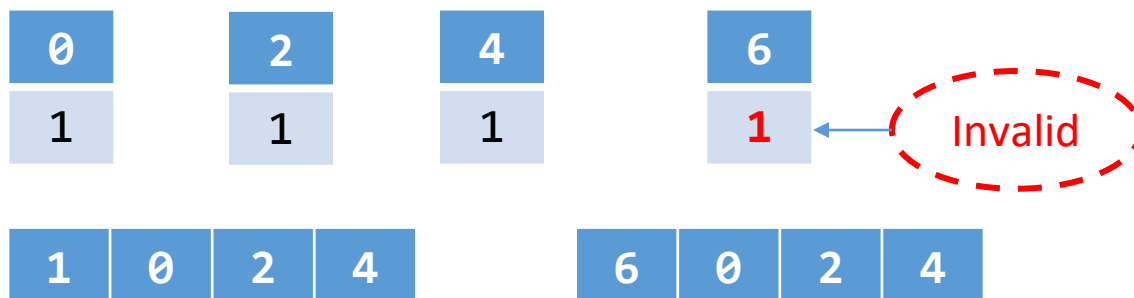


# 多级包容性(multilevel inclusive)

- ◆ L1 cache的块总是存在于L2 cache中
  - 浪费了L2 cache空间，L2还应当有存放其他块的空间
  - L1中miss，但在L2中命中，则从L2拷贝相应的块到L1
- ◆ 在L1和L2中均miss，则从更低级存储器拷贝相应的块到L1和L2
- ◆ 对L1写操作导致将数据同时写到L1和L2
- ◆ Write-through策略用于L1到L2
- ◆ Write-back策略可用于L2到更低级存储器，以降低存储总线的数据传输压力
- ◆ L2的替换动作（或无效）对L1可见
  - 即L2的一块被替换出去，那么其在L1中对应的块也要被替换出去。

# 多级包容性-举例

- 访问的块序列 1 0 2 4 6
- 替换策略：LRU
- 块大小相同
- cache容量
  - L1: 2                  L2: 4

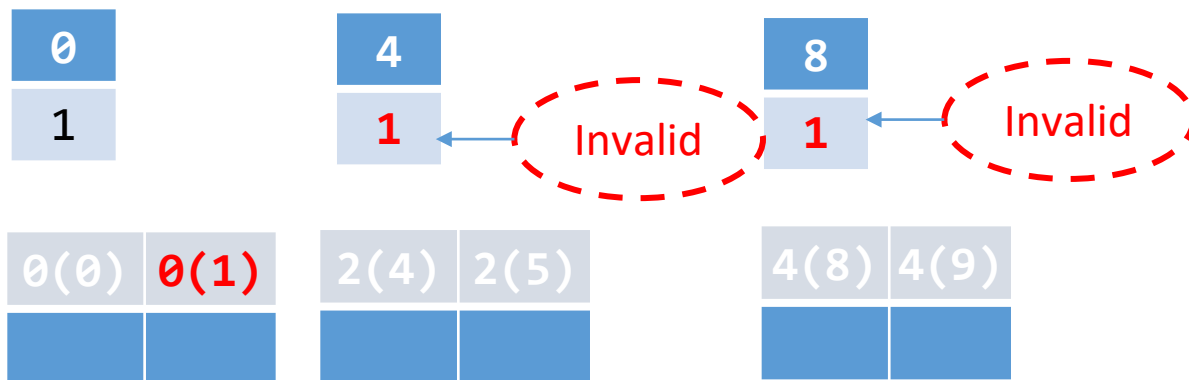


|   |
|---|
| 0 |
| 1 |
| 2 |
| 3 |
| 4 |
| 5 |
| 6 |
| 7 |

# 多级包容性-举例

- L1和L2的块大小不同，L1的块大小是L2的块大小的1/2
- 替换策略：LRU
- 考察按L1的块划分的访问序列：

0 1 4 8



|   |      |
|---|------|
| 0 | 0(0) |
| 1 | 0(1) |
| 2 | 1(2) |
| 3 | 1(3) |
| 4 | 2(4) |
| 5 | 2(5) |
| 6 | 3(6) |
| 7 | 3(7) |
| 8 | 4(8) |
| 9 | 4(9) |

## 多级不包容 (Multilevel Exclusive)

- ◆ **L1 cache中的块不会在L2 cache中**，以避免浪费空间
  - 在L1中miss，但在L2中命中，将导致Cache间块的互换
- ◆ 在L1和L2中均miss，将仅仅从更低层拷贝相应的块到L1
  - L1的被替换的块移至L2
- ◆ L2存储L1抛弃的块，以防后续L1还需要使用
- ◆ L1到L2的写策略为Write-Back
- ◆ L2到更低级存储器的写策略为Write-Back
- ◆ L1和L2的块大小可以相同也可以不同
  - Pentium 4 had 64-byte blocks in L1 but 128-byte blocks in L2
  - Core i7 uses 64-byte blocks at all cache levels (simpler)

# 多级不包容-举例

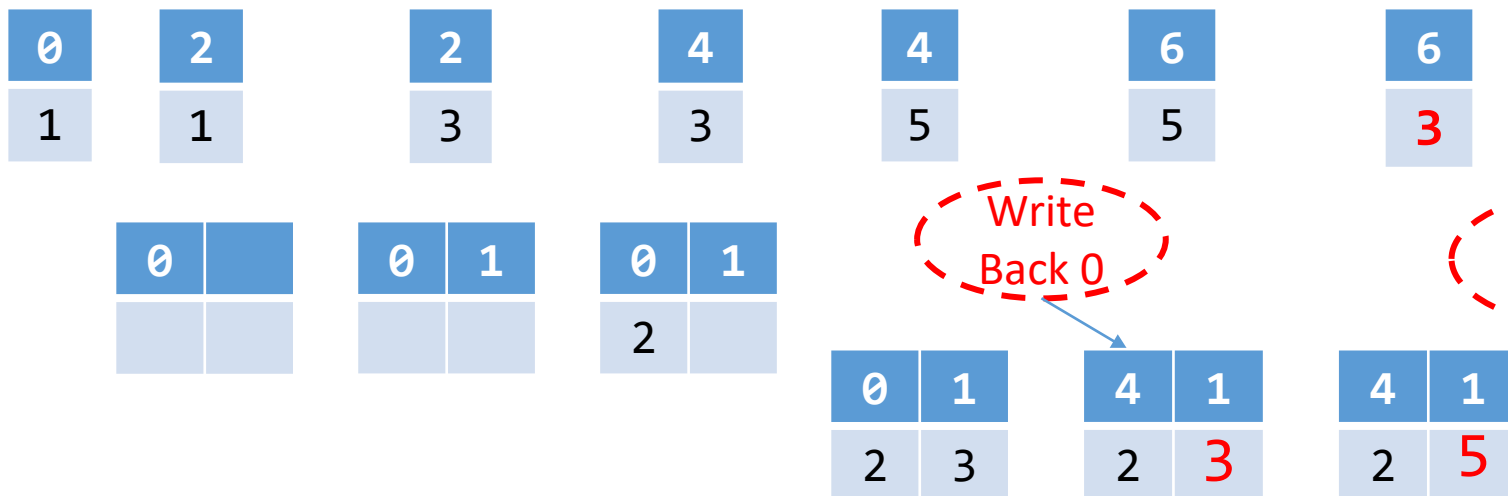
L1: 1-way L2: 2-way

访问的块序列: 0 1 **2** 3 4 5 **6** **3**

替换策略: LRU

块大小相同

|   |
|---|
| 0 |
| 1 |
| 2 |
| 3 |
| 4 |
| 5 |
| 6 |





## 7.4.1 采用两级Cache

### ◆ 1、应把Cache做得更快？还是更大？

- 答案：
- **二者兼顾**，再增加一级Cache
- 第一级Cache(L1)小而快
- 第二级Cache(L2)容量大

### ◆ 2、性能分析

- 平均访存时间 = 命中时间<sub>L1</sub> + 不命中率<sub>L1</sub> × 不命中开销<sub>L1</sub>  
不命中开销<sub>L1</sub> = 命中时间<sub>L2</sub> + 不命中率<sub>L2</sub> × 不命中开销<sub>L2</sub>
- **平均访存时间** = 命中时间<sub>L1</sub> + 不命中率<sub>L1</sub> ×  
(命中时间<sub>L2</sub> + 不命中率<sub>L2</sub> × 不命中开销<sub>L2</sub>)



## 7.4.1 采用两级Cache

### ◆ 3、局部不命中率与全局不命中率

- **局部不命中率** = 该级Cache的不命中次数 / 到达该级Cache的访问次数
  - 例如：上述式子中的不命中率 $L_2$
- **全局不命中率** = 该级Cache的不命中次数 / CPU发出的访存的总次数
- **全局不命中率 $L_2$**  = 不命中率 $L_1$   $\times$  不命中率 $L_2$

**评价第二级Cache时，应使用全局不命中率这个指标。**  
它指出了在CPU发出的访存中，究竟有多大比例是穿过各级Cache，最终到达存储器的。

### ◆ 4、采用两级Cache时，每条指令的平均访存停顿时间

- **每条指令的平均访存停顿时间** = 每条指令的平均不命中次数 $L_1$   
 $\times$  命中时间 $L_2$  + 每条指令的平均不命中次数 $L_2$   $\times$  不命中开销 $L_2$

# 两级Cache性能分析-举例

- ◆ 例7.7 考虑某一两级Cache：第一级Cache为L1，第二级Cache为L2。

(1) 假设在1000次访存中，L1的不命中是40次，L2的不命中是20次。求各种局部不命中率和全局不命中率。

(2) 假设L2的命中时间是10个时钟周期，L2的不命中开销是100时钟周期，L1的命中时间是1个时钟周期，平均每条指令访存1.5次，不考虑写操作的影响。问：平均访存时间是多少？每条指令的平均停顿时间是多少个时钟周期？

- ◆ 解： (1)

第一级Cache的不命中率（全局和局部）是 $40/1000$ ，即4%

第二级Cache的局部不命中率是 $20/40$ ，即50%；

第二级Cache的全局不命中率是 $20/1000$ ，即2%；

或者  $4\% \times 50\% = 2\%$  。

# 两级Cache性能分析-举例

$$\begin{aligned} (2) \text{ 平均访存时间} &= \text{命中时间}_{L_1} + \text{不命中率}_{L_1} \\ &\quad \times (\text{命中时间}_{L_2} + \text{不命中率}_{L_2} \times \text{不命中开销}_{L_2}) \\ &= 1 + 4\% \times (10 + 50\% \times 100) \\ &= 1 + 4\% \times 60 = 3.4 \text{ 个时钟周期} \end{aligned}$$

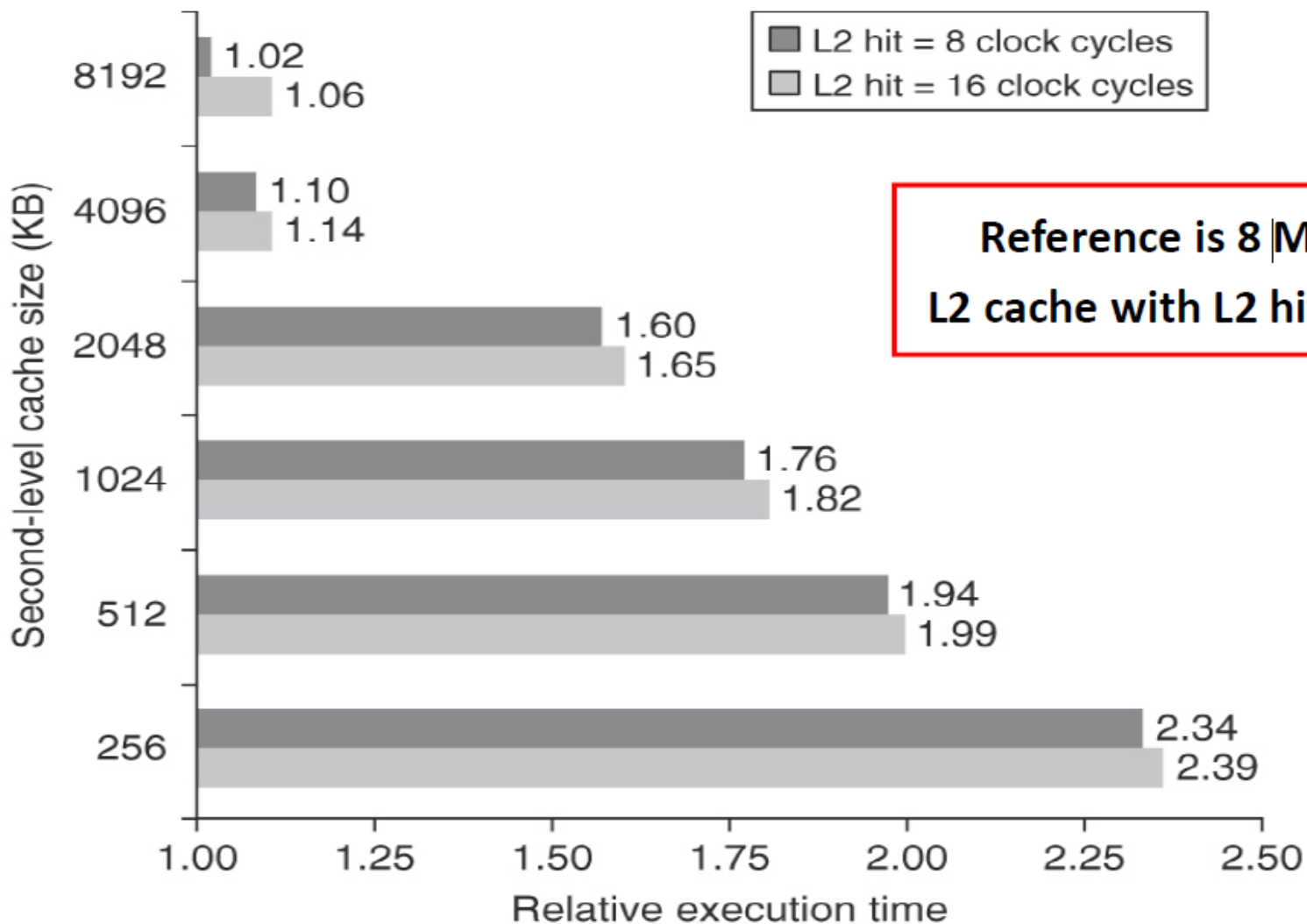
由于平均每条指令访存1.5次，且每次访存的平均停顿时间为： $3.4 - 1.0 = 2.4$

所以：**每条指令的平均停顿时间**  $= 2.4 \times 1.5 = 3.6$  个时钟周期

◆ 4、对于第二级Cache，我们有以下结论：

- 在第二级Cache比第一级Cache大得多的情况下，两级Cache的全局不命中率和容量与第二级Cache相同的单级Cache的不命中率非常接近。
- 局部不命中率不是衡量第二级Cache的一个好指标，因此，**在评价第二级Cache时，应使用全局不命中率这个指标。**

# 不同大小的L2 Cache对应的执行时间



## 7.4.1 采用两级Cache

- ◆ 5、第二级Cache不会影响CPU的时钟频率，因此其设计有更大的考虑空间。
  - **两个问题：**
  - 它能否降低CPI中的平均访存时间部分？
  - 它的成本是多少？
- ◆ 6、第二级Cache的参数
  - **容量：**第二级Cache的容量一般比第一级的大许多
    - 大容量意味着第二级Cache可能实际上没有容量不命中，只剩下一些强制性不命中和冲突不命中。
  - **相联度**
    - 第二级Cache可采用较高的相联度或伪相联方法。

# 两级Cache性能分析-举例

- ◆ 例7.8 给出有关第二级Cache的以下数据：
    - (1) 对于直接映像，命中时间 $L_2 = 10$ 个时钟周期
    - (2) 两路组相联使命中时间增加0.1个时钟周期，即为10.1个时钟周期。
    - (3) 对于直接映像，局部不命中率 $L_2 = 25\%$
    - (4) 对于两路组相联，局部不命中率 $L_2 = 20\%$
    - (5) 不命中开销 $L_2 = 50$ 个时钟周期
- 试问第二级Cache的相联度对不命中开销的影响如何？

# 两级Cache性能分析-举例

解：对一个直接映像的第二级Cache来说，**第一级Cache的不命中开销**为：

不命中开销<sub>直接映像, L1</sub> =  $10 + 25\% \times 50 = 22.5$ 个时钟周期

对于两路组相联第二级Cache来说，命中时间增加了10% (0.1) 个时钟周期，故第一级Cache的不命中开销为：

不命中开销<sub>两路组相联, L1</sub> =  $10.1 + 20\% \times 50 = 20.1$ 个时钟周期

**把第二级Cache的命中时间取整，得10或11，则：**

不命中开销<sub>两路组相联, L1</sub> =  $10 + 20\% \times 50 = 20.0$ 个时钟周期

不命中开销<sub>两路组相联, L1</sub> =  $11 + 20\% \times 50 = 21.0$ 个时钟周期

**故对于第二级Cache来说，两路组相联优于直接映像**

## 7.4.2 让读不命中优先于写

- ◆ 1、由于读操作为大概率事件，需要读失效优先，以提高性能
- ◆ 2、写直达Cache → **写缓冲**
  - 依靠写缓冲来减少对下一级存储器写操作的时间
  - **Cache中的写缓冲器导致对存储器访问的复杂化**
    - 在读不命中时，所读单元的最新值有可能还在写缓冲器中，尚未写入主存。
    - 例如，直接映射、写直达、512和1024映射到同一块。如下所示：

SW R3, 512(R0)

LW R1, 1024(R0)                      失效

LW R2, 512(R0)                        失效



## 7.4.2 让读不命中优先于写

### ◆ 2、写直达Cache → 写缓冲

#### ● 解决问题的方法

- 推迟对读失效的处理，直到写缓冲器清空，导致新的问题——读失效开销增大。
- 在读失效时，检查写缓冲的内容，如果没有冲突，而且存储器可访问，就可以继续处理读失效

#### ● 写回法时，也可以利用写缓冲器来提高性能

- 把脏块放入缓冲区，然后读存储器，最后写存储器

### ◆ 3、写回法Cache → Victim Buffer

#### ● 被替换的脏块放到了victim buffer

#### ● 在脏块被写回前，需要处理读失效

#### ● 问题：victim buffer 可能含有该读失效要读取的块

- Solution: 查找victim buffer，如果命中直接将该块调入Cache

## 7.4.3 写缓冲合并

- ◆ 1、目的：**提高写缓冲器的效率**
- ◆ 2、写直达Cache
  - 依靠写缓冲来减少对下一级存储器写操作的时间。
  - 如果写缓冲器为空，就把数据和相应地址写入该缓冲器。从CPU的角度来看，该写操作就算是完成了。
  - 如果写缓冲器中已经有了待写入的数据，就要把这次的写入地址与写缓冲器中已有的所有地址进行比较，看是否有匹配的项。如果有地址匹配而对应的位置又是空闲的，就把这次要写入的数据与该项合并。这就叫**写缓冲合并**。
  - **如果写缓冲器满且又没有能进行写合并的项，就必须等待**
- ◆ 作用：**提高了写缓冲器的空间利用率，而且还能减少因写缓冲器满而要进行的等待时间。**

# 写缓冲合并-举例

| 写地址 | V |          | V |  | V |  | V |  |
|-----|---|----------|---|--|---|--|---|--|
| 100 | 1 | Mem[100] | 0 |  | 0 |  | 0 |  |
| 108 | 1 | Mem[108] | 0 |  | 0 |  | 0 |  |
| 116 | 1 | Mem[116] | 0 |  | 0 |  | 0 |  |
| 124 | 1 | Mem[124] | 0 |  | 0 |  | 0 |  |

(a) 不采用写合并

| 写地址 | V |          | V |          | V |          | V |          |
|-----|---|----------|---|----------|---|----------|---|----------|
| 100 | 1 | Mem[100] | 1 | Mem[108] | 1 | Mem[116] | 1 | Mem[124] |
|     | 0 |          | 0 |          | 0 |          | 0 |          |
|     | 0 |          | 0 |          | 0 |          | 0 |          |
|     | 0 |          | 0 |          | 0 |          | 0 |          |

(b) 采用了写合并

- ◆ 写缓冲有4项，4\*8B/项，V代表有效位。如不采用写合并，3/4空间浪费。采用写合并，只需占1项。

## 7.4.4 请求字处理技术

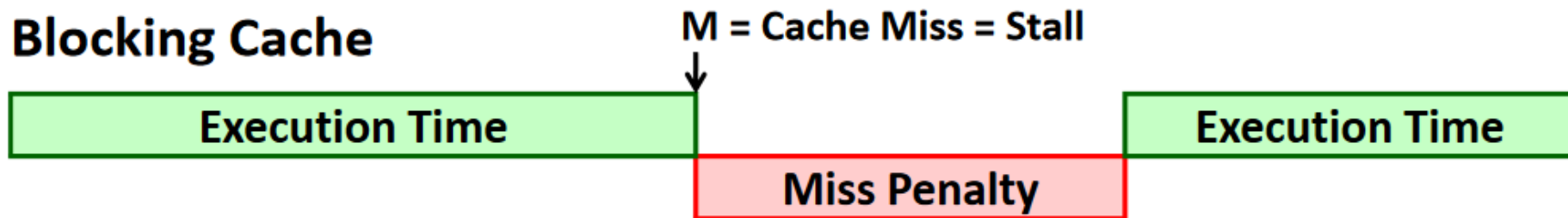
- ◆ 1、**请求字**（Requested Word）：从下一级存储器调入Cache的块中，**只有一个字是CPU立即需要的**。
- ◆ 2、基本思路：应尽快把请求字发送给CPU
  - **尽早重启动**：从块的起始位置开始读起。一旦请求字到达，就立即发送给CPU，让CPU继续执行。
  - **请求字优先**：调块时，从请求字所在的位置读起，然后再读取该块其余部分至Cache。
- ◆ 3、这种技术在以下情况下效果不大：
  - Cache块较小
  - 下一条指令正好访问同一Cache块的另一部分

## 7.4.5 非阻塞Cache技术

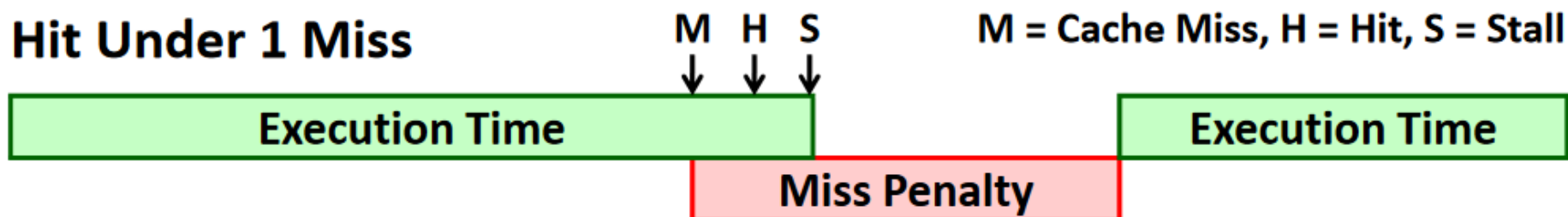
- ◆ 1、非阻塞Cache：Cache不命中时仍允许CPU进行其他的命中访问。即允许“**不命中下命中**”。
  - 在Cache失效时，CPU无需stall
  - 主要用于乱序执行和多线程处理器
  - 减少有效的失效开销
  - 增加Cache的带宽
- ◆ 2、进一步提高性能：“多重不命中下命中”
  - “不命中下不命中”（存储器必须能够处理多个不命中）
  - 针对多个未解决的Cache失效
  - 可能会更多地减少有效的失效开销
  - 增加了Cache控制器的复杂性
  - 存储系统可以支持多个失效时的存储服务

## 7.4.5 非阻塞Cache技术

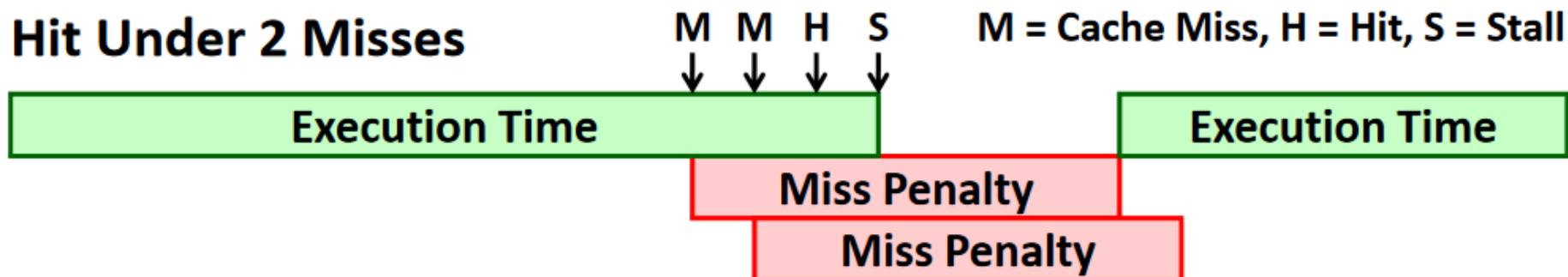
### Blocking Cache



### Hit Under 1 Miss



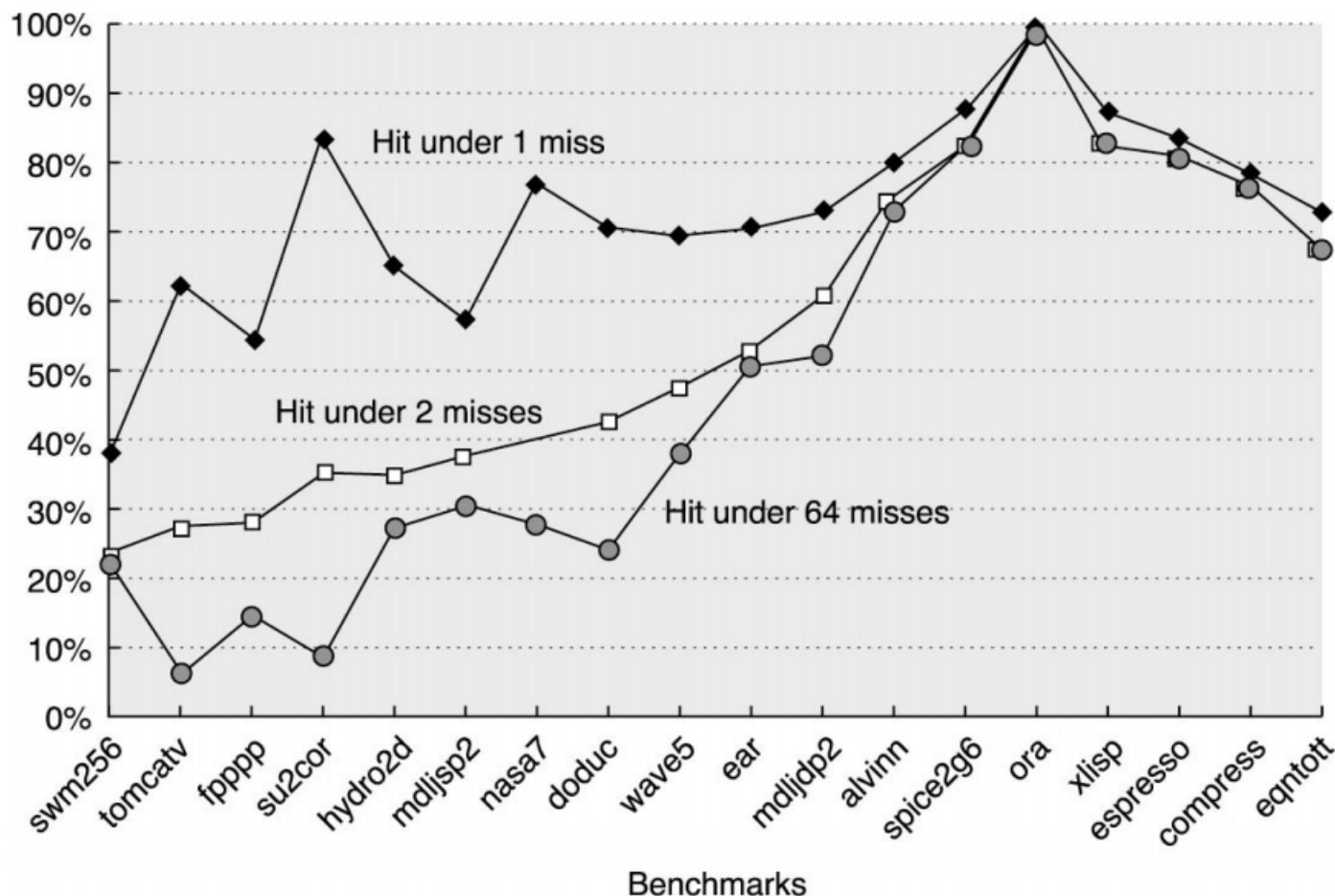
### Hit Under 2 Misses



### ◆ Nonblocking Cache Timeline

# 非阻塞Cache技术应用效果

Percentage of  
the average  
memory stall  
time



- ◆ FP programs on average: AMAT= 0.68 -> 0.52 -> 0.34 -> 0.26
- ◆ Int programs on average: AMAT= 0.24 -> 0.20 -> 0.19 -> 0.19
- ◆ 8 KB Data Cache, Direct Mapped, 32B block, 16 cycle miss

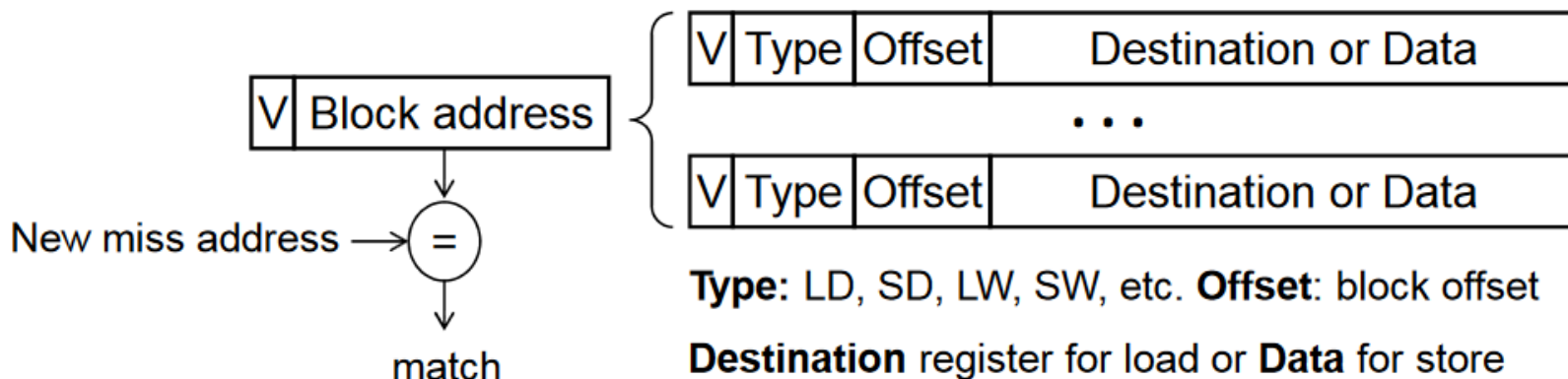
# Miss Status Holding Register (MSHR)

## ◆ MSHR包含正在等待处理的失效

- 相同的块可以包含多个未解决的Load/Store失效
- 可以有多个未解决的块地址

## ◆ 失效可以分为

- Primary: 第一次发起存取请求时的失效块
- Secondary: 在后续过程中的失效
- Structural Stall miss: MSHR硬件资源耗尽





# 非阻塞Cache技术

- ◆ **当Cache失效时**，检查MSHR是否有匹配的块地址
  - 如果有，为该地址分配新的load/store 表项
  - 如果没有，分配新的MSHR和load/store表项
  - 如果所有MSHR资源都分配完，则Stall（结构相关）
- ◆ **当从底层传输Cache块时**
  - 处理该块中的Load和Store指令引起的失效
  - Load：根据block offset从该块中装载数据到寄存器
  - Store：根据block offset将数据写入该块指定位置
  - 完成该块所有的失效的Load/Store后，释放MSHR中的对应表项

# Multi-Ported Cache

## ◆ Dual-Ported Data Cache

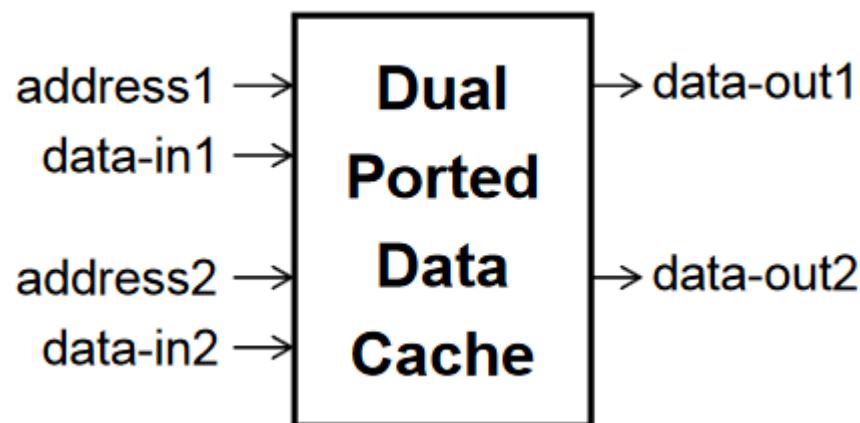
- 两个地址端口（每个Cycle支持两条load/store 指令）
- 在现代处理器中保持较高的指令吞吐率

## ◆ True Multi-ported Cache Design

- 所有的控制和数据通路在Cache中是多份的
- **显著地增加了Cache的面积和访问时间**

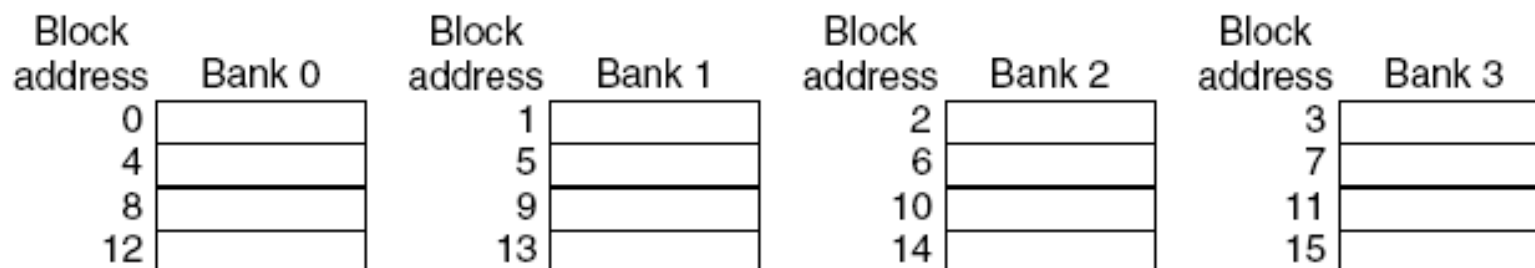
## ◆ Multi-Banked Cache

- 将cache组织成多个banks
- 每个bank是一个端口
- 可以并行访问不同的Bank



# Multibanked Caches

- ◆ 将Cache组织为多个独立的banks，以支持并行访问
  - ARM Cortex-A8 supports 1-4 banks for L2
  - Intel i7 supports 4 banks for L1 and 8 banks for L2
- ◆ 根据块号进行多体交叉编址

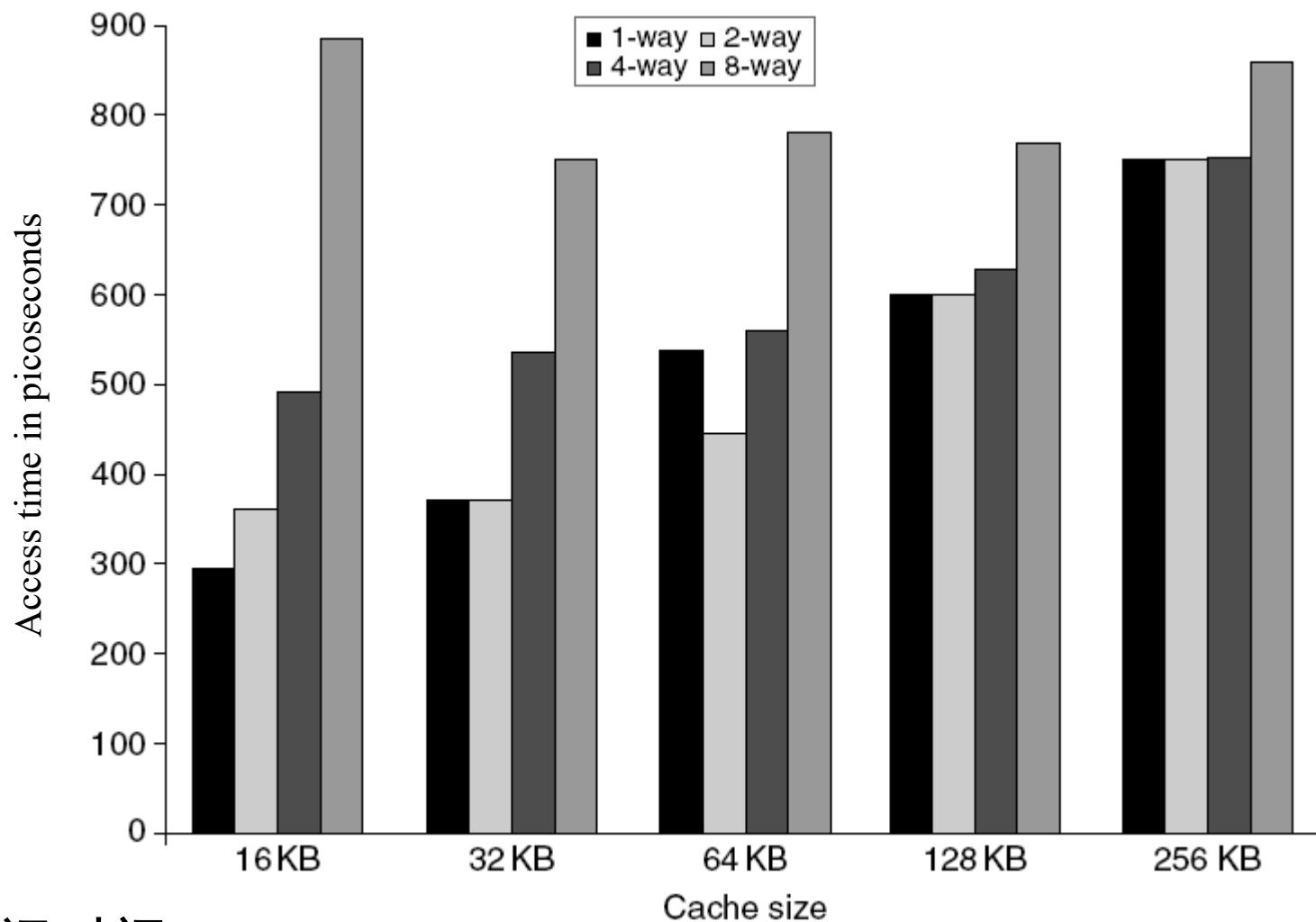


**Figure 2.6** Four-way interleaved cache banks using block addressing. Assuming 64 bytes per blocks, each of these addresses would be multiplied by 64 to get byte addressing.

## 7.5 减少命中时间

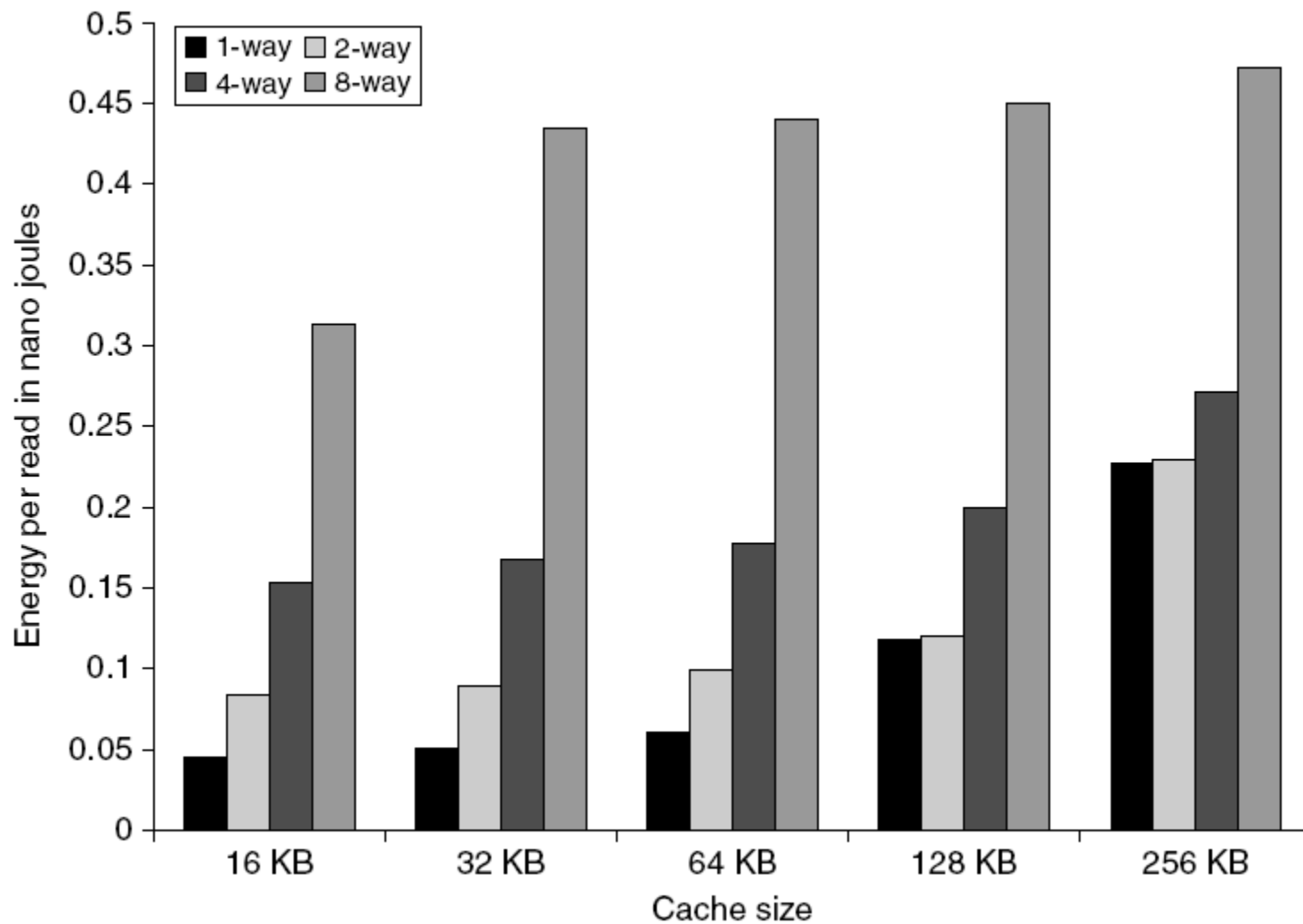
- ◆ 7.5.1 容量小、结构简单的Cache
  - 容量小，一般命中时间短，有可能做在片内
  - 另一方案，保持Tag在片内，块数据在片外，如DEC Alpha
  - 第一级Cache应选择容量小且结构简单的设计方案
- ◆ Critical timing path:
  - 1) 定位组(tag)，确定tag的位置
  - 2) 比较tags
  - 3) 选择正确的块
- ◆ 直接映像Cache可使数据传输和tag比较并行
- ◆ 简单的Cache结构、可有效减少tag比较的次数，进而降低功耗

# L1大小和相联度



## ◆ 访问时间

# L1大小和相联度

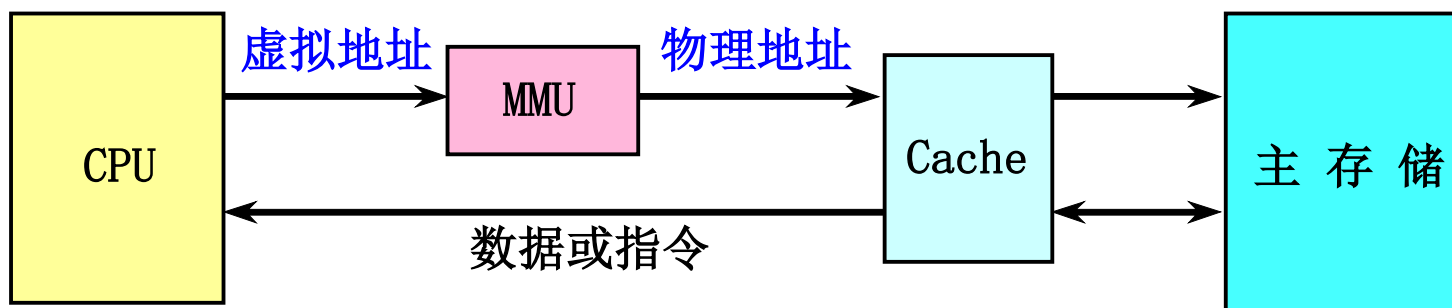


## ◆ 能耗

## 7.5.2 虚拟Cache

### ◆ 1、物理Cache

- 使用物理地址进行访问的传统Cache。
- **标识存储器中存放的是物理地址**，进行地址检测也是用物理地址。
- 缺点：地址转换和访问Cache串行进行，访问速度很慢。



物理Cache存储系统

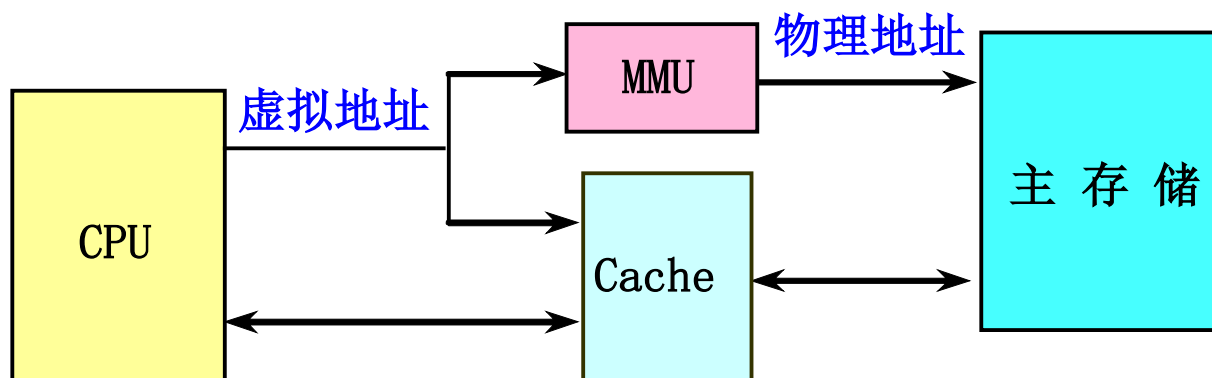
## 7.5.2 虚拟Cache

### ◆ 虚拟Cache

- 可以**直接用虚拟地址进行访问的Cache**。标识存储器中存放的是虚拟地址，进行地址检测用的也是虚拟地址。

### ◆ 优点：

- 在命中时不需要地址转换，省去了地址转换的时间。
- 即使不命中，地址转换和访问Cache也是并行进行的，其速度比物理Cache快很多。



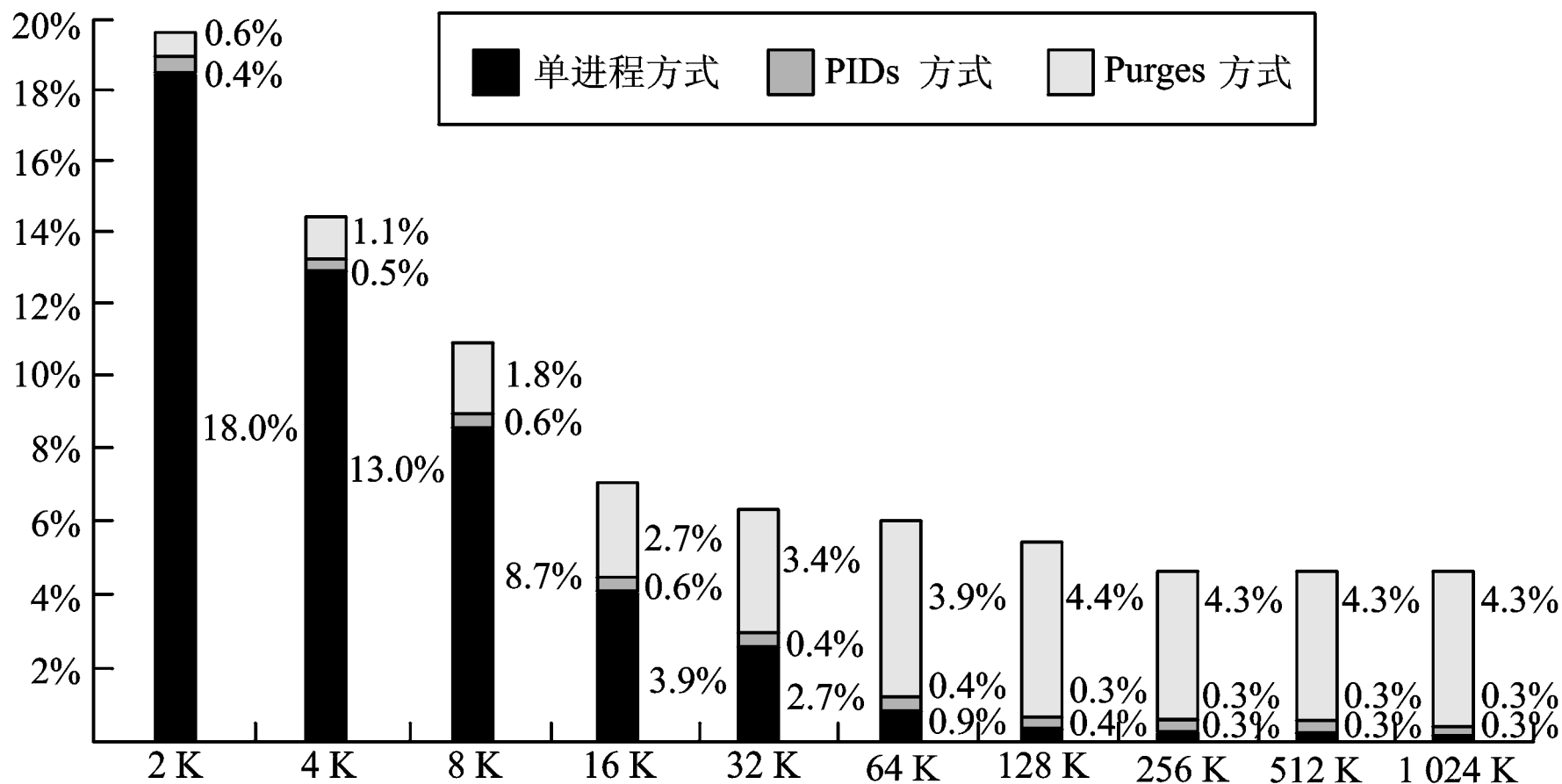


# 为什么大多不采用虚拟Cache?

- ◆ 虚拟Cache的清空问题
  - 解决方法：在地址标识中增加PID字段（进程标识符）
- ◆ 三种情况下不命中率的比较
  - 单进程，PIDs，清空
  - PIDs与单进程相比： $+0.3\% \sim +0.6\%$
  - PIDs与清空相比： $-0.6\% \sim -4.3\%$
- ◆ 虚拟Cache没有流行的原因是同一个物理地址有2种以上的虚拟地址，这些地址称为**同义**（Synonym）和**别名**（Alias）：
  - 两个虚拟地址对应同一个数据。
  - 软件解决方法：**反别名法、页着色**

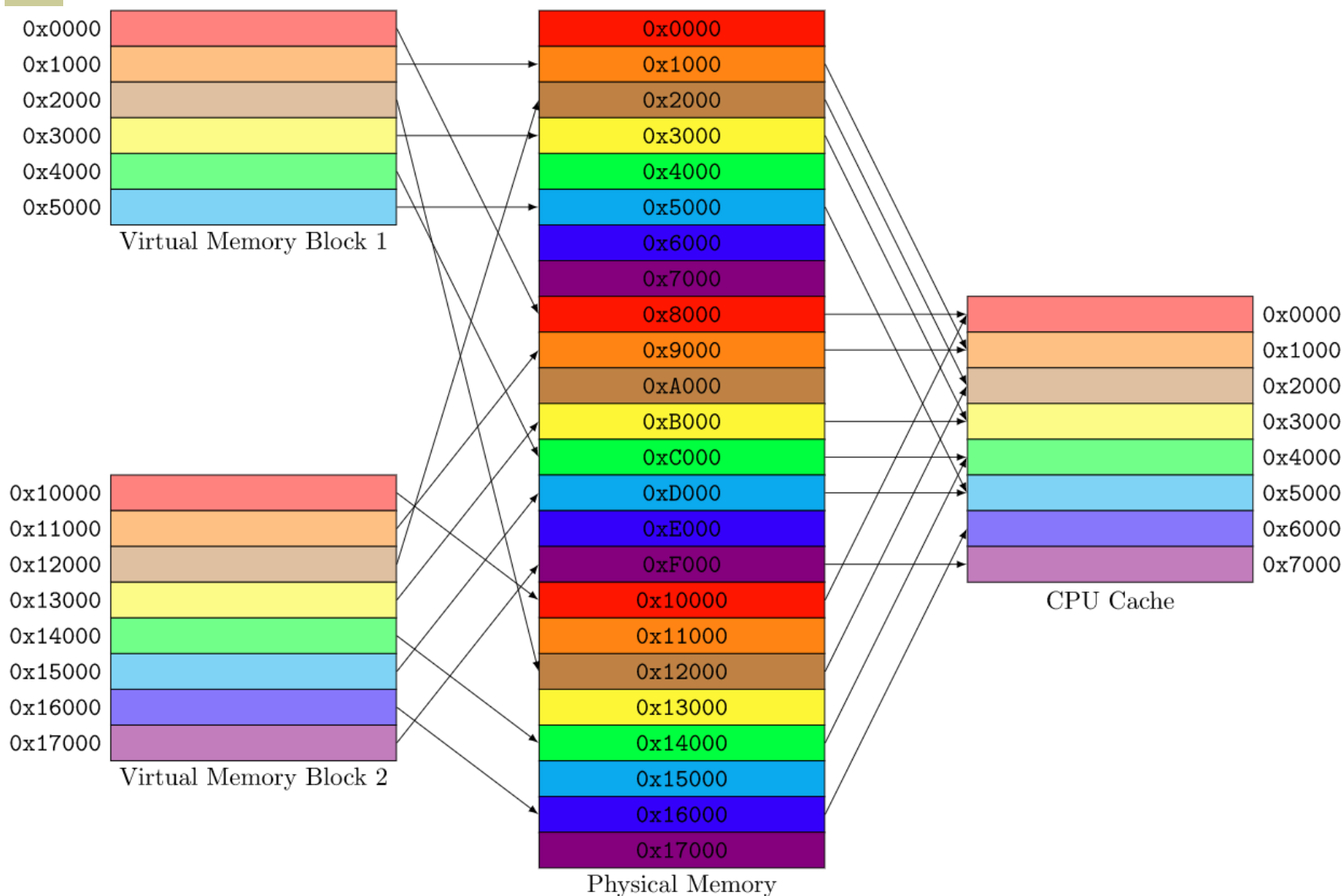
# 三种情况下不命中率的比较

不命中率



Cache容量/B

# 页着色举例



## 7.5.2 虚拟Cache

### ◆ 4、虚拟索引+物理标识

- **优点**：兼得虚拟Cache和物理Cache的好处
- **局限性**：Cache容量受到限制(页内位移)
- 例如：Alpha AXP 21064，Cache容量8KB，最小页大小为8KB，块大小为32B，8位索引（可直接从虚拟地址的页内位移中得到）
- 更大的Cache，提高相联度。 $\text{Cache容量} \leq \text{页大小} \times \text{相联度}$

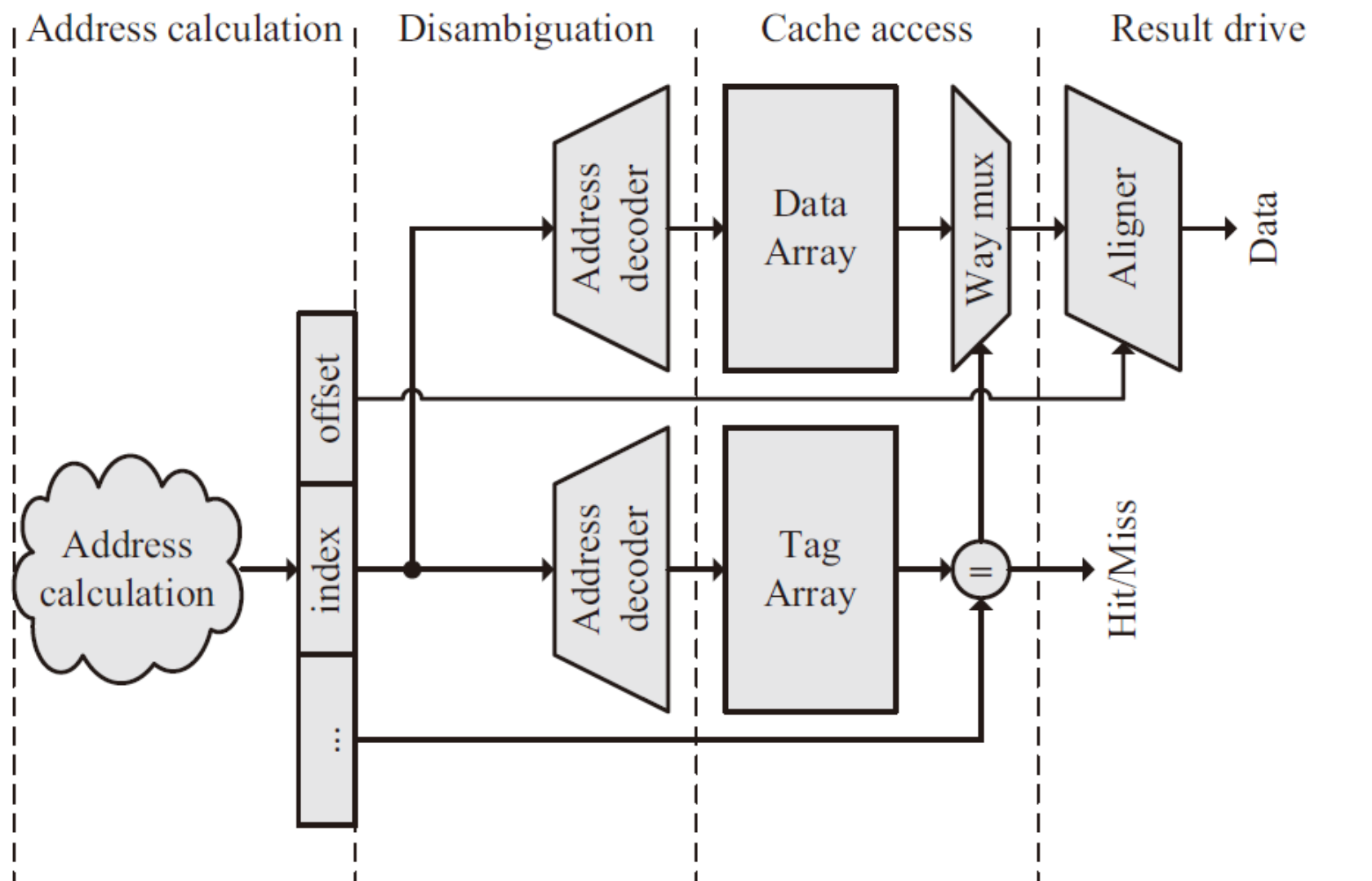
### ◆ 5、举例：IBM3033的Cache

- 页大小=4KB                  相联度=16
- $\text{Cache容量} = 16 \times 4\text{KB} = 64\text{KB}$

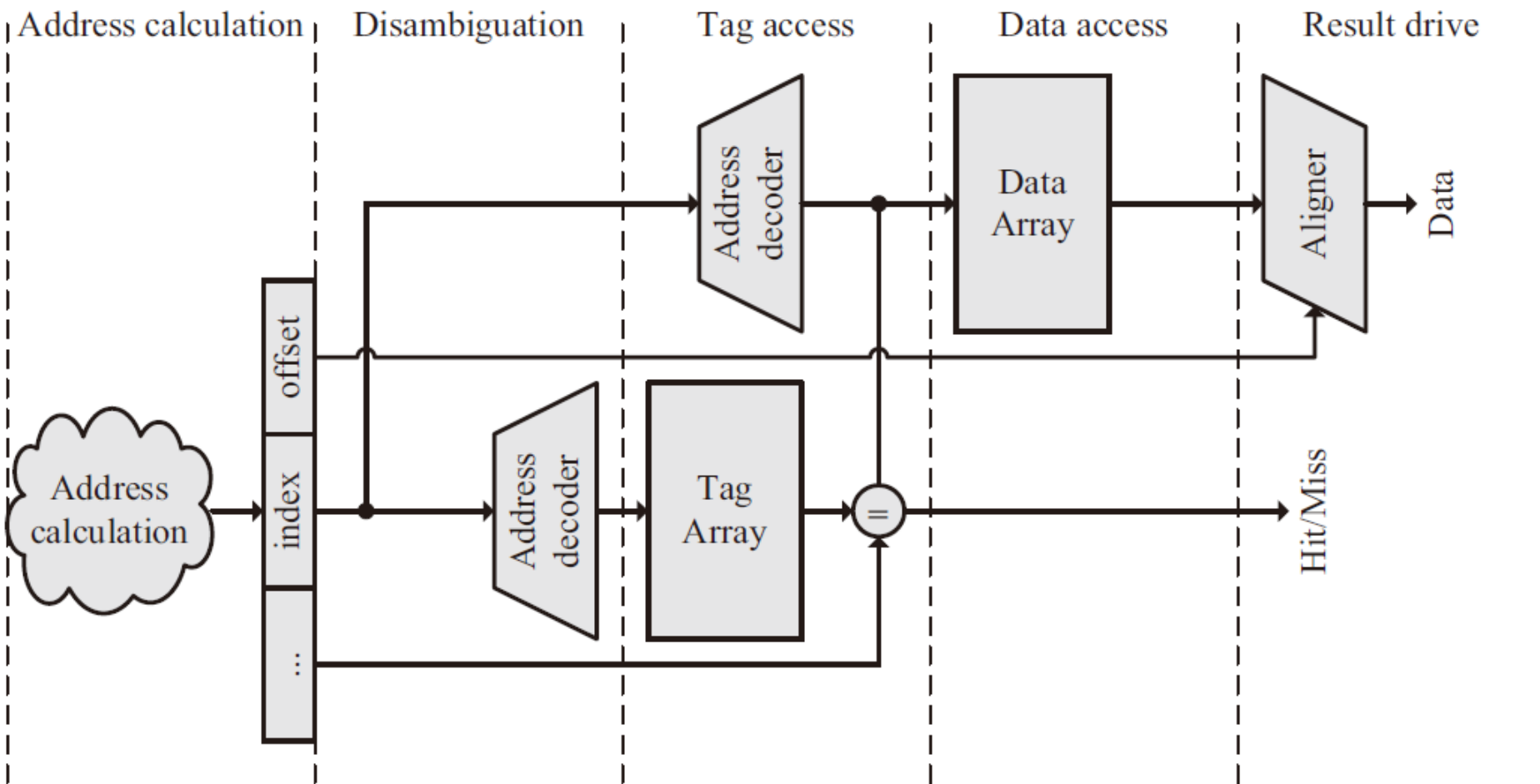
|             |    |                   |   |
|-------------|----|-------------------|---|
| 31          | 12 | 11                | 0 |
| 页地址<br>地址标识 |    | 页内位移<br>索引   块内位移 |   |

## 7.5.3 Cache访问流水化

- ◆ 对第一级Cache的访问按流水方式组织，实现Cache访问的流水化
  - 提高Cache的带宽，有利于采用高相联度的缓存
- ◆ L1 cache的访问由多个时钟周期构成
  - Pentium: 1 cycle
  - Pentium Pro – Pentium III: 2 cycles
  - Pentium 4 – Core i7: 4 cycles
  - IBM Power7: 3 cycles
- ◆ 缺点：增加流水线的段数
  - 增加了分支预测错误造成的额外开销
  - 增加了Load指令与要使用其结果的指令间的latency
  - 增加了I-Cache和D-Cache的延时



## ◆ Parallel tag and data array access pipeline



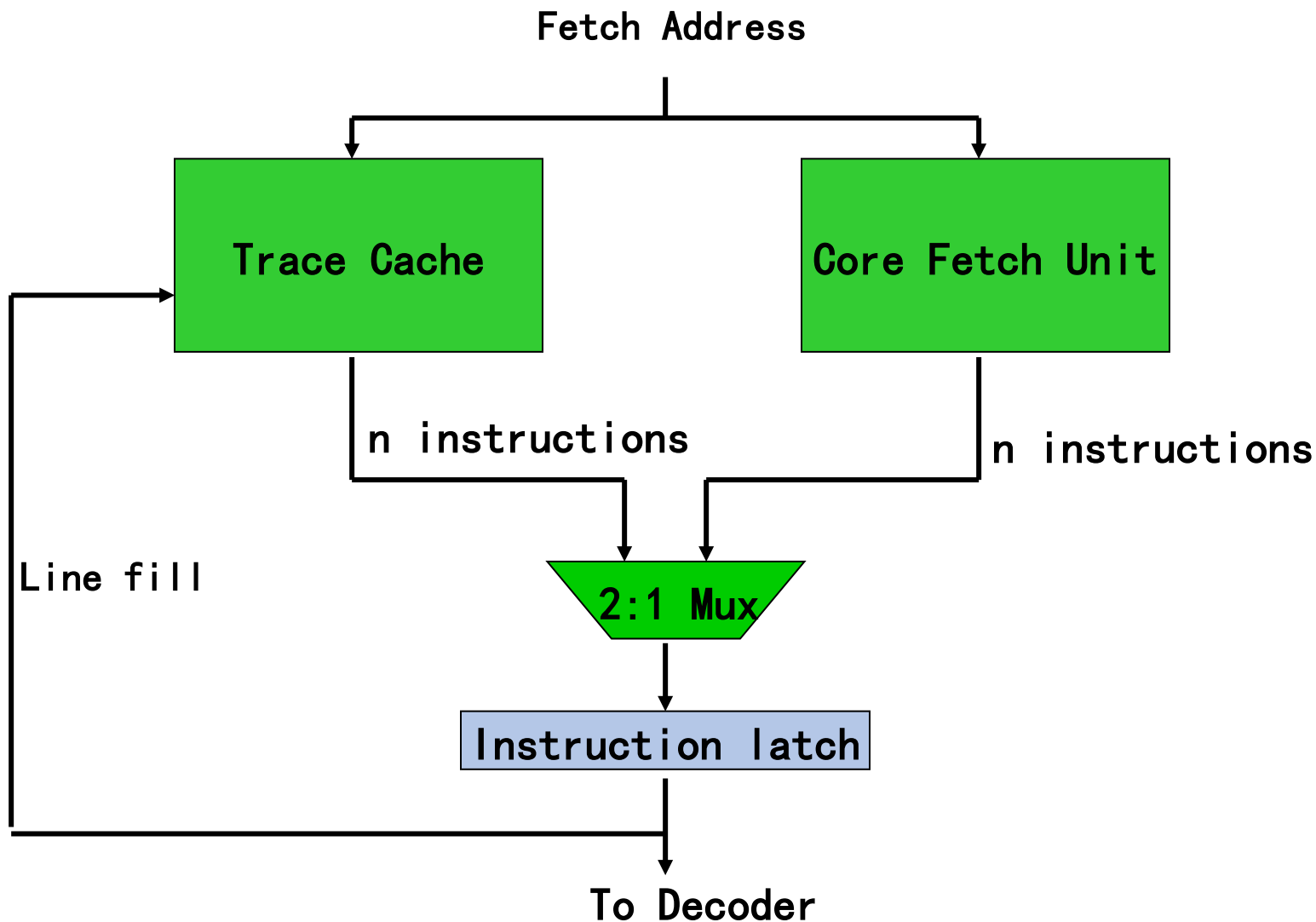
## ◆ Serial tag and data array access pipeline

## 7.5.4 踪迹 Cache

- ◆ 1、开发指令级并行性所遇到的一个挑战是：
  - 当要每个时钟周期流出超过4条指令时，要提供足够多条彼此互不相关的指令是很困难的。
- ◆ 2、一个解决方法：采用踪迹Cache
  - 存放CPU所执行的动态指令序列
  - 包含了由分支预测展开的指令，该分支预测是否正确需要在取到该指令时进行确认。
- ◆ 3、优缺点
  - 地址映像机制复杂，
  - 相同的指令序列有可能被当作条件分支的不同选择而重复存放，
  - 能够提高指令Cache的空间利用率。

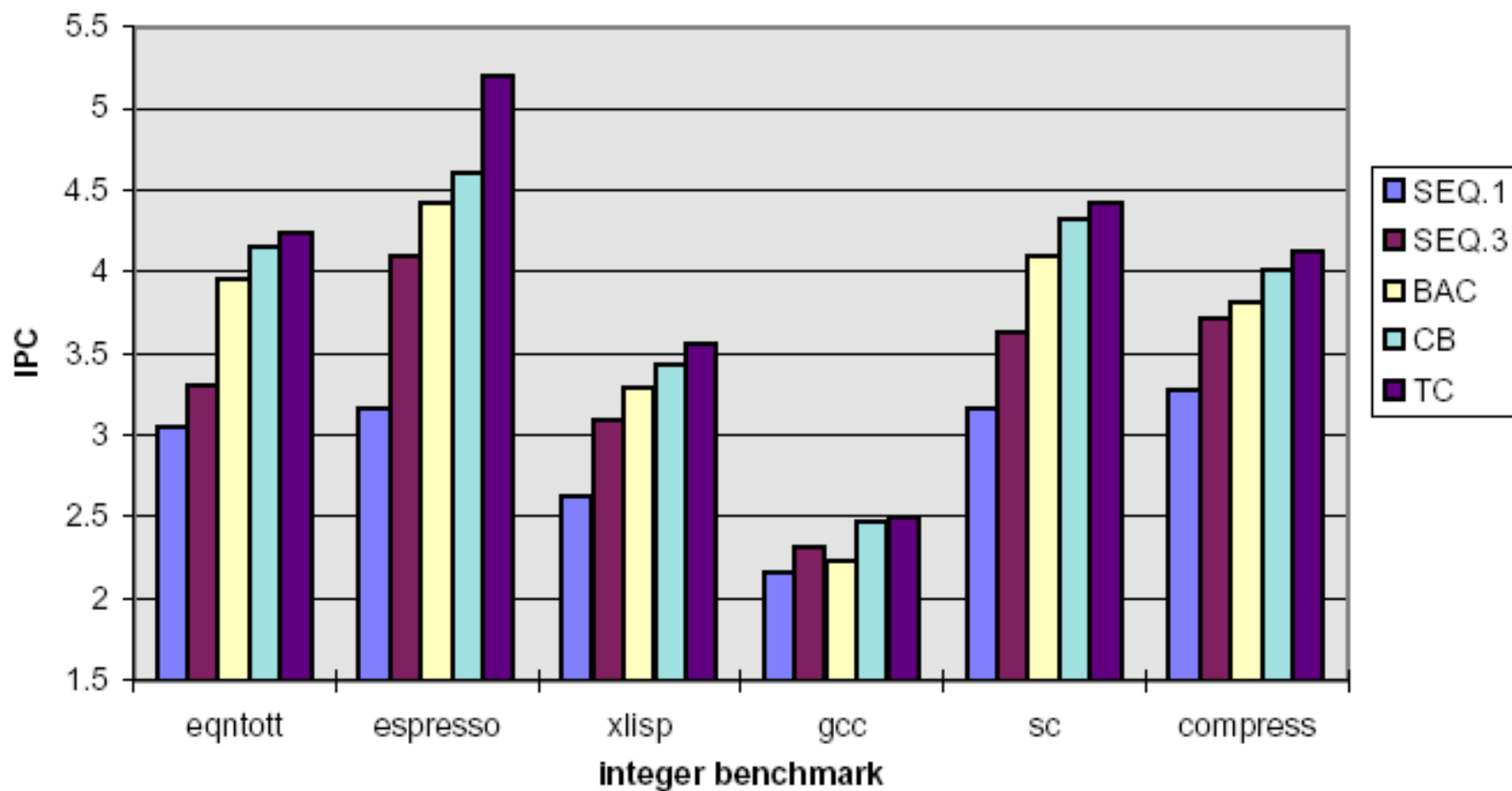


# Trace Cache – Dissection

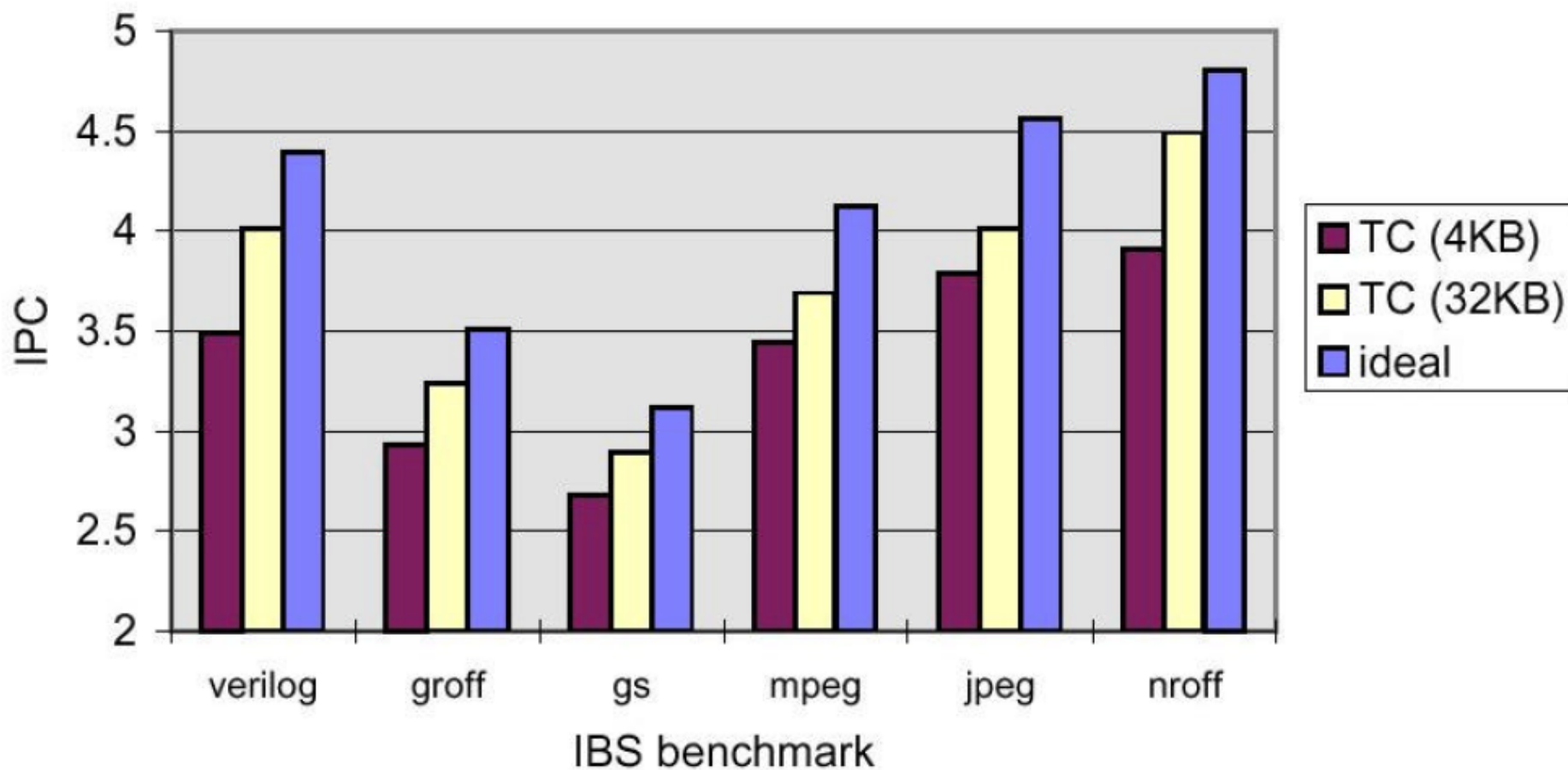


# Trace Cache性能

IPC for the Various Fetch Mechanisms, Single-Cycle Latency



# Trace Cache性能



## ◆ Comparing Trace Cache to Ideal

## 7.5.5 Cache优化技术总结

| 优化技术         | 失效率      | 失效开销 | 命中时间 | 硬件复杂度 | 说明                               |
|--------------|----------|------|------|-------|----------------------------------|
| 增加块大小        | 先+<br>后- | —    |      | 0     | 实现容易；Pentium 4的第二级Cache采用了128B的块 |
| 增加Cache容量    | +        |      |      |       | 被广泛采用，特别是第二级Cache                |
| 提高相联度        | +        |      | —    | 1     | 被广泛采用                            |
| Victim Cache | +        |      |      | 2     | AMD Athlon采用了8个项的Victim Cache    |
| 伪相联Cache     | +        |      |      | 2     | MIPS R10000的第二级Cache采用           |
| 硬件预取指令和数据    | +        |      |      | 2~3   | 许多机器预取指令，UltraSPARC III预取数据      |

## 7.5.5 Cache优化技术总结

| 优化技术             | 失效率 | 失效开销 | 命中时间 | 硬件复杂度 | 说明                                     |
|------------------|-----|------|------|-------|----------------------------------------|
| 编译器控制的预取         | +   |      |      | 3     | 需同时采用非阻塞Cache；<br>有几种微处理器提供了对这种预取的支持   |
| 用编译技术减少Cache失效次数 | +   |      |      | 0     | 向软件提出了新要求；有些机器提供了编译器选项                 |
| 使读失效优先于写         |     | +    | -    | 1     | 在单处理机上实现容易，被广泛采用                       |
| 写缓冲归并            |     | +    |      | 1     | 与写直达合用，广泛应用，<br>例如21164，UltraSPARC III |
| 尽早重启动和关键字优先      |     | +    |      | 2     | 被广泛采用                                  |
| 非阻塞Cache         |     | +    |      | 3     | 在支持乱序执行的CPU中使用                         |

## 7.5.5 Cache优化技术总结

| 优化技术                | 失效率 | 失效开销 | 命中时间 | 硬件复杂度 | 说明                                              |
|---------------------|-----|------|------|-------|-------------------------------------------------|
| 两级Cache             |     | +    |      | 2     | 硬件代价大；两级Cache的块大小不同时实现困难；被广泛采用                  |
| 容量小且结构简单的Cache      | -   |      | +    | 0     | 实现容易，被广泛采用                                      |
| 对Cache进行索引时不必进行地址变换 |     |      | +    | 2     | 对于小容量Cache来说实现容易，已被Alpha 21164和UltraSPARC III采用 |
| 流水化Cache访问          |     |      | +    | 1     | 被广泛采用                                           |
| 踪迹Cache             |     |      | +    | 3     | Pentium 4 采用                                    |

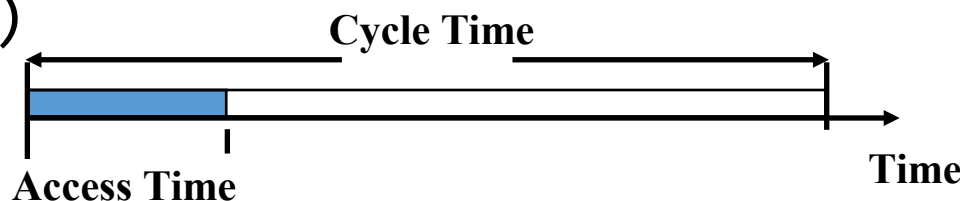
## 7.6 并行主存系统

### ◆ 存储器的访问源

- 取指令、取操作数、写操作数和I/O

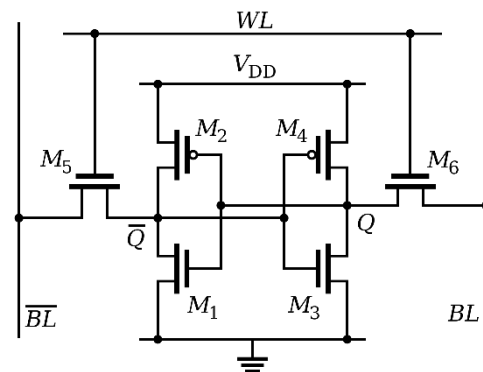
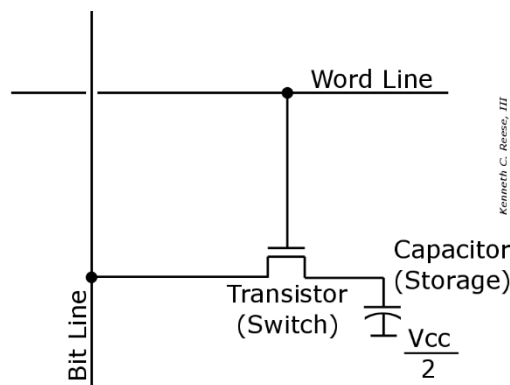
### ◆ 存储器性能指标

- 容量、速度、带宽和每位价格
- 访问时间 (Access Time)
- 存储周期 (Cycle Time)



### ◆ 种类：DRAM和SRAM

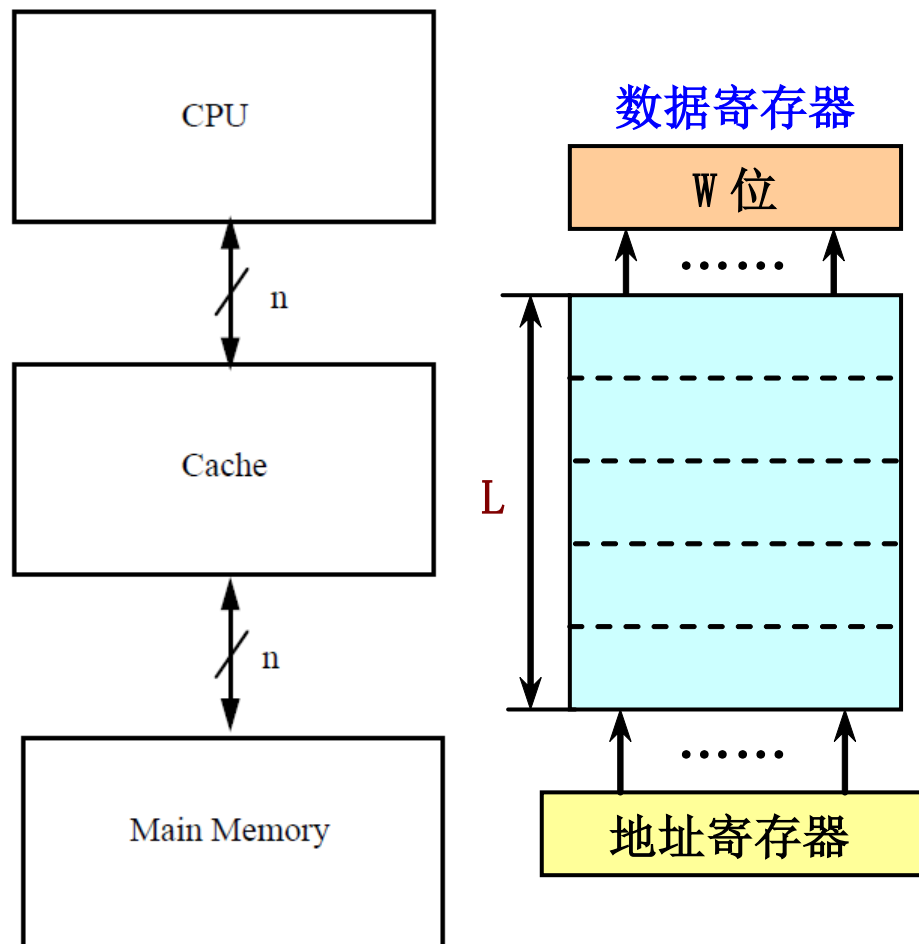
- Memory: DRAM
- Cache: SRAM



# 普通存储器

- ◆ 一个单体单字宽的存储器
  - 字长与CPU的字长相同
  - 每一次只能访问一个存储字。
- ◆ 假设该存储器的访问周期是 $T_M$ ，字长为 $W$ 位，则其带宽为：

$$B_M = \frac{W}{T_M}$$





## 7.6 并行主存系统

- ◆ 存储器体系中的频带平衡问题：
  - 1) CPU访问存储器的字长和速度；
  - 2) 各级存储器间的数据交换效率。
- ◆ 问题的背景：
  - 存储器访问的速度能否跟上系统的要求，是影响整个计算机系统性能极为重要的关键问题，也是系统结构设计者的主要问题。
- ◆ 举例说明：
  - 设一台计算机的执行速度为200MIPs(2亿指令/秒)，若采用普通DRAM存储器，能否满足CPU的访存需求？

## 7.6 并行主存系统

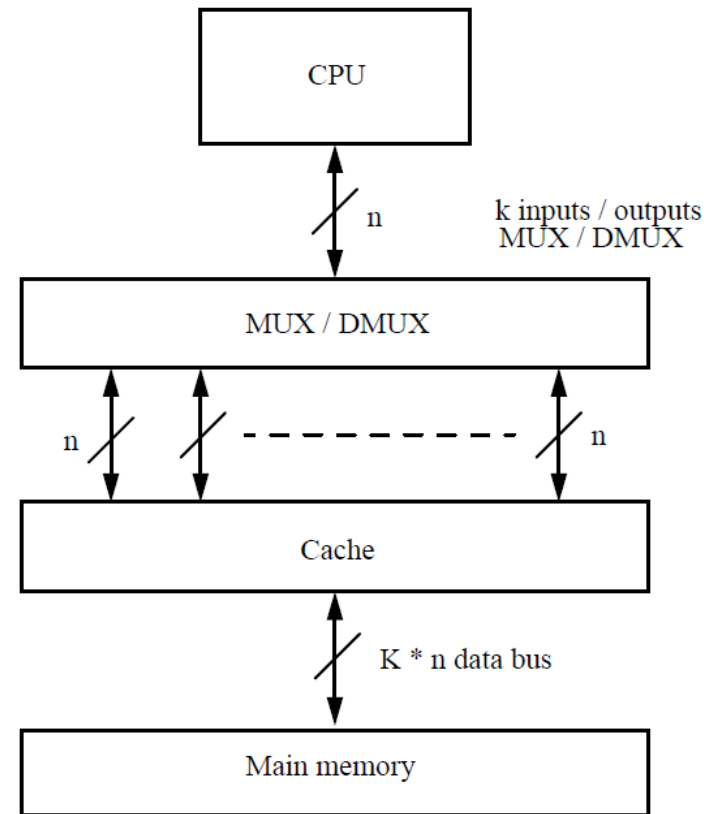
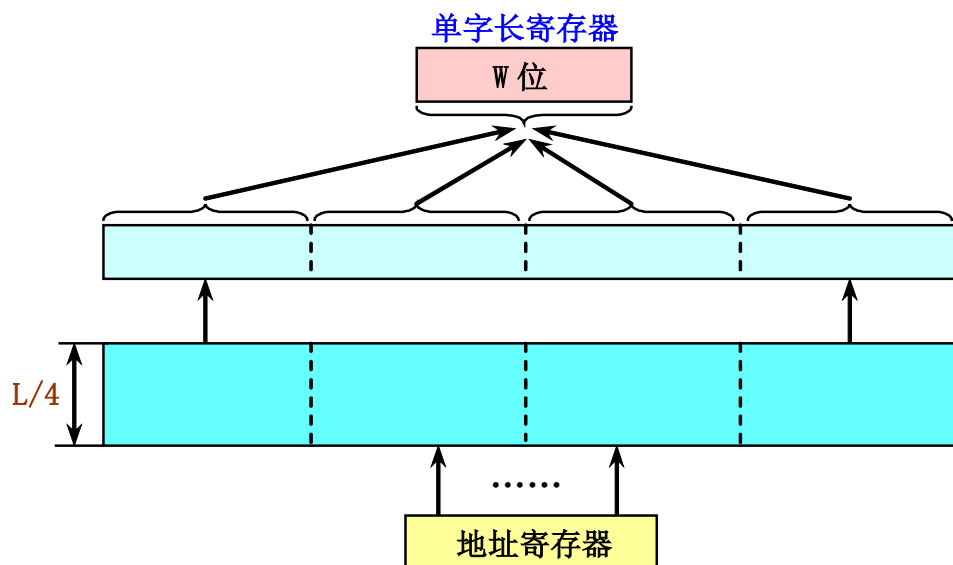
- ◆ 解：CPU取指令：200MW/S(设每条指令长为一个字)  
CPU取/存操作数：400MW/S(平均每条指令访问周期2个操作数)  
各种I/O设备访问存储器：5MW/S  
要求存储器的总频宽为605MW/S  
要求 $T \leq 16.5\text{ns}$ 。这与目前的DRAM的访存周期是不符的，也就是说达不到这一要求的。
- ◆ 并行主存系统是在一个访存周期内能并行访问多个存储字的存储器。
  - 能有效地提高存储器的带宽。
- ◆ 两种方式
  - ◆ 单体多字存储器
  - ◆ 多体交叉存储器

## 7.6.1 单体多字存储器

### ◆ 一个单体m字存储器

- $m=4$
- 存储器能够每个存储周期读出m个CPU字。因此其最大带宽提高到原来的m倍

$$B_M = m \times \frac{W}{T_M}$$



**单体多字存储器的实际带宽比最大带宽小**

## 7.6.1 单体多字存储器

### ◆ 优缺点

- 优点：实现简单
- 缺点：访存效率不高

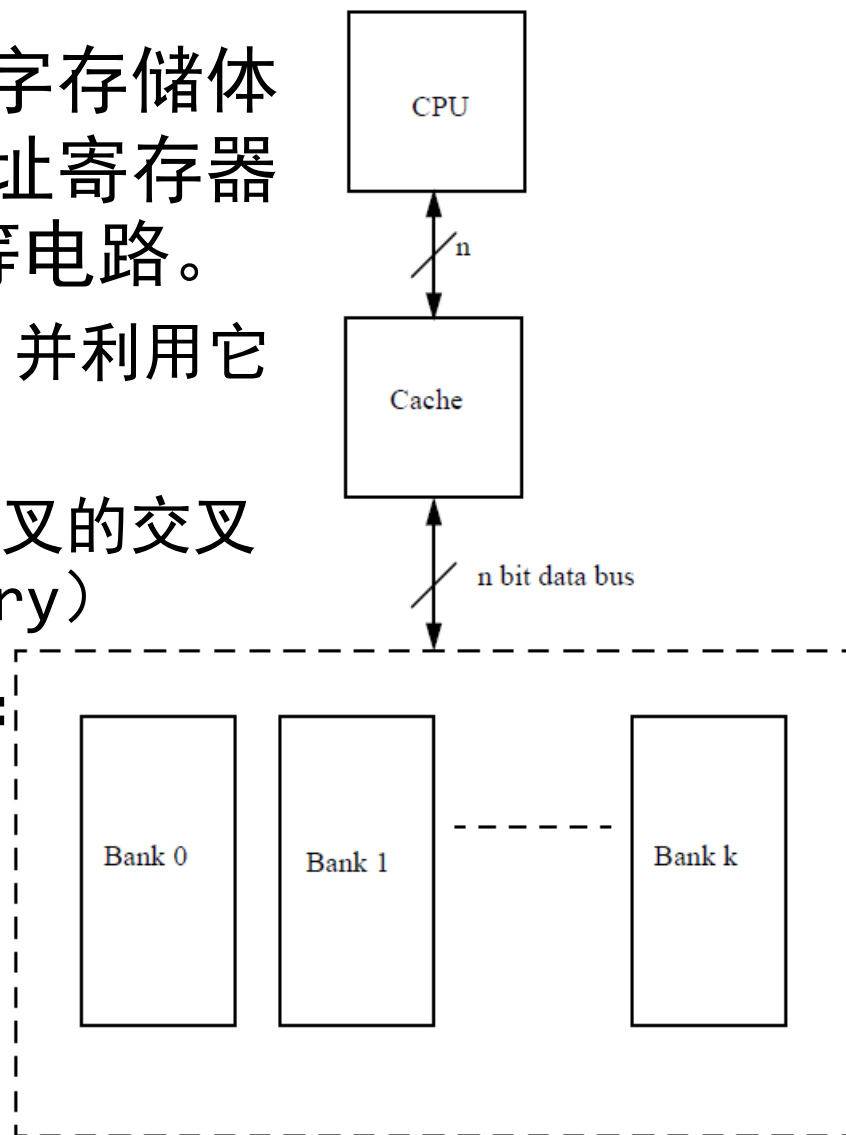
### ◆ 原因：

- 如果一次读取的 $m$ 个指令字中有分支指令，而且分支成功，那么该分支指令之后的指令是无用的。
- 一次取出的 $m$ 个数据不一定都是有用的。另一方面，当前执行指令所需要的多个操作数也不一定正好都存放在同一个长存储字中。
- 增加一个多路选择器，写入有可能变得复杂。
- 当要读出的数据字和要写入的数据字处于同一个长存储字内时，读和写的操作就无法在同一个存储周期内完成。

### ◆ 举例：DEC的Alpha Axp21064：256位宽

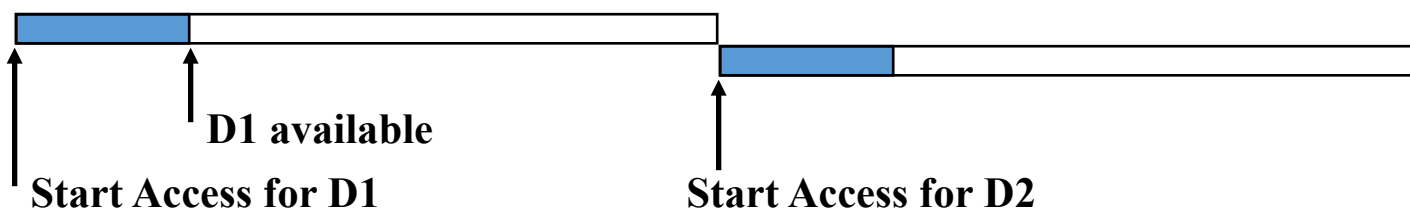
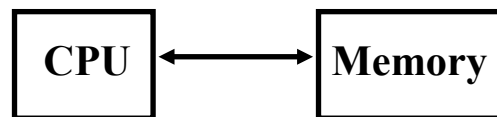
## 7.6.2 多体交叉存储器

- ◆ 多体交叉存储器：由多个单字存储体构成，每个体都有自己的地址寄存器以及地址译码和读/写驱动等电路。
  - 在存储系统中采用多个DRAM，并利用它们潜在的并行性。
  - 存储器的各个体一般是按字交叉的交叉存储器（interleaved memory）
- ◆ 字交叉存储器非常适合处理：
  - Cache读失效
  - 写回法Cache中的写回

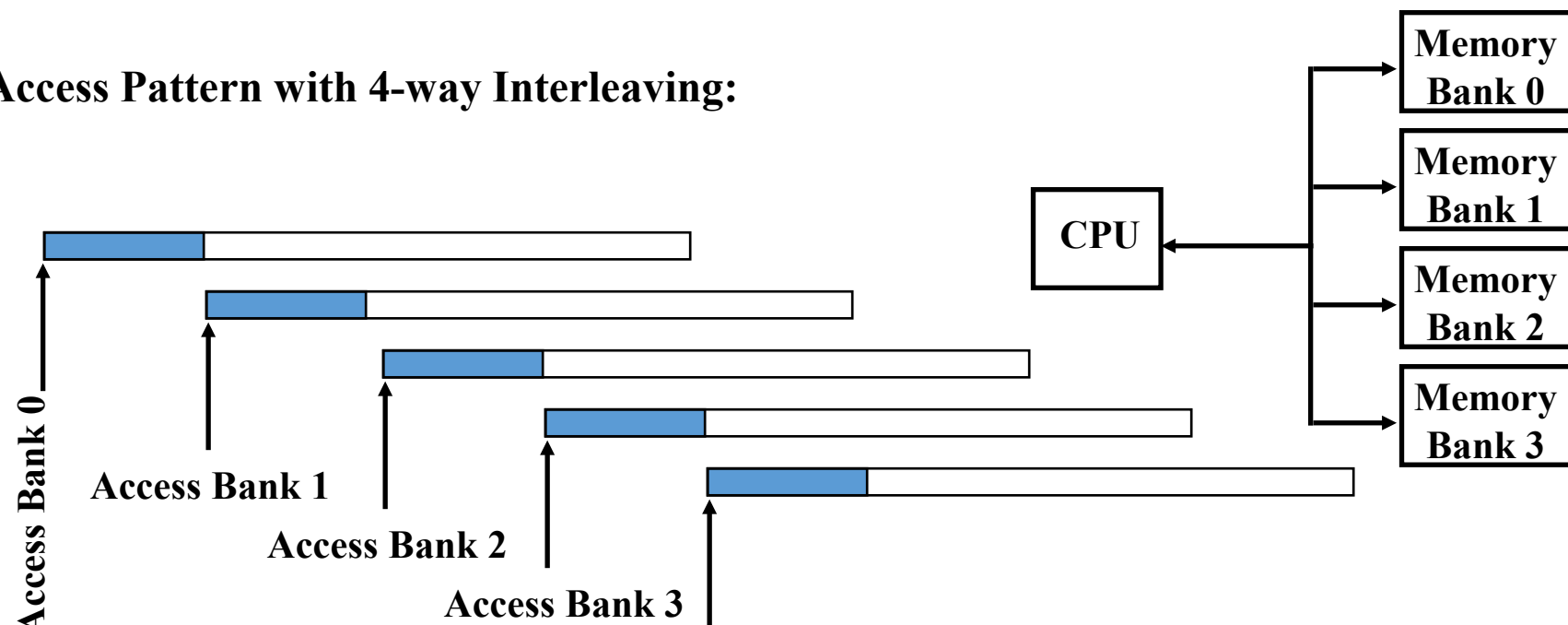


# Increasing Bandwidth - Interleaving

Access Pattern without Interleaving:



Access Pattern with 4-way Interleaving:



We can Access Bank 0 again

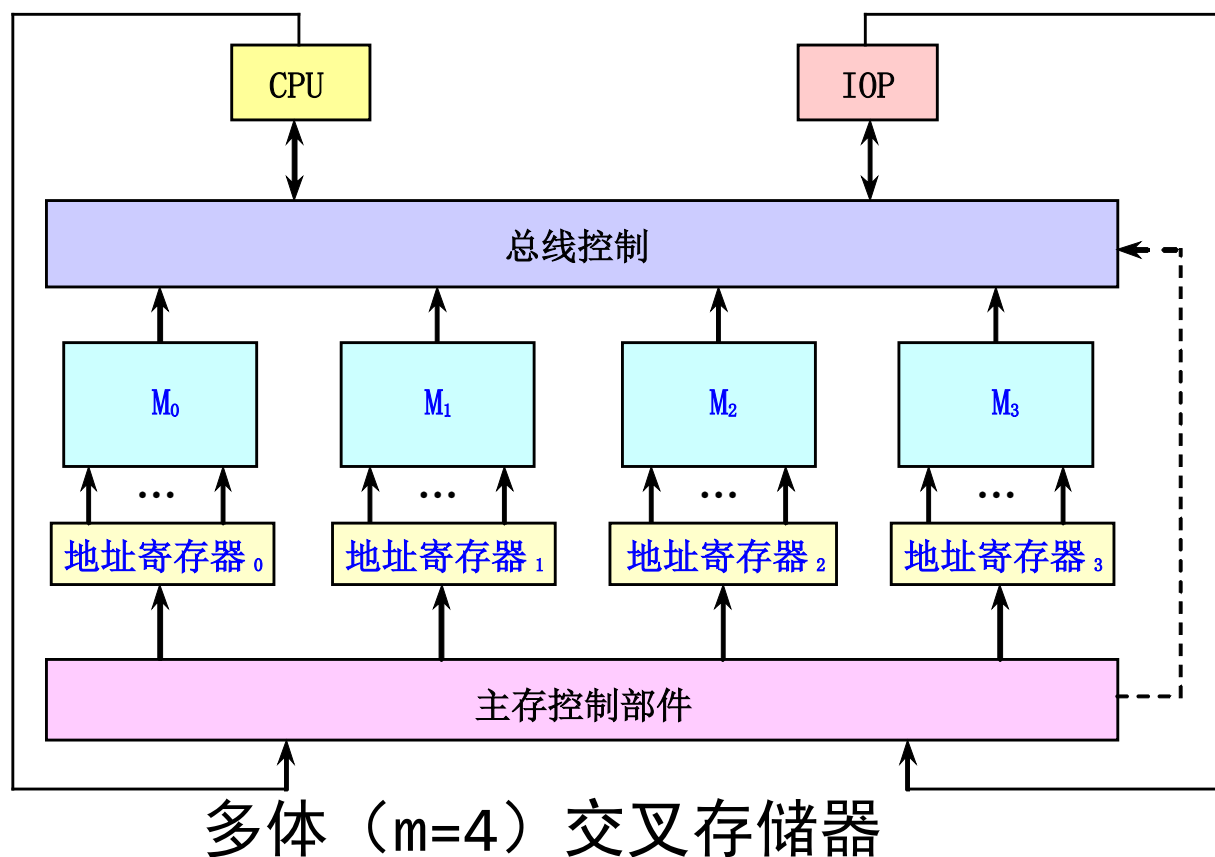
## 7.6.2 多体交叉存储器

### ◆ 问题：对多体存储器如何进行编址？

- 存储器是按顺序线性编址的。如何在二维矩阵和线性地址之间建立对应关系？

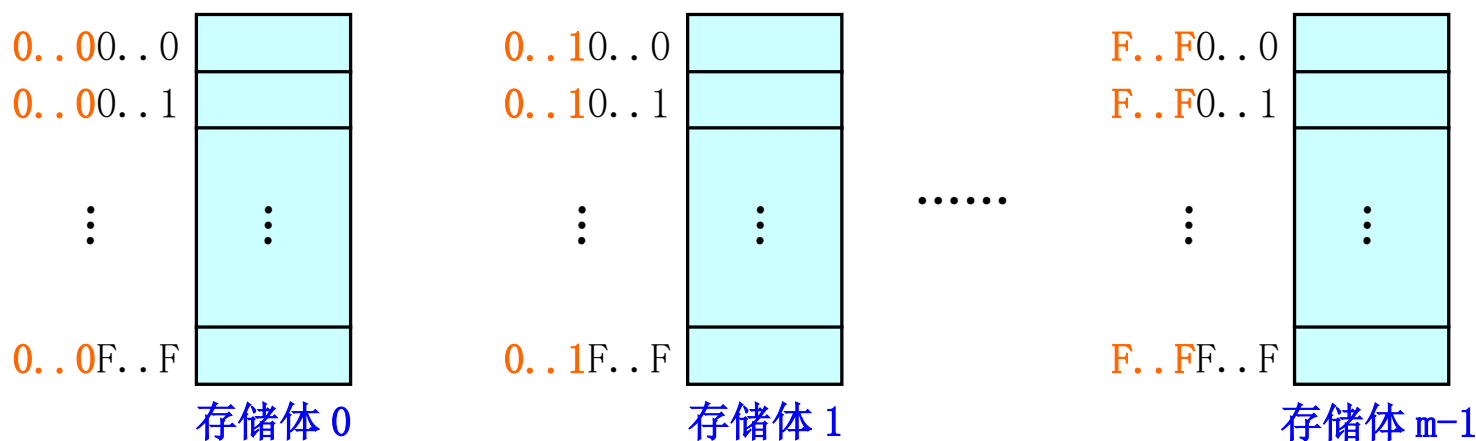
### ◆ 两种编址方法

- 高位交叉编址
  - 低位交叉编址
- 有效地解决  
访问冲突问题



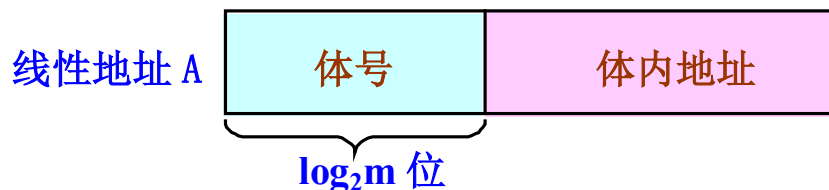
# 高位交叉编址

- ◆ 对存储单元矩阵按列优先的方式进行编址
- ◆ 特点：同一个体中的**高 $\log_2 m$ 位都是相同的** (**体号**)



- ◆ 体号为 $j$ 、体内地址为 $i$ 的单元，其线性地址为： $A = j \times n + i$
- ◆ 一个单元的线性地址为 $A$ ，则其体号 $j$ 和体内地址 $i$ 为：

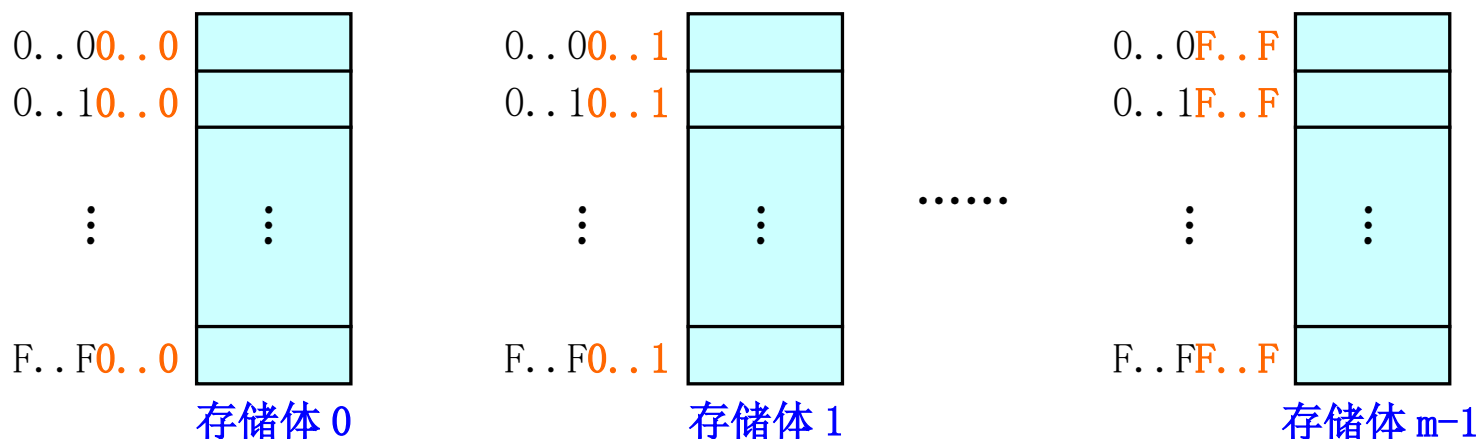
$$j = \left\lfloor \frac{A}{n} \right\rfloor \quad i = A \bmod n$$





# 低位交叉编址

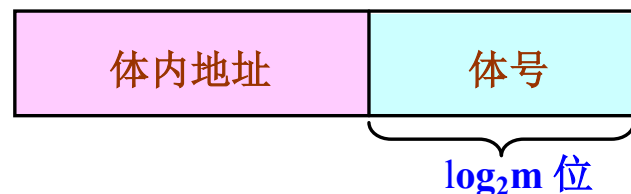
- ◆ 对存储单元矩阵按行优先进行编址
- ◆ 特点：同一个体中的**低 $\log_2 m$ 位都是相同的**



- ◆ 体号为 $j$ 、体内地址为 $i$ 的单元，其线性地址为： $A = i \times m + j$
- ◆ 一个单元的线性地址为 $A$ ，则其体号 $j$ 和体内地址 $i$ 为：

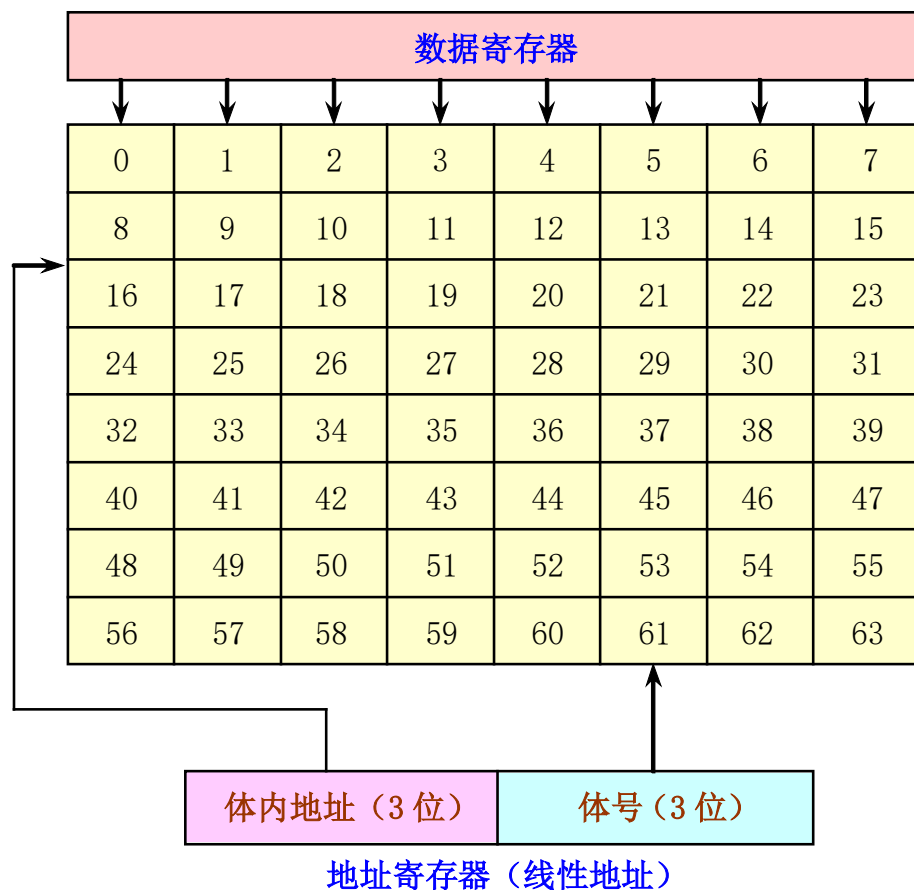
$$i = \left\lfloor \frac{A}{m} \right\rfloor \quad j = A \bmod m$$

线性地址  $A$



# 采用低位交叉编址的存储器

- ◆ 举例：由8个存储体构成、总容量为64。格子中的编号为线性地址。
- ◆ 假设8个存储体的地址是在字一级交叉的，
  - ◆ 即存储体0中每个字的地址对8取模都是0，
  - ◆ 体1中每个字的地址对8取模都是1，
  - ◆ 依此类推。



# 如何提高存储器的访问效率-举例

- ◆ 例7.9 假设某台计算机的特性及其Cache的性能为：

- (1) 块大小为1个字；
- (2) 存储器总线宽度为1个字；
- (3) Cache失效率为3%；
- (4) 平均每条指令访存1.2次；
- (5) Cache失效开销为32个时钟周期；
- (6) 平均CPI（忽略Cache失效）为2。

- **假设：**

- 1. 送地址：4个周期，
- 2. 每个字的访问时间（存取周期）为24个周期，
- 3. 传送一个字的数据需要4个周期

- ◆ 试问多体交叉和增加存储器宽度对提高性能各有何作用？
- ◆ 如果当把Cache块大小变为2个字时，失效率降为2%；块大小变为4个字时，失效率降为1%。根据前面给出的访问时间，求在采用2路、4路多体交叉存取以及将存储器和总线宽度增加一倍时，性能分别提高多少？

# 如何提高存储器的访问效率-举例

## ◆ 【解】

- ◆ 在改变前的计算机中，Cache块大小为一个字，
- ◆ 考虑Cache失效开销时，其CPI为：
$$2 + (1.2 \times 3\% \times 32) = 3.15$$
- ◆ 多体交叉和增加存储器宽度性能提升的影响？  
无

# 如何提高存储器的访问效率-举例

- ◆ 当将块大小增加为2个字时，
- ◆ 在下面三种情况下的CPI分别为：
- ◆ (1) 32位总线和存储器，不采用多体交叉：

$$2 + (1.2 \times 2\% \times 2 \times 32) = 3.54$$

**性能降低了11%**

- ◆ (2) 32位总线和存储器，采用多体交叉：

$$2 + (1.2 \times 2\% \times (4 + 24 + 2 \times 4)) = 2.86$$

**性能提高了10%**

- ◆ (3) 64位总线和存储器，不采用多体交叉：

$$2 + (1.2 \times 2\% \times 1 \times 32) = 2.77$$

**性能提高了14%**

# 如何提高存储器的访问效率-举例

- ◆ 当将块大小增加为4个字时，
- ◆ 在下面三种情况下的CPI分别为：
- ◆ (1) 32位总线和存储器，不采用多体交叉：

$$2 + (1.2 \times 1\% \times 4 \times 32) = 3.54$$

**性能降低了11%**

- ◆ (2) 32位总线和存储器，采用多体交叉：

$$2 + (1.2 \times 1\% \times (4 + 24 + 4 \times 4)) = 2.53$$

**性能提高了25%**

- ◆ (3) 64位总线和存储器，不采用多体交叉：

$$2 + (1.2 \times 1\% \times 2 \times 32) = 2.77$$

**性能提高了14%**

## 7.6.3 避免存储体冲突

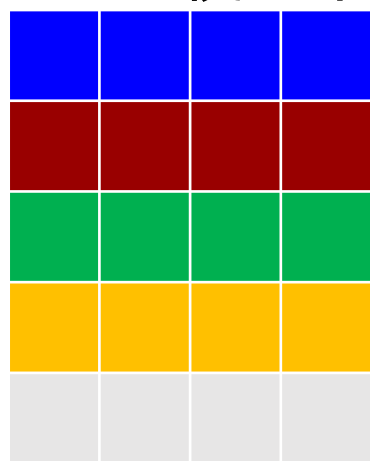
- ◆ 体冲突：两个访问请求要访问同一个存储体。
  - 在传统的多体交叉结构中，顺序访问被处理的很好，不会发生冲突。地址相差奇数值的访存也是如此。
  - 但当地址相差偶数值时，冲突的频度就增加了。
- ◆ 减少体冲突次数的一种方法：采用许多体
  - 在较大规模的机器中使用
  - 例如，NEC SX/3最多可使用128个体
- ◆ 假设有128个存储体，按字交叉方式工作
  - 因为512是128的整数倍，同一列中的所有元素都在同一个体内，该程序都会在数据Cache不命中时暂停

```
int x[256][512];
for (j=0; j<512; j=j+1)
 for (i=0; i<256; i=i+1)
 x[i][j] = 2 * x[i][j];
```

# 解决体冲突的方法

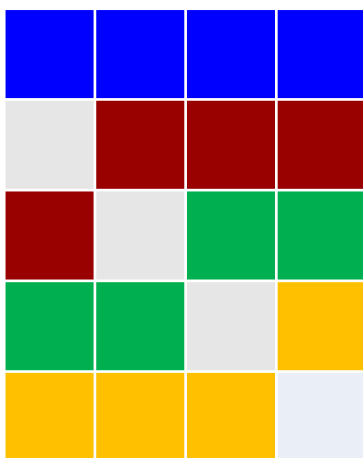
## ◆ 软件方法(编译器)

- 循环交换优化
- 扩展数组的大小, 使之不是2的幂



重新定义:  
`arr[4][5]`

举例:  
`arr[4][4]`



## ◆ 硬件方法

- 使体数为素数
- 体内地址  $i$ :  
$$i = A \bmod n$$

| 体内地址 | 存储体  |    |    |      |    |    |
|------|------|----|----|------|----|----|
|      | 顺序交叉 |    |    | 取模交叉 |    |    |
|      | 0    | 1  | 2  | 0    | 1  | 2  |
| 0    | 0    | 1  | 2  | 0    | 16 | 8  |
| 1    | 3    | 4  | 5  | 9    | 1  | 17 |
| 2    | 6    | 7  | 8  | 18   | 10 | 2  |
| 3    | 9    | 10 | 11 | 3    | 19 | 11 |
| 4    | 12   | 13 | 14 | 12   | 4  | 20 |
| 5    | 15   | 16 | 17 | 21   | 13 | 5  |
| 6    | 18   | 19 | 29 | 6    | 22 | 14 |
| 7    | 21   | 22 | 23 | 15   | 7  | 23 |

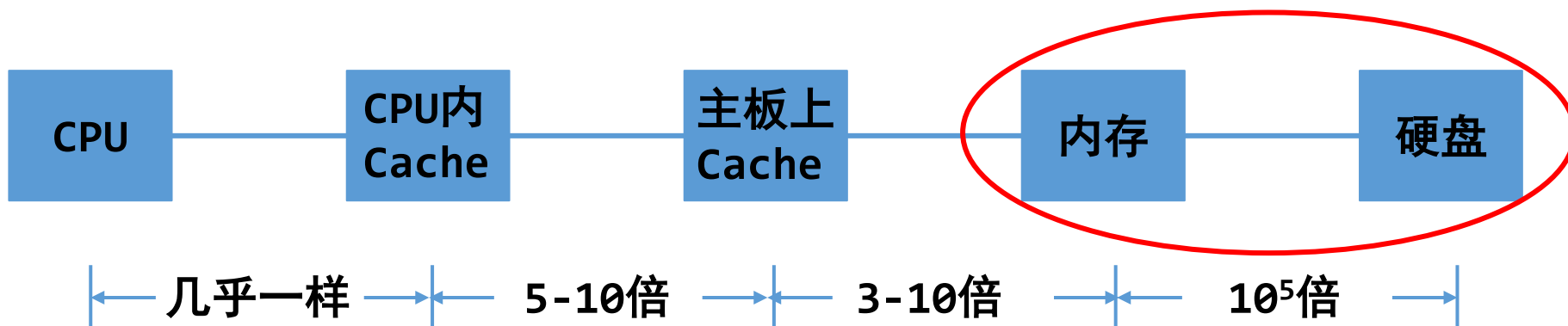


## 7.7 虚拟存储器

### ◆ 7.7.1 虚拟存储器的基本原理

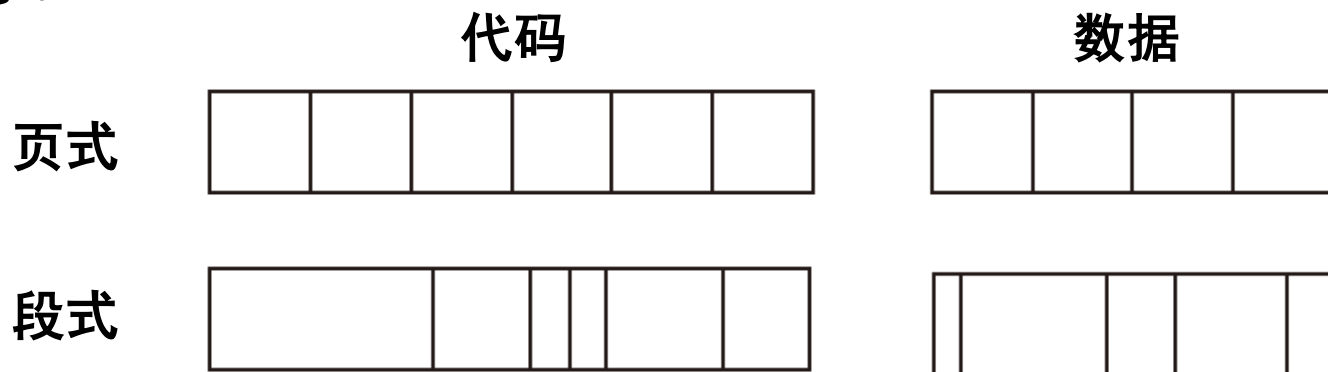
#### ◆ 背景：

- 1961年由英国曼彻斯特大学的Tom Kilburn等人提出的，其目的主要是解决主存容量不足的问题
- ◆ 和操作系统中讲到“存储管理”不同的是，本课程主要阐述虚拟存储的原理



## 7.7.1 虚拟存储器的基本原理

- ◆ 虚拟存储管理的是主存—辅助存储器这个层面上
  - 允许应用程序的大小 **超过** 主存容量
  - 目的是提高存储系统的容量
- ◆ 虚拟存储器可以分为两类：页式和段式
  - 页式虚拟存储器把空间划分为大小相同的块。（页面）
  - 段式虚拟存储器则把空间划分为可变长的块。（段）
  - 页面是对空间的机械划分，而段则往往是按程序的逻辑意义进行划分。



# Cache与VM的区别

## ◆ 目的不同

- Cache是为了提高访存速度
- VM是为了提高存储容量

## ◆ 替换的控制者不同

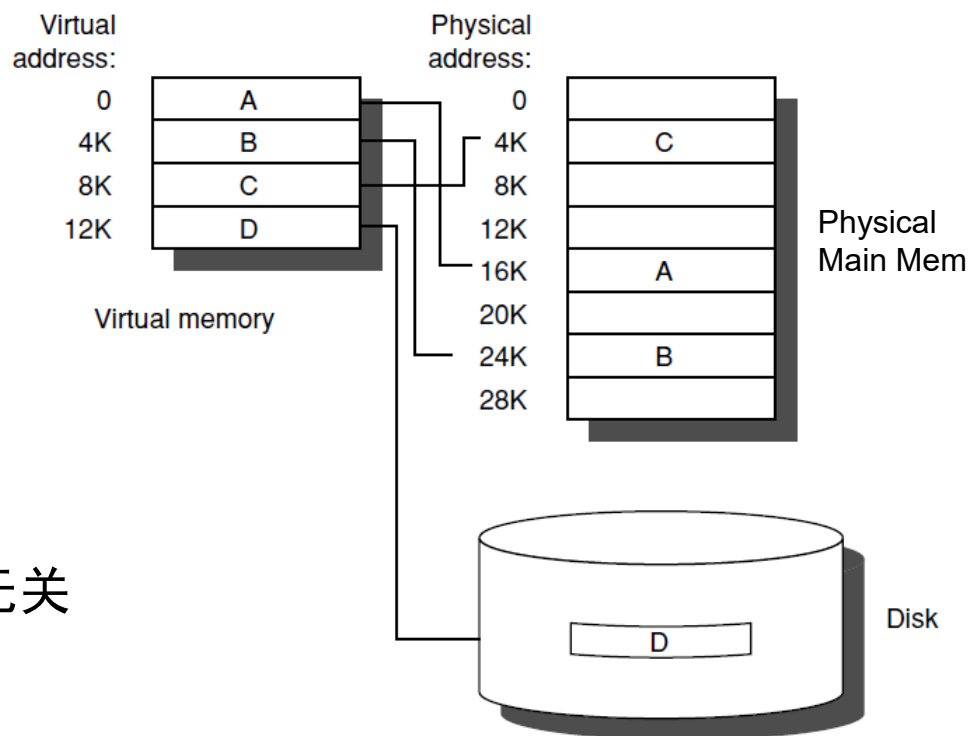
- Cache失效由硬件处理
- VM的页失效通常由OS处理

## ◆ 地址空间

- Cache的大小与CPU地址尺寸无关
- VM空间由CPU的地址尺寸确定

## ◆ 下一级存储器

- Cache下一级是主存
- VM下一级是磁盘，大多数磁盘含有文件系统，文件系统寻址与主存不同，它通常在I/O空间中，VM的下一级通常称为SWAP空间



# Cache和虚拟存储器的参数取值范围

| Parameter         | First-level cache                                        | Virtual memory                                             |
|-------------------|----------------------------------------------------------|------------------------------------------------------------|
| Block (page) size | 16–128 bytes                                             | 4096–65,536 bytes                                          |
| Hit time          | 1–3 clock cycles                                         | 100–200 clock cycles                                       |
| Miss penalty      | 8–200 clock cycles                                       | 1,000,000–10,000,000 clock cycles                          |
| (access time)     | (6–160 clock cycles)                                     | (800,000–8,000,000 clock cycles)                           |
| (transfer time)   | (2–40 clock cycles)                                      | (200,000–2,000,000 clock cycles)                           |
| Miss rate         | 0.1–10%                                                  | 0.00001–0.001%                                             |
| Address mapping   | 25–45-bit physical address<br>to 14–20-bit cache address | 32–64-bit virtual address to<br>25–45-bit physical address |

## ◆ 典型容量数据

- 第一级数据Cache容量为1MB,
- 物理内存容量为256MB~1TB

## ◆ 从表中看（与Cache参数相比）

- 除了失效率较低，其他参数都比Cache大

# 虚拟存储的四个问题

## ◆ 映象规则

- 选择策略：低失效率和复杂的映象算法，还是简单的映射方法，高失效率
- 由于失效开销很大，一般选择低失效率方法，即全相联映射

## ◆ 查找算法 — 用附加数据结构

- 固定页大小 — 用页表
  - $VPN \rightarrow PPN$
  - Tag标识该页是否在主存
- 可变长段 — 段表
  - 段表中存放所有可能的段信息
  - 段号  $\rightarrow$  段基址 再加段内偏移量
  - 可能存在许多小尺寸段

# 虚拟存储的四个问题

## ◆ 替换规则

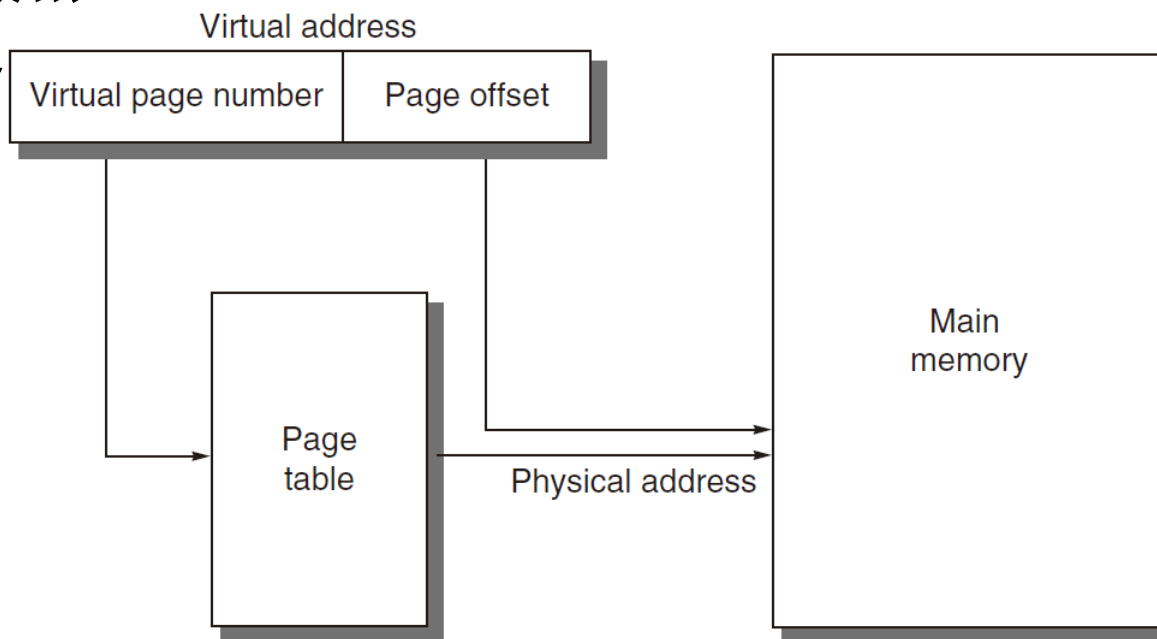
- LRU是最好的
- 但真正的LRU方法，硬件代价较大
- 用硬件简化，通过OS来完成
  - 为了帮助OS寻找LRU页，每个页面设置一个use bit
  - 当访问主存中一个页面时，其use bit置位
  - OS定期复位所有使用位，这样每次复位之前，使用位的值就反映了从上次复位到现在的这段时间中，哪些页曾被访问过。
  - 当有失效冲突时，由OS来决定哪些页将被换出去。

## ◆ 写策略

- 总是用写回法，因为访问硬盘速度很慢。

# 虚拟地址转换过程

- ◆ 使用页表完成虚拟地址向物理地址的转换
- ◆ 页面选择较大的优点
  - 减少了页表的大小
  - 如果局部性较好，可以提高命中率
- ◆ 页面选择较大的缺点
  - 内存中的碎片较多  
内存利用率低
  - 进程启动时间长
  - 失效开销加大



## 7.7.2 快速地址转换技术

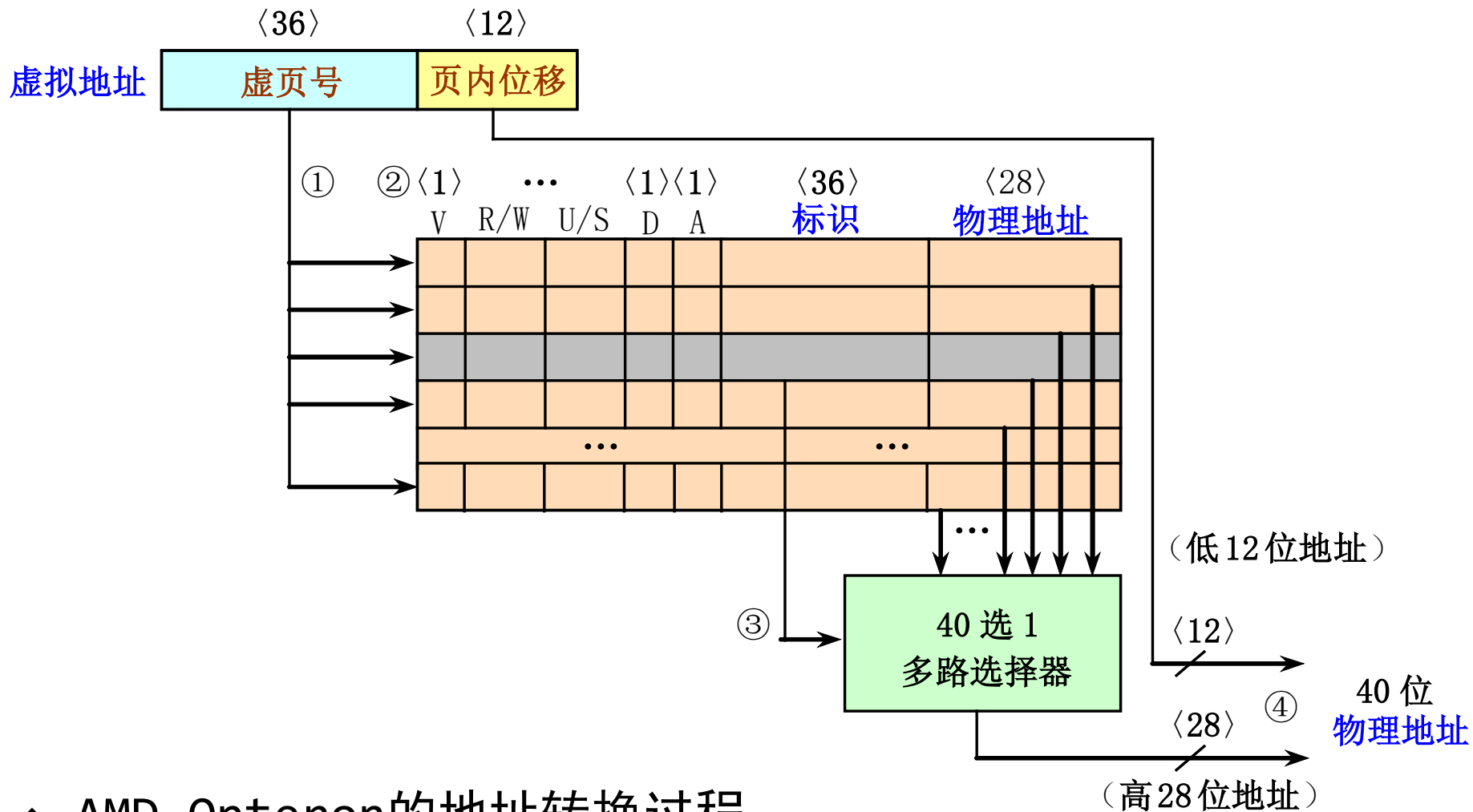
- ◆ 页表一般很大，存放在主存中。
  - 导致每次访存可能要两次访问主存，一次读取页表项，一次读写数据
  - 解决办法：采用TLB(快表)
- ◆ 地址变换缓冲器TLB
  - TLB是一个专用的高速缓冲器，用于存放近期经常使用的页表项；
  - TLB中的内容是页表部分内容的一个副本；
  - TLB也利用了局部性原理。
- ◆ TLB必须在片内
  - 速度至关重要
  - TLB过小，意义不大
  - TLB过大，代价较高
  - 相联度较高（容量小）



## 7.7.2 快速地址转换技术

- ◆ TLB中的项由两部分构成：标识和数据
  - 标识中存放的是虚地址的一部分。
  - 数据部分中存放的则是物理页帧号、有效位、存储保护信息、使用位、修改位等。
- ◆ AMD Opteron的数据TLB的组织结构
  - 包含40个项
  - 采用全相联映像
- ◆ 一般TLB比Cache的标识存储器更小、更快。
  - 保证TLB的读出操作不会使Cache的命中时间延长。
- ◆ 快表和慢表间的关系：
  - 快表（TLB）只是慢表的一个副本，而且只存放了慢表中的一小部分。

## 7.7.2 快速地址转换技术

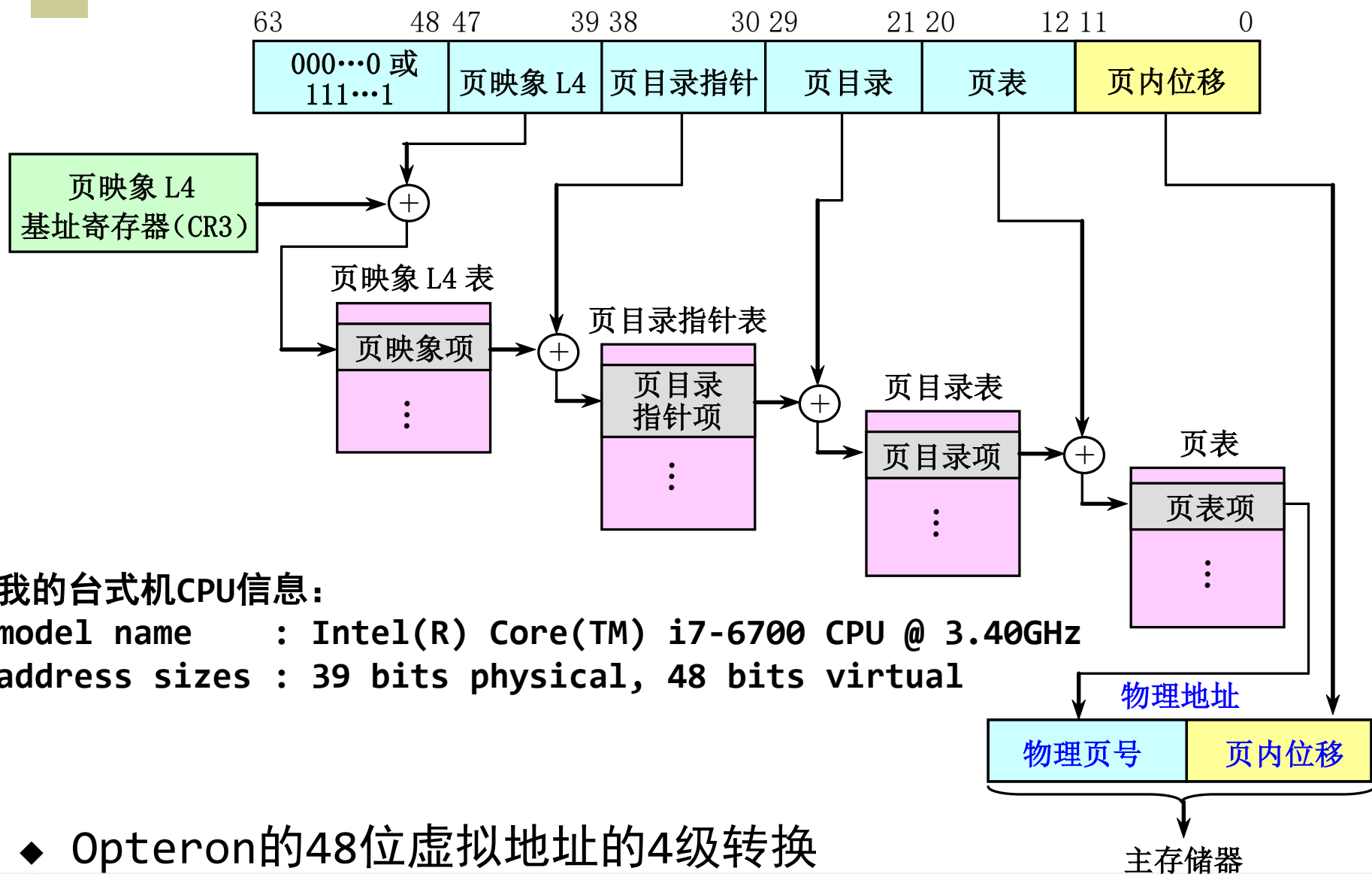


◆ AMD Opteron的地址转换过程

## 7.7.3 页式虚拟存储器实例

- ◆ 64位Opteron的存储管理
- ◆ AMD64系统结构
  - 虚拟地址： 64位    物理地址： 52位    （根据需要可精简）
  - 进行虚 → 实 地址转换时，是把64位的虚拟地址映射到52位的物理地址。
  - 要求： 64位虚拟地址中的高16位是由低48位进行符号位扩展而来的
- ◆ 采用多级分层页表结构来映射地址空间，以便使页表大小合适。
  - 分级的级数取决于虚拟地址空间的大小
  - Opteron的48位虚拟地址的4级转换
  - 每个分级页表的偏移量分别来自4个9位的字段

## 7.7.3 页式虚拟存储器实例



### ◆ Opteron的48位虚拟地址的4级转换

## 7.7.3 页式虚拟存储器实例

- ◆ Opteron通常在TLB不命中时要遍历所有四级页表，故有3个位置可以进行保护限制的检查。
  - 仅遵从底层的PTE，而在其他级上只需确认有效位是有效的即可。
- ◆ Opteron使用4个TLB以减少地址转换时间
  - 两个用于访问指令，两个用于访问数据。
- ◆ 和多级Cache类似，Opteron通过采用两个更大的第二级TLB来减少TLB不命中。
  - 一个用于访问指令
  - 另一个用于访问数据

## 7.7.3 页式虚拟存储器实例

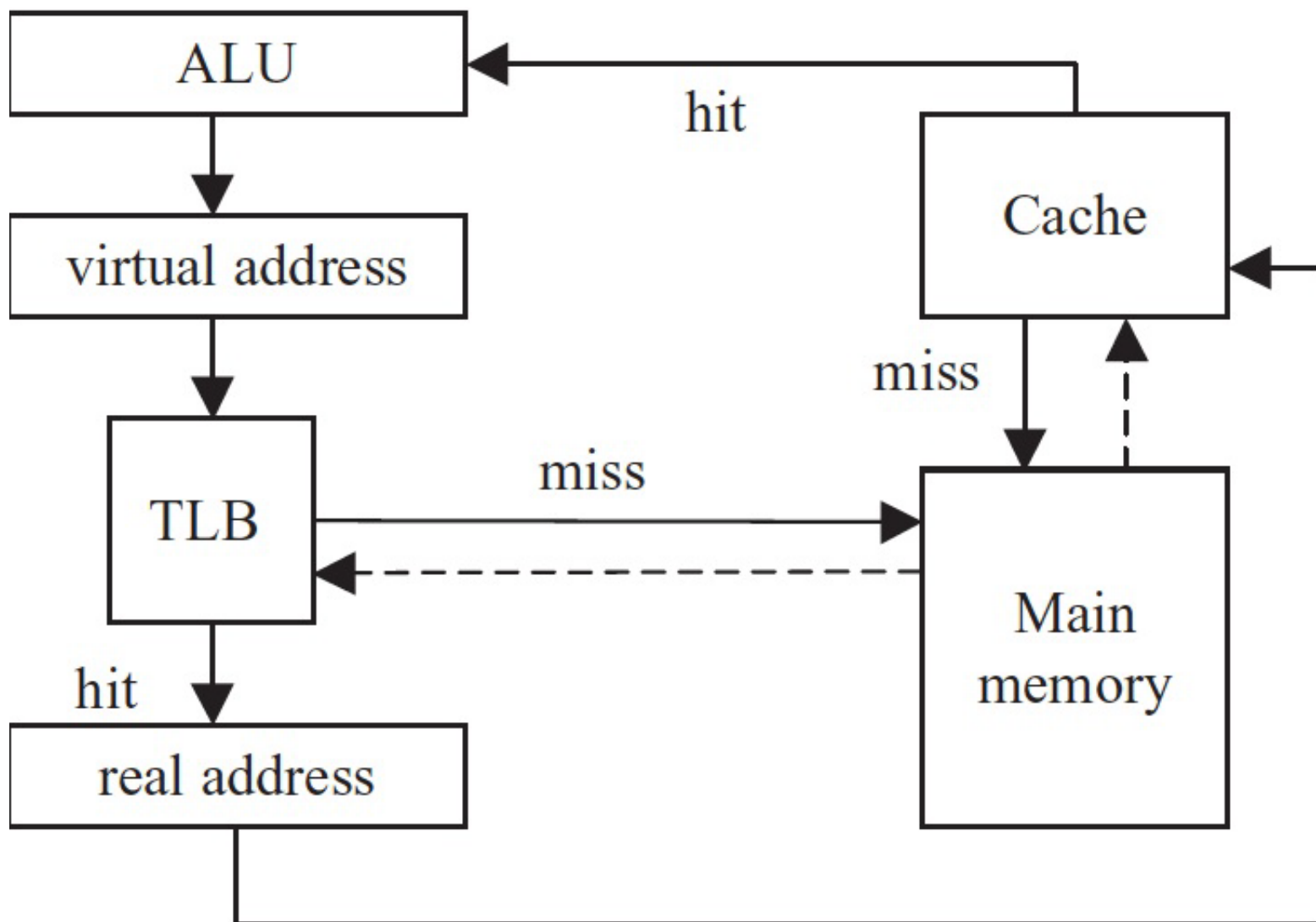
| 参数       | 描述                                              |
|----------|-------------------------------------------------|
| 块大小      | 1个PTE (8字节)                                     |
| L1命中时间   | 1个时钟周期                                          |
| L2命中时间   | 7个时钟周期                                          |
| L1 TLB大小 | 指令和数据TLB都是40个PTE，其中32个用于4KB大小的页面，8个用于2MB或4MB页面。 |
| L2 TLB大小 | 指令和数据TLB都是512个PTE，用于4KB页面                       |
| 块选择      | LRU                                             |
| L1映像规则   | 全相联                                             |
| L2映像规则   | 4路组相联                                           |

- ◆ Opteron中第一级和第二级指令、数据TLB的参数

# TLB特性

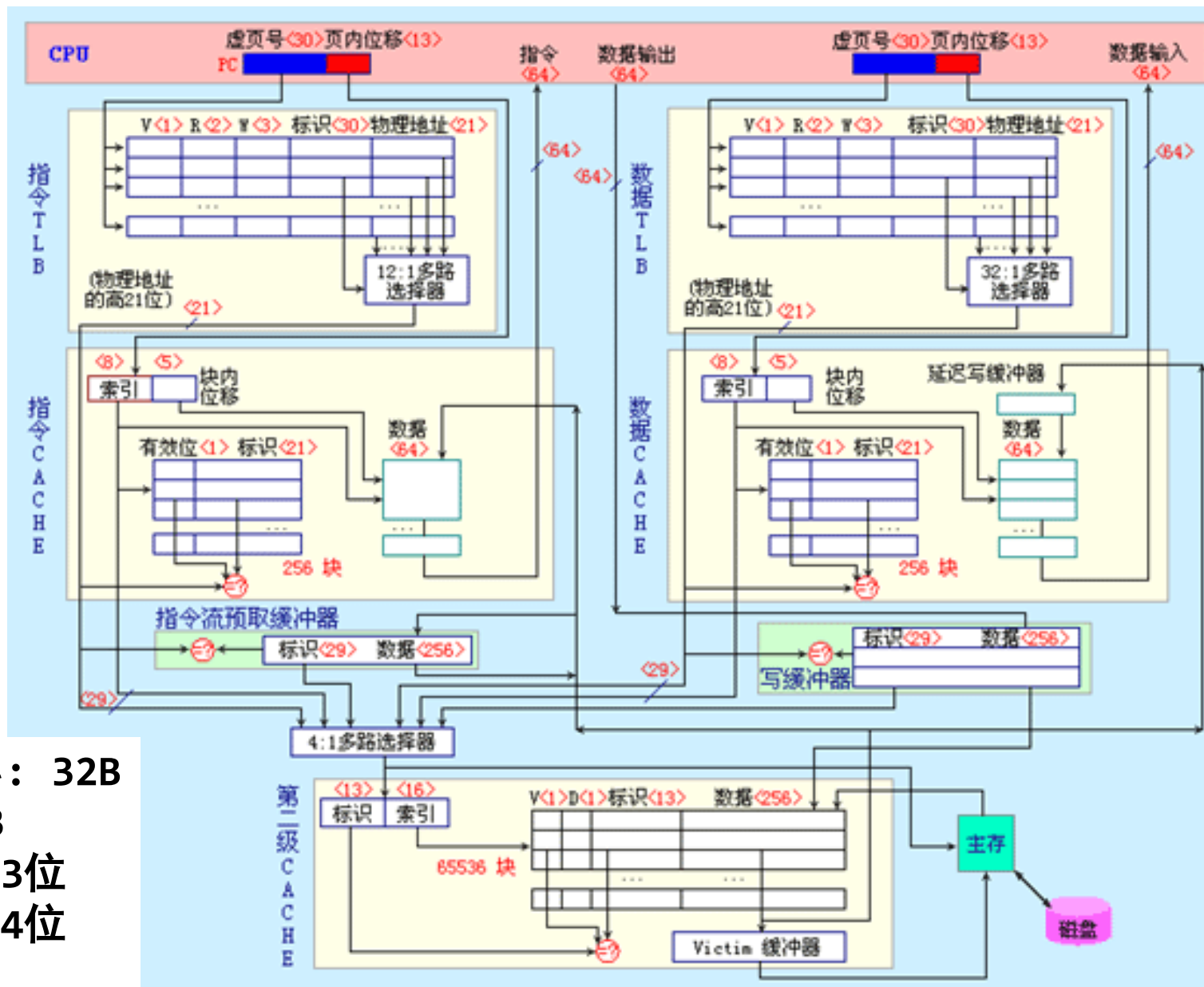
| Architecture | Page Size<br>(KB) | Number of Entries |             |
|--------------|-------------------|-------------------|-------------|
|              |                   | I-TLB             | D-TLB       |
| Alpha 21064  | 8                 | 8 (FA)            | 32 (FA)     |
| Alpha 21164  | 8                 | 48 (FA)           | 64 (FA)     |
| Alpha 21264  | 8                 | 64 (FA)           | 128 (FA)    |
| Pentium      | 4                 | 32 (4-way)        | 64 (4-way)  |
| Pentium II   | 4                 | 32 (4-way)        | 64 (4-way)  |
| Pentium III  | 4                 | 32 (4-way)        | 64 (4-way)  |
| Pentium 4    | 4                 | 64 (4-way)        | 128 (4-way) |
| Core Duo     | 4                 | 64 (FA)           | 64 (FA)     |

# Referencing Memory





# Alpha AXP 21064存储层次



cache块大小: 32B  
页大小: 8KB  
虚拟地址: 43位  
物理地址: 34位